Системи Електронного Урядування

Формальна модель та програмна архітектура функціонального верифікованого мовного забезпечення для побудови інфраструктурних процесінгових систем орієнтованих на державну модель управління процесами, зовнішнім аудитом, різними видами розподілених сховищ, телекомунікаційними та реєстровими фреймворками, інтернет-утворюючими сервісами зокрема та для автоматизації захищених автономних офісів і державних підприємств України у цілому.

Навчальний посібник курсу «Системи Електронного Урядування»

Максим Сохацький

18 лютого 2024, Київ, Україна

УДК 002
УДК 004.4, 004.6, 004.9

Присвячується всім державним
службовцям України

Система управління державними підприємствами ERP/1 визначає формальну специфікацію та її імплементацію для сучасних оптимізованих підприємств які вимагають сучасних засобів контролю операцій та цілістності даних.

Телекомунікаційна платформа Erlang/OTP від Ericsson успішно застосовується в індустрії мобільними операторами понад 30 років, а її віртуальна машина досі вважається однією з найкращих в галузі. Системи ERP на її базі також уже не один рік використовується у банківській сфері, процесінгу транзакцій, розподілених системах повідомлень, в IoT секторі. Ви можете переглянути демо модулі системи ERP/1 в нашому захищеному середовищі зі своїм центром випуску ECC X.509 сертифікатів. У цій книзі ви знайдете класичну авторську монографію на тему архітектури та імплементації такої системи, побудованої на міжнародних та державних стандартах України:

RFC: 7363, 6350, 4180, 5126, 5652, 8567, 9006, 9011, 9019, 9159, 9100, 8323, 7815, 7228, 6455, 8927, 8259, 4627, 7493, 7159, 4227, 3288, 6025, 5911, 4120, 4122, 7363, 6537, 6940, 7890, 2251-2256, 6960, 5280, 1034-1035, 4033-4035.

ISO: 19510, 19514, 42010, 18033, 14888, 10118, 10116, 15946, 29146, 9075, 27001, 19464, 20922, 21823, 27402, 30161, 30165, 20452, 42010, 19501, 19505, 8824-8825.

NIST: 800-162.

ДСТУ: 28147, 15946, 9798, 4145, 319-422, 319-122.

Постійне посилання твору: https://axiosis.top/erp/
Видавець: Державний науково-дослідний інститут МВС України

ISBN — 978-617-8027-23-0

Підготовлено до друку на Подолі, м. Київ.

© 2024 Максим Сохацький

Зміст

1 Вступ **5**

2 Національна програма інформатизації **7**
2.1 Загальні принципи 8
2.2 Модель 9
2.3 Структурне ядро 10
2.4 Протокол державної інституційної трансформації 11

3 Органи судової влади **13**

4 Органи виконавчої влади **15**
4.1 Міністерство освіти і науки України 15
4.2 Міністерство охорони здоров'я України 17
4.3 Міністерство внутрішніх справ України 18
4.4 Міністерство закордонних справ України 19
4.5 Міністерство оборони України 20
4.6 Міністерство юстиції України 28
4.7 Міністерство фінансів України 29
4.8 Міністерство економіки України 30
4.9 Міністерство енергетики України 31
4.10 Міністерство соціальної політики України 32
4.11 Міністерство регіональної політики України 33
4.12 Міністерство цифрової трансформації України 34
4.13 Міністерство захисту довкілля та природних ресурсів України 35

5 Специфікація та сертифікація **37**
5.1 Законодавча база 37
5.1.1 Загальні положення 37
5.1.2 Базова версія «ERP/1: Документи» 38
5.1.3 Розширення та додаткові модулі 39
5.2 Функціональні модулі та вимоги 40
5.2.1 Призначення та цілі впровадження 40
5.2.2 Ревізія поточних систем 40

6 Політики **41**
6.1 Політики . . . 41
6.1.1 Загальні принципи . . . 41
6.2 Технічні вимоги . . . 41
6.2.1 Перелік технічних вимог наведених в політиках . . . 41
6.2.2 Повний зміст технічних вимог . . . 41
6.3 Технічне завдання . . . 41
6.3.1 Зміст технічного завдання . . . 41
6.4 Посилання на стандарти ДСТУ . . . 41
6.5 Функціональні вимоги . . . 42
6.5.1 Вимоги до процесів системи . . . 42
6.5.2 Вимоги до викликів API . . . 42
6.5.3 Вимоги до інтерфейсу користувача . . . 42
6.6 Нефункціональні вимоги . . . 43
6.6.1 Вимоги до архітектури . . . 43
6.6.2 Вимоги до безпеки . . . 43
6.6.3 Вимоги до потужності та ємності . . . 43
6.6.4 Вимоги до системи логування . . . 43
6.6.5 Вимоги до контролю якості . . . 44
6.6.6 Вимоги до інтерфейсів . . . 44
6.6.7 Юридичні вимоги . . . 44

7 Структура ERP/1 **45**
7.1 Бюджетне планування і контроль видатків . . . 45
7.1.1 Вимоги до сутностей BUD . . . 45
7.1.2 Вимоги до процесів BUD . . . 45
7.2 Фінансовий і бухгалтерський облік . . . 46
7.2.1 Вимоги до сутностей FIN . . . 46
7.2.2 Вимоги до процесів FIN . . . 46
7.2.3 Вимоги до сховища та серіалізації FIN . . . 46
7.3 Кадрова система . . . 47
7.3.1 Вимоги до сутностей ACC . . . 47
7.3.2 Вимоги до процесів ACC . . . 47
7.3.3 Вимоги до сховища та серіалізації ACC . . . 47
7.4 Публічний документообіг і сервіс-деск . . . 48
7.4.1 Вимоги до сутностей CRM . . . 48
7.4.2 Вимоги до процесів CRM . . . 48
7.4.3 Вимоги до сховища та серіалізації CRM . . . 48
7.5 Шаблони документів . . . 49
7.5.1 Вимоги до сутностей . . . 49
7.5.2 Вимоги до процесів . . . 49
7.6 Генерація документів . . . 50
7.6.1 Вимоги до сутностей . . . 50
7.6.2 Вимоги до процесів . . . 50
7.7 Генерація бар- та штрих-кодів . . . 50
7.7.1 Вимоги до сутностей . . . 50

7.7.2 Вимоги до процесів 50
7.8 Розпізнавання Tesseract 51
7.8.1 Вимоги до сутностей 51
7.8.2 Вимоги до процесів 51
7.9 Система сканування 52
7.9.1 Вимоги до сутностей 52
7.9.2 Вимоги до процесів 52
7.10 Багатокористувацький редактор 53
7.10.1 Вимоги до сутностей 53
7.10.2 Вимоги до процесів 53
7.10.3 Призначення та цілі впровадження 53
7.11 Інфраструктура безпеки 54
7.11.1 Провайдери української криптографії 54
7.11.2 Вимоги до сутностей 54
7.11.3 Вимоги до процесів 54
7.11.4 Вимоги до серіалізації і сховища 54
7.12 Рольова модель і система доступів 55
7.12.1 Вимоги до рольової моделі ABAC 55
7.12.2 Вимоги до сутностей ABAC 55
7.12.3 Вимоги до процесів ABAC 55
7.12.4 Вимоги до сутностей IAM 56
7.12.5 Вимоги до процесів IAM 56
7.12.6 Вимоги до рольової моделі IAM 56
7.12.7 Вимоги до сховища 56
7.12.8 Вимоги до серіалізації і валідації 56
7.13 Оркестрація процесів 57
7.13.1 Вимоги до сутностей BPMN 57
7.13.2 Вимоги до процесів 57
7.13.3 Вимоги до сховища 57
7.14 Словникова підсистема 58
7.14.1 Загальні відомості 58
7.14.2 Вимоги до сутностей HL7 58
7.14.3 Вимоги до процесів 58
7.14.4 Вимоги до сховища 58
7.15 Мета сервіси та конструктор 58
7.15.1 Реєстр сутностей системи (API) 58
7.15.2 Біблітека валідації (JSON Schema) 58
7.15.3 Конструктор сутностей та форм 58
7.15.4 Конструктор бізнес процесів 58
7.15.5 Конструктор словників 58
7.15.6 Конструктор правил доступу 58
7.15.7 Конструктор шаблонів профілів користувачів 58

8 Державна система 59
8.1 Юридично-документальний рівень 59
8.2 Обліково-реєстровий рівень 60
8.3 Технологічний рівень зв'язності людей та пристроїв 61
8.3.1 Локальний 61
8.3.2 Крос-системний 61
8.4 Генерація, валідація і верифікація 62
8.5 Безпека інтернету та інфраструктури 62

9 Юридично-документальний рівень 63
9.1 Вступ 63
9.1.1 Види документообігів 63
9.1.2 Функціональні можливості 63
9.2 Модулі підприємства 64
9.3 Управління ресурсами 65
9.4 Архітектура врядувальних CRM-систем 66
9.4.1 Сторінки 66
9.4.2 Комболукап 66
9.4.3 Сервіси 66
9.4.4 СЕВ ОВВ 66
9.4.5 Шаблони 66
9.4.6 Дерева 66
9.4.7 Процеси 67
9.4.8 Елементи інтерфейсу 72
9.4.9 Редактори та додатки 77
9.4.10 Конструктор процесів 79
9.4.11 Мова програмування FormalTalk 79

10 Обліково-реєстраційний рівень 81
10.1 Вступ 81
10.1.1 Види реєстрів 81
10.1.2 Функціональні можливості 81
10.2 Модулі підприємства 82
10.3 Архітектура облікових CART систем 83
10.3.1 Облік метаінформації 83
10.3.2 Облік ABAC правил 83
10.3.3 Облік інфраструктури і само-моніторинг 83
10.3.4 Облік словників і класифікаторів 83
10.3.5 Адміністративний облік 83
10.3.6 Облік таксономії предметної облісті 83
10.3.7 Облік процесів предметної області 83

11 Технологічний рівень зв'язності людей та пристроїв 85
11.1 Вступ . . . 85
11.2 Виробничий процес . . . 86
11.3 Системи сховищ даних . . . 86
11.3.1 Реляційні бази даних . . . 86
11.3.2 Бази даних з єдиним простором ключів . . . 86
11.3.3 Шини комунікації та брокери повідомлень . . . 86
11.3.4 Розміщені в пам'яті гарячі дані . . . 86
11.4 Обчислювальні ресурси . . . 87
11.4.1 Накопичувальні ресурси . . . 88
11.5 Типові специфікації . . . 88
11.6 Середовище . . . 89
11.6.1 Бібліотеки . . . 90
11.6.2 Приклади . . . 90
11.7 Протоколи, схеми та мови їх опису . . . 91
11.7.1 Мова опису протоколів ASN.1 . . . 91
11.7.2 Мова опису протоколів SOAP/XSD/XML . . . 91
11.7.3 JSON валідатори draft-07 і JTD . . . 91
11.8 Формати передачі даних . . . 91
11.8.1 Бінарні формати ETF/BERT . . . 91
11.8.2 Бінарні формати DER/BER/PER . . . 91
11.8.3 Колоночний текстовий формат CSV/CSM . . . 91
11.8.4 Текстові формати JSON і XML . . . 91
11.9 Розробка Інтернет додатків . . . 92
11.9.1 Erlang та сучасний веб . . . 92
11.9.2 DSL vs Шаблони . . . 92
11.9.3 Історія . . . 93
11.9.4 Інтерфейс NITRO . . . 93
11.9.5 Сховище KVS . . . 93
11.9.6 Логіка BPMN . . . 93
11.9.7 Додатки MQTT та WebSocket . . . 93

12 Генерація, валідація і верифікація 95
12.1 Формальна верифікація та валідація . . . 95
12.2 Формальна специфікація . . . 96
12.2.1 Програмне забезпечення та логіка . . . 96
12.2.2 Математичні компоненти . . . 96
12.3 Формальні методи верифікації . . . 96
12.3.1 Алгебраїчні мови та System F . . . 97
12.4 Моделі процесів . . . 97
12.5 Формальні мови та середовища виконання . . . 97
12.5.1 Формальні інтерпретатори та екстракція . . . 97
12.6 Базова схема підприємства ERP/1 . . . 98

13 Інфраструктурний рівень безпеки інтернету 99
13.1 Електронний підпис і цифрова печатка 99
13.1.1 Приклад використання 101
13.2 Криптографічні інформаційні повідомлення 102
13.2.1 Головна функція 102
13.2.2 CMS-KARI-ECC 103
13.2.3 CMS-KEKRI-KEK 104
13.2.4 CMS-KTRI-RSA 105
13.2.5 KDF 105
13.2.6 AES-KW 106
13.2.7 AES-256 107
13.3 Імплементація CMP сервера у складі АЦСК 108
13.3.1 CSR 109
13.3.2 CMS 111
13.3.3 CA, АЦСК, ЦЗО та ОЗО 116
13.4 Безпечна система доменних імен DNSSEC 117
13.5 Захищений месенджер SYNRC CHAT 118
13.6 Система директорії підприємства LDAP 118
13.6.1 Вертикальні бази 119
13.6.2 Предметна область 120
13.6.3 TCP сервер 120
13.6.4 Висновки 128
13.7 ASN.1 Компілятор 129
13.7.1 Компілятори 129
13.7.2 Фірмові і стандартні 131
13.7.3 Бібліотеки Apple 132
13.7.4 Техніка компіляції 132
13.7.5 Висновки 134
13.8 Протокол розмежування доступу ABAC 134
13.8.1 PEP (Policy Enforcement Point) 135
13.8.2 PIP (Policy Information Point) 135
13.8.3 PAP (Policy Administration Point) 135
13.8.4 PDP (Policy Decision Point) 135

14 Апробація 137

15 Висновки 139

Передмова автора

Присвята

У колофоні зазначено, що цей посібник присвячено всім державним службовцям України. Це означає, що як невід'ємна частина соціоінформаційної системи державного устрою вони є основними кінцевими користувачами нашої платформи. Саме для них — майбутніх розробників, операторів, архітекторів, керівників та аналітиків — і написано цей твір. Їхня щоденна праця забезпечує стабільність та розвиток державних інституцій, а впровадження новітніх технологічних рішень має на меті полегшити й оптимізувати цей надзвичайно важливий процес.

Ідея виникнення та історія розвитку системи

Ідея розробки ERP/1 зародилася одразу після того, як я став на шлях підприємництва у 2005 році. Тоді я стояв перед вибором платформи для побудови моделі, котру планував впроваджувати, досліджуючи OCaml, Haskell і Erlang. Зрештою, моїм критеріям мотивації та технічним вимогам найкраще відповідала саме екосистема Erlang. Так розпочався мій професійний шлях: спершу був проєкт турецької соціальної мережі, згодом — робота у ПриватБанку, створення архітектури для месенджера NYNJA, навчання в аспірантурі з розробкою AXIO/1 і, нарешті, масштабні державні та міністерські системи.

Окремі частини системи, зокрема клієнтські компоненти для TWAIN-сканування та взаємодії з документами Word і Excel, було написано мовами C# та F#, оскільки мій найперший професійний досвід був тісно пов'язаний зі стеком .NET. Ось уже понад 18 років ERP/1 успішно обслуговує потреби замовників різного рівня, а фундаментальні ідеї платформи за цей час поширилися на екосистеми багатьох сучасних технологій, таких як LiveView, HTMX, Turbo, і навіть вплинули на такі старіші вебфреймворки, як Ocsigen, WebSharper, Lift та Nitrogen.

Сьогодні ERP/1 на базі Elixir повністю охоплює всі технологічні рівні державних інформаційних систем. При цьому вартість володіння платформою — що є одним із головних показників якості — зведено до абсолютного мінімуму. Це стало можливим завдяки тому, що всю систему написано єдиною мовою програмування і вона здатна повноцінно функціонувати на

одному вузлі в межах однієї віртуальної машини. Для порівняння: зібраний код кожного з базових модулів ERP/1 легко вміщується на одній дискеті формату 3.5” ємністю 2.88 МБ.

Ефективність подібних підходів та низьку вартість володіння ними давно довели відомі продуктові компанії: WhatsApp, ПриватБанк, RabbitMQ, Basho, AdRoll, розробники Crytek Warface. Яскравим прикладом є компанія WhatsApp: на момент продажу гіганту Facebook її інфраструктуру обслуговували лише 6 інженерів, а вартість компанії сягала 19 мільярдів доларів. Системи на базі ERP/1 побудовано з використанням тієї ж самої надійної виробничої технології.

Розкриття компонентів основної мотивації

Під час розробки системи ми послідовно застосовували філософію мінімалізму в галузі програмного забезпечення[1], про яку я вперше детально розповідав на одному з київських технологічних форумів ще 2013 року. Якщо викласти її суть стисло, вона зводиться до низки оптимізаційних критеріїв у вимірах часу, якості, людського та ресурсного капіталу:

1) зменшення часу виконання та розробки поряд зі значним подовженням життєвого циклу системи і підвищенням її ефективності. Основні метрики: Computation/Time, Weeks/Release, Feature/Hour, Hour/Lifetime; 2) подолання надмірної складності та зниження вартості розробки разом зі збільшенням надійності та концептуальної простоти. Основні метрики: Bugs/Code, Failures/Period, Cost/Support, Time/Fix; 3) усунення невизначеності на користь максимальної зрозумілості, відкритості та передбачуваності коду. Основні метрики: Committers/App, Issues/Feature, Msgs/Issue, Commits/User; 4) скорочення витрат на інфраструктурне обслуговування і додаткові ресурси задля значного збільшення обсягів обробки даних і обчислювальної спроможності. Основні метрики: Cost/Information, Machines/User, Data/User, Size/Features, Requests/Time, Information/User.

Якщо звести всі ці показники до єдиної всеосяжної максими, вона звучатиме так: ідеальний код — це той, котрий найлегше верифікувати (візуально, інструментально чи суворо математично), що, у свою чергу, неодмінно зумовлює його фундаментальну мінімалістичність. Наприклад, повна формальна верифікація рівня Г7 згідно з класифікатором ТЗІ (технічного захисту інформації) є недосяжною для таких систем, як типові SQL-бази даних. Адже практично неможливо математично довести всі без винятку властивості закритих пропрієтарних платформ на кшталт Oracle чи навіть потужних, але вкрай складних відкритих продуктів зразка PostgreSQL. Тому для підтвердження того, що система зберігає свої задані властивості, інженерам доводиться застосовувати безпосередній контроль типів і стовпців у таблицях, а всі функції згортки (рекурсори й фолди) власноруч писати в коді, наче це роблять курсорні генератори запитів у SQL. Тим не менш, до стандартного арсеналу Ericsson/OTP входить розподілена реляційна база даних Mnesia. І

[1] https://slides.com/maximsokhatsky/minimal/

хоча вона теж не позбавлена певної академічної складності, її дизайн елегантно обмежений простим інтерфейсом, що гармонійно вписується в парадигму типізації System F.

Коментарі та настанови для слухачів курсу

Цей навчальний курс є унікальним передусім тому, що предмет нашого дослідження — інформаційна система ERP/1 — відзначається продуманою формалізацією і чудово підходить як дидактичний матеріал для навчання фахівців. Разом із цим, ця ж система може похвалитися багатою історією успішних промислових впроваджень і безкомпромісною підтримкою сумісності.

Курс логічно поділено на 12 частин, кожна з яких послідовно розкриває певний етап розробки програмного забезпечення відповідно до стандартів ISO-9001. Матеріал охоплює повний цикл вітчизняного виробництва: від системного аналізу предметної області та роботи зі складною нормативно-юридичною документацією, через формування високорівневої архітектури, створення ескізного проєкту (з оглядкою на модель OSI та парадигму Закмана), — і аж до низькорівневих нюансів. До таких належать, зокрема, внутрішні механізми перевірки кваліфікованого електронного підпису (КЕП) та симетричного шифрування за алгоритмами ДСТУ-4145, реалізовані повністю автономно без залучення зовнішніх допоміжних криптографічних бібліотек. Завдяки високій щільності інформації та прикладній спрямованості цей посібник може стати вкрай корисним підручником і для фахівців напряму електронного урядування та цифрової трансформації державного апарату.

Матеріал створено з суворим урахуванням жорстких вимог, що висуваються до архітектури EDGE-офісів. Тут розглядається робота з системами, яким притаманні підвищені пороги допуску щодо розміру компільованої кодової бази, затримок мережевої взаємодії (latency), стійкості до відмов, рівня розподіленості, вартості подальшої підтримки, обчислювальної ефективності, багатства функціональності та експлуатаційної відповідності актуальним стандартам.

Подяки

Я хотів би висловити найщирішу вдячність усім своїм вчителям, наставникам та академічним колегам, чия праця часто залишається недооціненою, адже у глобальному масштабі інститут педагогіки переживає відчутну кризу поваги до професії викладача. Особлива подяка викладачам математики, інженерії та філософії, які свого часу прищепили мені фундаментальне прагнення до логічної чистоти та краси архітектурних рішень. Крім того, неабияк дякую всім контриб'юторам платформи ERP/1, які невтомно вдосконалювали цей продукт, а також своїм рідним і близьким за їхню багаторічну безмежну підтримку.

Розділ **1**

Вступ

Цей посібник докладно описує формальну модель та програмну архітектуру сучасного функціонального верифікованого мовного забезпечення. Його призначення — слугувати фундаментом для побудови складних інфраструктурних процесінгових систем, орієнтованих передусім на державну модель управління процесами. Платформа створена з прицілом на можливість прозорого проведення глибокого зовнішнього аудиту; вона сумісна з різноманітними видами розподілених сховищ, сучасними телекомунікаційними та реєстровими фреймворками, базовими інтернет-утворюючими сервісами, і головне — ідеально підлаштована під завдання автоматизації захищених автономних офісів та великих державних підприємств України у цілому.

Система управління державними підприємствами ERP/1, що презентується в цьому посібнику, є не тільки ідіоматичним зразком побудови сучасних інформаційних державних та комерційних екосистем. Вона насамперед визначає сувору формальну специфікацію та демонструє її надійну імплементацію (що налічує вже безліч реальних національних впроваджень) для потреб інноваційних, високооптимізованих підприємств, які вимагають максимально точних інструментів контролю операцій і беззаперечних гарантій цілісності даних.

Телекомунікаційна платформа Erlang/OTP, створена в надрах компанії Ericsson, успішно та безперебійно застосовується у світовій індустрії передовими мобільними операторами вже понад 30 років, а її віртуальна машина і досі заслужено вважається однією з найнадійніших у галузі. Інформаційні системи управління підприємствами на її базі вже багато років використовуються не лише в телекомі, але й в елітарній банківській сфері, високошвидкісному фінансовому процесінгу транзакцій, стійких розподілених брокерах повідомлень та в IoT-секторі. Ви в будь-який час можете переглянути роботу наших демо-модулів системи ERP/1, розгорнутих у спеціальному захищеному середовищі з власним автономним центром генерації та випуску ECC X.509 сертифікатів. На сторінках цієї книги ви знайдете вичерпний перелік ключових модулів системи та опис фундаментальних сутностей схеми.

В основі нашої концепції лежить ідея створення універсальної базової

платформи для розробки та стабільного функціонування різноманітних інформаційних реєстрів і розподілених баз (чи банків) даних будь-яких масштабів: від невеликих міжсистемних довідників і спеціалізованих класифікаторів до високонавантажених корпоративних, місцевих та масштабних державних інформаційних ресурсів. Переконаний, що цей посібник буде вкрай корисним для всіх, хто бажає глибоко зрозуміти принципи роботи, архітектуру та методики створення надійних інформаційних платформ, які сьогодні активно застосовуються як у вітчизняному державному, так і в комерційному секторах.

Список скорочень

ГСЦ — Головний сервісний центр МВС.

ДСНС — Державна служба України з надзвичайних ситуацій.

ЕСОЗ — Електронна система охорони здоров'я МОЗ.

ЄІС — Єдина інформаційна система МВС.

ЄРЗ — Єдиний реєстр зброї НПУ.

НГУ — Національна гвардія України.

СУСЗЦЗ — Система управління силами та засобами цивільного захисту ДСНС.

ФП МТРЗ — Функціональна підсистема матеріально-технічного та ресурсного забезпечення МВС.

Розділ **2**

Національна програма інформатизації

Мета Національної програми інформатизації (НПІ) — створення цифрового простору як проміжного етапу розвитку інформаційного суспільства. Це прозоре правове середовище, яке є надійно захищеним, базується на міжнародних стандартах, повноцінно забезпечує інформаційні потреби та реалізує права і свободи громадян на основі своєчасної, достовірної та вичерпної інформації, водночас істотно підвищуючи ефективність державного управління.

Національна програма інформатизації веде свою історію від Закону України № 74/98-ВР 1998 року, який містив 28 статей, і до поточного Закону 2024 року № 2807-IX, що складається з 15 статей. Головним чином НПІ закріплює протоколи запуску та термінації (завершення) програм, які охоплюють такі функції: експертизу, аналіз, формування, контроль, виконання та звітування щодо програм інформатизації як на державному (галузеві програми), так і на місцевому (самоврядування) рівнях. Вона також чітко визначає суб'єктів керування: генерального замовника — Міністерство цифрової трансформації, Керівника (відповідальну посадову особу), а також виконавців та спеціалізованих підрядників. Зокрема, НПІ є власником і розробником централізованої системи обліку таких програм. Крім того, Програма регламентує процеси розробки апаратного та програмного забезпечення згідно з міжнародними стандартами ISO/IEC 12207 та ISO 9001, а процеси супроводу та підтримки — згідно з ISO/IEC/IEEE 14764:2022.

Основне завдання Національної програми інформатизації — безперервна оптимізація підприємств у структурі органів виконавчої влади (ОВВ). Для досягнення цієї мети система державного управління повинна мати інструменти для визначення протоколів інституційної самотрансформації, а також забезпечувати узгоджену архітектуру програмних комплексів серед низки критично важливих міністерств.

2.1 Загальні принципи

На мета-рівні безперервний процес реформування органів виконавчої влади може бути описаний як такий, що керується трьома фундаментальними правилами. У структурі кожного міністерства мають бути наявні:

1. **Ін'єктивність управління юстиції** (як необхідний атрибут задля забезпечення внутрішнього аудиту та службових розслідувань);
2. **IT-департамент або IT-управління** (яке виступає як окрема юридична особа з власним кодом ЄДРПОУ, стає продуктовим оунером (власником) для цифрових рішень Міністерства);
3. **Управління трансформації** (підрозділ, що автоматизує та механізує організаційні процеси за допомогою IT-продуктів, розроблених IT-департаментом).

Далі вже інші профільні управління можуть додаватися залежно від специфічних функцій відповідного міністерства, але ці три стовпи, а також патронатна служба — є обов'язковими. Міністерство юстиції, судова система, Генеральна прокуратура, поліція та інші антикорупційні чи правоохоронні агенції, такі як НАЗК і НАБУ, повинні мати безпосередній або опосередкований доступ до першого компонента (управління юстиції). Цей доступ має суворо регулюватися відповідними правилами ABAC (Attribute-Based Access Control). Зі свого боку, Мінцифра, як координатор усього метапроцесу цифрової трансформації, отримує авторизований доступ до третього компонента (управління трансформації). Також Мінцифра координує роботу та безпосередньо взаємодіє з IT-департаментами кожного міністерства для забезпечення захищених каналів зв'язку до державних реєстрів, які підтримуються силами IT-управлінь конкретних міністерств. Архітектура та шини обміну даними між усіма системами електронного документообігу й реєстрами координуються IT-управлінням Мінцифри та платформою «Дія».

Завдання державної цифровізації та трансформації передбачає виконання таких стратегічних цілей:

- кожне міністерство матиме свій автономний і висококомпетентний IT-департамент, який операційно обслуговуватиме відповідні реєстри;
- міністерства матимуть власні управління трансформації, що працюватимуть на базі єдиного продукту, у якому вони зможуть зручно моделювати свої бізнес- та адміністративні процеси;
- Мінцифра, як головний координатор, розроблятиме й затверджуватиме єдині регламенти роботи для IT-управлінь та управлінь трансформації кожного міністерства.

Платформа «Дія» при цьому матиме не лише базову шину документообігу та коробочне рішення «Дія: Документообіг», але й виступатиме провайдером ліцензій для інших учасників ринку (наразі вже видано понад 30 таких ліцензій).

2.2 Модель

Для розуміння архітектури міжвідомчої взаємодії ми розглядаємо таку базову перелікову модель державних установ:

1. Кабінет Міністрів України;
2. Міністерство освіти і науки України (МОН);
3. Міністерство охорони здоров'я України (МОЗ);
4. Міністерство внутрішніх справ України (МВС);
5. Міністерство закордонних справ України (МЗС);
6. Міністерство оборони України (МО);
7. Міністерство юстиції України (Мін'юст);
8. Міністерство фінансів України (Мінфін);
9. Міністерство економіки України (Мінекономіки);
10. Міністерство енергетики України (Міненерго);
11. Міністерство соціальної політики України (Мінсоцполітики);
12. Міністерство розвитку громад і територій України (Мінрегіон);
13. Міністерство цифрової трансформації України (Мінцифра);
14. Міністерство захисту довкілля та природних ресурсів України (Мінекології).

2.3 Структурне ядро

Структурне ядро сучасного цифрового та оптимізованого органу виконавчої влади зазвичай включає три ключові стовпи:

1. Управління юстиції внутрішнього контролю;
2. Департамент інформаційних систем (або виробничо-промислове IT-управління);
3. Управління інституційної трансформації.

Оскільки соціоінформаційні системи повинні залишатися максимально автономними та мати прозорий керований життєвий цикл, технічне відображення організаційної структури неодмінно повинно перебувати під загальним контролем профільного міністерства. Найефективніше це реалізовувати у формі окремого державного підприємства. Такий підхід дає змогу надійно убезпечити ключові інтелектуальні ресурси й інфраструктуру від неконтрольованого ручного керування, а також обумовлений тим фактом, що IT-департаменти міністерств відповідають за цілісність державних реєстрів та сувору безпеку персональних даних громадян.

Для ефективного керування такою структурою в режимі реального часу ми пропонуємо створити окремий вид протоколу «ОВВ 2.0». Він стане наступним після розробки класичних НПА (нормативно-правових актів) розширенням до вже існуючого базового протоколу системи електронного документообігу, що функціонує згідно з постановою №55 Кабінету Міністрів України. Формальним втіленням цього підходу може стати, наприклад, протокол ДІТ (Державна інституційна трансформація).

У процесі функціонування як операційного документообігу (що генерується структурними підрозділами міністерств), так і інституційного (який породжується управліннями або агенціями державної цифрової трансформації) безальтернативно формуються криптографічні зліпки кваліфікованих електронних підписів (КЕП) відповідальних посадових осіб. Ці сліди дозволяють здійснювати надійний аудит системи — вони перевіряються та досліджуються як внутрішніми контролюючими органами (відділами аудиту та внутрішніх розслідувань), так і безпосередньо координуючим наглядовим органом — Міністерством юстиції України.

2.4 Протокол державної інституційної трансформації

Цей спеціалізований інформаційний протокол визначає такі базові операції над організаційною структурою відомства та її політиками:

1. Створення та ліквідація структурних підрозділів;
2. Розробка, погодження та модифікація установчих та конституційних документів;
3. Проєктування та розробка технічних проєктів для структурних підрозділів та їхніх внутрішніх систем;
4. Збір вимог, вибір та подальше впровадження відповідних інформаційних систем;
5. Запуск та супровід операційної (безперервної виробничої) діяльності структурних підрозділів.

Розділ **3**

Органи судової влади

Розділ **4**

Органи виконавчої влади

Цей розділ описує структуру органів виконавчої влади (ОВВ), які виступають ключовими об'єктами інформатизації.

4.1 Міністерство освіти і науки України

Цей підрозділ є статтею-дослідженням таксономії структури Міністерства освіти і науки як з погляду інформаційної автоматизованої системи, так і в розрізі соціальної структури державного управління. Як приклад, тут наведено цільову предметну модель, що є оптимізованою модифікацією чинної ієрархічної системи Міністерства освіти і науки України.

Структурні підрозділи

1. Патронатна служба
2. Управління початкової школи
3. Управління середньої школи
4. Управління вищої школи
5. Управління юстиції
    - Департамент експертизи і сертифікації
    - Інститут інтелектуальної власності
    - Департамент кадрового забезпечення
    - Департамент аудиту та внутрішніх розслідувань
    - Департамент архівної справи (SCAN)
    - Технічний департамент (CA)
    - Департамент соціального і гуманітарного забезпечення
    - Відділ кадрів (ACC)
    - Юридичний департамент
    - Департамент міжнародного співробітництва

6. Управління науково-дослідними інститутами (агенція)
7. Національна академія наук (агенція)
    - Інститут формальної математики
        - Ректорат формальної філософії
        - Ректорат чистої математики
        - Ректорат прикладної математики
        - Ректорат мовного забезпечення
        - Ректорат теоретичної інформатики
    - Інститут формальної літератури
    - Інститут музики, кіно й образотворчого мистецтва
        - Національна консерваторія
        - Національна академія мистецтв
        - Національна кінематика
    - Інститут фізики і матеріалознавства
    - Інститут геології і геохімії
    - Інститут хімії і біології
    - Інститут соціальних і гуманітарних наук
        - Ректорат філософії
        - Ректорат археології
        - Ректорат національної історії
        - Ректорат права
        - Національна бібліотека
8. Управління політиками і структурними підрозділами (агенція)
    - Департамент комунікації (відділ кадрів)
    - Департамент контролю виконання показників (CRM)
    - Департамент планування переходу (аналіз процесів BPMN)
    - Департамент трансформації (широкий спектр спеціалізацій)

4.2 Міністерство охорони здоров'я України

Таксономія Міністерства охорони здоров'я базується на послідовній інтеграції існуючих клінічних баз, системи громадського здоров'я та профільних дослідницьких інституцій у єдину організаційно-інформаційну мережу. Ця модель зорієнтована на оперативне забезпечення сталого розвитку галузі, медичної освіти й науки.

Структурні підрозділи

1. Патронатна служба
2. Управління медичної освіти та науки
   - Медичні університети й академії
   - Науково-дослідні інститути (НДІ) системи МОЗ
3. Управління державного санітарного контролю та громадського здоров'я
   - Центр громадського здоров'я (ЦГЗ)
   - Департамент епідеміологічного нагляду
4. Департамент фармаконагляду і медичного ліцензування
5. Управління юстиції
   - Департамент медичної експертизи
   - Департамент внутрішнього аудиту
   - Департамент кадрового забезпечення та юридичної підтримки
6. Управління політиками та структурними підрозділами (агенція)
   - Департамент цифрової трансформації (взаємодія з eHealth та ЕСОЗ)
   - Департамент стратегічного планування

4.3 Міністерство внутрішніх справ України

Архітектура МВС чітко розподіляє управлінські повноваження між своїм центральним апаратом та підзвітними йому центральними органами виконавчої влади (ЦОВВ). Це забезпечує ефективне реагування, незалежний внутрішній контроль і надання сервісних функцій населенню без дублювання повноважень.

Структурні підрозділи

1. Патронатна служба
2. Головний департамент формування політик безпеки
3. Управління координації ЦОВВ
   - Національна поліція України (НПУ)
   - Державна служба України з надзвичайних ситуацій (ДСНС)
   - Державна прикордонна служба України (ДПСУ)
   - Національна гвардія України (НГУ)
   - Державна міграційна служба України (ДМС)
4. Головний сервісний центр (ГСЦ)
5. Управління юстиції
   - Департамент внутрішньої безпеки
   - Департамент аудиту та службових розслідувань
6. Управління політиками та структурними підрозділами (агенція трансформації)

4.4 Міністерство закордонних справ України

Структурна модель МЗС відображає вимоги сучасного протоколу та міжнародного права. Вона оптимізована для швидкого обміну захищеною аналітичною інформацією з дипломатичними представництвами та для підтримки зовнішньополітичного консолідованого курсу.

Структурні підрозділи

1. Патронатна служба
2. Управління дипломатичної освіти та аналітики
   - Дипломатична академія
   - Інститут актуальних міжнародних відносин
3. Територіальні управління
   - Департаменти європейської, євроатлантичної, азійської та американської політики
4. Департамент міжнародних організацій (ООН, НАТО тощо)
5. Управління юстиції
   - Департамент міжнародного права
   - Консульська служба
   - Департамент кадрового аудиту та безпеки
6. Управління політиками та інституційного розвитку

4.5 Міністерство оборони України

Цей підрозділ є дослідженням таксономії структури Міністерства оборони України з погляду як інформаційної автоматизованої системи, так і соціальної структури в розрізі державного управління. Як приклад у статті розглядається конкретна структурна модель, що є адаптованою модифікацією чинної ієрархічної системи відомства. Ця структура підсилена повним спектром інституцій, необхідних для організації безперервного науково-освітнього та технологічно-виробничого циклу функціонування оборонного державного органу виконавчої влади.

Як модель гранулярності використано типову українську державну класифікацію (міністерство, управління, департамент, відділ, сектор). Оскільки ця робота зосереджена насамперед на логіці архітектури процесів, тут значною мірою надається перевага формальним моделям, які потребують мінімальних зусиль для своєї верифікації, моделювання та прогнозування. З огляду на те, що формалізація процесів безпосередньо стосується програмного забезпечення, у пригоді стають міжнародні стандарти телекомунікаційних протоколів. Їх сертифікація своєю чергою залучає університети, науково-дослідні та науково-виробничі інститути. Науково-виробничі інститути тут розглядаються у широкому сенсі — як такі, що можуть функціонувати в симбіозі з комерційними об'єднаннями, міжнародними фундаціями тощо. Для адекватної підтримки автономної діяльності всі ці інституції повинні входити до єдиного арсеналу функціональних можливостей міністерства. Насамперед це стосується повноцінного університету четвертого рівня акредитації, навчальні програми якого мають перегукуватися (наприклад) з пропозиціями ключових кафедр Інституту математики НАНУ (див. Додаток 1).

Фізичне розміщення базової інформаційної інфраструктури розгалуженої національної мережі міністерства та його підрозділів також передбачає автономне забезпечення (модель класу EDGE-офіс) із власними ресурсами охолодження, водо-електро-постачання та засобами посиленої охорони. Інформаційна політика формального моделювання допускає використання сучасних інструментальних мов розвитку, здатних підтримувати сувору машинну верифікацію на рівні комплексів ТЗІ Г7 (з можливостями повної математичної доводжуваності), покладаючись здебільшого на надійні телекомунікаційні протоколи та міжнародні стандарти серії ISO (див. Додаток 5).

Мотивація

Основна мотивація цієї роботи полягає у висвітленні таксономії Міністерства оборони України через призму структурної оптимізації, максимальної інституційної автономності та життєстійкості (sustainability), а також здатності до самовідтворення й ефективної підтримки операційного життєвого циклу міністерства в умовах динамічних викликів.

Структурні підрозділи

1. Патронатна служба
2. Управління освіти і науки (агенція)
   - Університет четвертого рівня акредитації (див. Додаток 1)
   - Науково-дослідні інститути (НДІ)
   - Науково-виробничі інститути
3. Управління медицини (агенція)
   - Клінічні наукові дослідження та лабораторії, НДІ профілактичної та військової медицини (MED)
   - Заклади реабілітації («Пуща-Водиця», «Трускавецький», «Хмільник»)
   - Клінічні та мобільні (4) лікарні, військові госпіталі (14)
   - Медичні служби (5 родів військ), центри тактичної медицини, Командування Медичних сил
   - Інститут медицини (на базі ЗДМУ, ХНМУ, ЛНМУ, ТНМУ)
4. Виробничо-промислове управління (агенція)
   - Конструкторські бюро (КБ), концерн «Укроборонпром», ДАХК «Артем»
   - Департамент економічного моделювання та планування
   - Департамент ресурсного забезпечення (SCM, TMS, WMS)
   - Департамент бюджетування і закупівель (FIN)
   - Департамент будівництва, архітектури та масштабування ліній виробництва
   - Телекомунікаційний департамент інформаційних систем
     - Відділ систем урядування
     - Відділ облікових систем
     - Відділ телекомунікаційних систем
     - Відділ безпекових протоколів інтернет-зв’язку
   - Департамент авіації, авіоніки, аеронавтики та безпілотних літальних апаратів (БПЛА) і їхніх систем
     - Відділ авіації
     - Відділ авіоніки
     - Відділ аеронавтики
     - Відділ безпілотних систем
   - Департамент машинобудування (тестування і проектування місцевості, SolidWorks)
   - Департамент суднобудування та військово-морської техніки

5. Управління юстиції
    - Відділ експертиз і сертифікації (ISO/IETF)
    - Департамент архівної справи (SCAN)
    - Департамент аудиту та внутрішніх розслідувань
    - Технічний департамент (CA)
    - Департамент соціального і гуманітарного забезпечення
    - Відділ кадрів (ACC)
    - Юридичний департамент
    - Департамент міжнародного співробітництва
6. Управління розвідки
7. Головне управління позиційною політикою та стратегією
    - Мобілізаційний департамент
    - Департамент військово-навчальних програм
    - Департамент координації сил та засобів оборони (CRM, див. Додаток 2)
        - Командування Сухопутних військ
        - Командування Повітряних сил
        - Командування Військово-Морських сил
        - Командування спеціальних та допоміжних сил (ССО, Морська піхота, РЕБ, РХБЗ, ТРО)
        - Командування кібербезпеки та ДШВ
    - Департамент контролю й оперативного управління DFR (див. Додаток 4)
    - Економічний департамент
        - Відділ матеріально-ресурсного забезпечення (WMS)
        - Відділ бюджетування і закупівель
8. Управління політиками та структурними підрозділами (агенція трансформації)
    - Департамент комунікації та кадрової політики
    - Департамент контролю виконання ключових показників (CRM)
    - Департамент планування структурного переходу (аналіз процесів BPMN)
    - Департамент цифрової та інституційної трансформації

Принципи

Одним із головних принципів, закладених у фундамент Міністерства оборони (МО), є створення повноцінної екосистеми вищої профільної освіти, яка повинна функціонувати відповідно до найвищих міжнародних стандартів. Для задоволення фахових потреб та розвитку працівників і військовослужбовців на всіх рівнях цієї таксономії в основу організації роботи покладено безперервне функціонування відомчого університету четвертого рівня акредитації, чиї спеціальності мають повністю покривати наукові та технологічні запити самого міністерства.

Іншим універсальним і життєво необхідним принципом є принцип чіткого розподілу влади, з якого випливає наявність трьох ключових макрокорпусів МО.

1. **Політичний корпус** (до якого належать головне управління позиційною політикою, управління політиками та переходом, а також патронатна служба). Він передбачає виокремлення спеціальної агенції трансформації, яка ініціює створення та виконує структурну апробацію інституцій міністерства. До цього корпусу також входить головне управління, що визначає стратегію застосування Збройних Сил України та відповідних засобів оборони.

2. **Виконавчий корпус** (управління освіти і науки, медичне управління та надважливе виробничо-промислове управління). Цей корпус гарантує сталий розвиток високоінтелектуальних та ресурсоємних структурних підрозділів і виводить їх під автономний контроль профільних агенцій, що дозволяє їм гнучкіше взаємодіяти із зовнішніми технологічними й медичними екосистемами.

3. **Судовий і правоохоронний корпус** (управління юстиції, галузевий трибунал). Це окремий, функціонально незалежний структурний блок, який реалізує процеси внутрішнього аудиту, комплаєнсу та службових розслідувань всередині соціоінформаційних систем оборонного відомства.

Самоорганізація

Для систематизації процесу організаційного будівництва варто виділити ключові структурні етапи та засади:

- Розподіл влади та мінімізація адміністративної таксономії;
- Нормалізація формальних процесів;
- Незалежний аудит міністерства та процес інституційної трансформації;
- Проєктування архітектури структурних підрозділів;
- Апробація результатів впровадження і циклічність процесу оновлення.

Розподіл влади традиційно залишається головним принципом ефективного державного управління. Контролюючі та ревізійні органи повинні спеціалізуватися винятково на перевірках і розслідуваннях, їхня структура має бути операційно виокремленою. Розвиток політик, регуляція бізнес-процесів управління та збір реквізитної інформації мають перебувати в компетенції політичного блоку, тоді як виконання завдань (наука, освіта, матеріальне виробництво та забезпечення) здійснюють суто виконавчі гілки відомства.

Вимоги до документування процесів і збереження даних повинні формуватися на найвищому технічному рівні. Це означає використання еквіваріантної семантики, спрощення архітектури процесів до нормальних форм, максимальну мінімізацію заплутаних горизонтальних зв'язків. Усі вони мають цілковито відповідати актуальним вимогам постанови КМУ №55. Водночас має бути організовано прозорий процес історіографії та цифрового аудиту діловодства й виробничих ланцюгів відповідно до суворого стандарту управління якістю ISO 19510 (що є важливим для сумісності з аудитом за стандартами НАТО). Запуск автоматизованого діловодства передбачається впроваджувати поступово і гранулярно: починаючи від ключових політико-формуючих центрів та їхніх найменш ресурсоємних проєктів, і далі розширюючи охоплення на всі інші департаменти та управління згідно з затвердженою магістральною стратегією.

4.5.0.1 Патронатна служба

Забезпечує сприяння реалізації стратегічних та політичних цілей Міністра оборони, безпосереднє персональне консультування, а також організаційне, інформаційно-публічне й експертно-аналітичне забезпечення його діяльності.

4.5.0.2 Управління освіти й науки (агенція)

Сучасний науковий потенціал, крім безпосередньо освітнього процесу, активно виділяє як науково-дослідний, так і науково-виробничий напрями.

Сфери досліджень повністю диктуються актуальними потребами бойових або організаційних департаментів Міністерства оборони. Ці високотехнологічні компанії й лабораторії повинні знаходитися, як і підпорядкований університет, в єдиній орбіті стратегічного впливу МОУ.

4.5.0.3 Виробничо-промислове управління (агенція)

Концепція передбачає повне дзеркальне дублювання сфер безпосереднього виробництва засобів оборони відповідно до профілю керуючих департаментів, що формують політики ресурсного забезпечення. Оскільки військова промисловість консолідує базові ресурси оборонної індустрії, є методично доцільним тримати всю картину планування та виробництва під єдиною ієрархічною парасолькою міністерства. Наявні сьогодні виробничі хаби й об'єднання, такі як АТ «УОП» (колишній «Укроборонпром») та ДАХК «Артем», пропонується інтегрувати в систему як потужні зовнішні конструкторські бюро в єдиному переліку з профільними приватними комерційними підприємствами, з якими планується вибудовувати гнучку проєктну співпрацю.

4.5.0.4 Управління юстиції

Управління юстиції як незалежна гілка бере на себе функцію неформального обліку сил (розширений відділ кадрів та прав) і юридичного фіксування протокольних дій усередині системи. Далі вони систематично аналізуються антикорупційним відділом та департаментом внутрішніх розслідувань і аудиту. Саме тут зосереджено ключовий штат юридичної експертизи, що обслуговує всю масштабну ієрархію міністерства, а також координує правову взаємодію із міжнародними зовнішніми партнерами, такими як блок НАТО. Також під парасолькою цього управління доцільно тримати технічний департамент як автономний центр емісії криптографічних сертифікатів і ключів управління для всього штату працівників.

4.5.0.5 Головне управління позиційною політикою та стратегією

Це сучасне осмислення спрощених і водночас глобально стратегічних функцій Генерального штабу, який безпосередньо взаємодіє з наявними резервами Сил та засобів оборони. Загальна тактична і стратегічна консоль управління підпорядковує всю логіку збройної місії веденню магістральної позиційної політики: від ефективного розгортання сил та засобів до просунутого дата-орієнтованого прогнозування еволюції подій. Відповідно, правильна позиційна політика завжди спирається на максимально достовірний локальний і глобальний облік наявних ресурсів.

4.5.0.6 Управління політиками та структурними підрозділами (агенція)

Це формальний мозковий центр адміністративного врядування та інституційного переходу від застарілої структури до будь-якої наперед заданої оп-

тимальної моделі (наприклад, тієї, що описана в цьому документі). Управління займається первинним аудитом чинних бізнес-процесів (їх юридичним і процедурним розтином) та проєктуванням їхньої поетапної трансформації в нові урядові й облікові цифрові системи. Свою діяльність структура тісно координує з телекомунікаційним департаментом, здійснюючи проєктний менеджмент: контролює процес впровадження, проводить апробацію систем і подальше їх калібрування.

Процес системної трансформації раціонально поділено на такі категорії (зворотно пропорційно до швидкоплинності впровадження):

1. Оцифровування (діджиталізація) базових існуючих процесів без кардинальної модифікації їхньої логіки (наприклад, класичне діловодство).

2. Створення архітектури нових функціональних процесів та надійного технологічного забезпечення для «вакантних» або новоутворених інноваційних екосистем, таких як виробничо-промислове і технологічне управління.

3. Планування та стратегічний розвиток нових управлінь (це найдовший за часом процес, що охоплює розробку навчальних програм, побудову наукомістких продуктів та їх всебічну підтримку).

Кожна фаза цього структурного метапроцесу вимагає побудови чіткого життєвого циклу проєкту, включно з продуктом, який лежить у його технологічній основі. До цього циклу обов'язково входить повний комплект супровідної документації: технічне завдання, ескізний проєкт, робочий технічний проєкт і детальна технологічна документація. Управління політиками природно розподіляє сфери компетенції з телекомунікаційним підрозділом і підтримує єдині стандарти документування процесів.

Ключові процеси й завдання управління переходом можна узагальнити так: 1) архівна систематизація процесного виробництва; 2) розробка гнучких протоколів запуску нових структурних підрозділів; 3) розробка сучасних процесів захищеного документообігу; 4) розробка процесів інтегрованого управління та субординаційної координації; 5) створення стандартів технологічних процесів на основі європейських стандартів сімейства ISO 9001.

Висновки та результати

У статті представлена розширена і запропонована до впровадження таксономія оптимізованого і нормалізованого міністерства. Наша модель створена з урахуванням передового міжнародного та корпоративного досвіду організації структур, підкреслює чіткі принципи прозорого розподілу влади та організації як глобального (стратегічного), так і локального (операційного) контекстів для відомств. Цілі стають легко досяжними, адже організація функцій здійснюється в найпряміший та найефективніший спосіб, різ-

ко мінімізуючи складність та кількість горизонтальних протоколів взаємодії всередині самої системи.

У додатках до статті наведено таксономію навчальних та наукових програм для залученого університету, а також детальну ієрархію телекомунікаційного департаменту, серед основних функцій якого є розробка та практичне впровадження новітніх ІТ-систем ЗСУ і МО. У процесі цього первинного концептуального моделювання ми спробували охопити всі ключові органи системи аж до найвищого рівня гранулярності — департаментів, відділів та секторів, вибудовуючи первинні індекси і їхні внутрішні протоколи взаємодії.

Бібліографія

- ДСТУ 2732-94 Діловодство й архівна справа. Терміни та визначення.
- Наказ МОУ від 26.05.2014 №333 «Інструкція з організації обліку особового складу».
- Наказ МВС від 21.11.2017 №608 «Порядок проведення службового розслідування».
- Постанова КМУ від 26.07.2018 №370 «Про затвердження Інструкції з діловодства».
- Наказ МОУ від 07.04.2017 №124 «Інструкція з діловодства в Міністерстві оборони України».
- Постанова КМУ від 07.10.2015 №393 «Положення про юридичну службу».
- Наказ МОУ від 29.11.2018 №604 «Інструкція з надання доповідей і донесень про події, кримінальні правопорушення, військові адміністративні правопорушення та адміністративні правопорушення, пов’язані з корупцією, порушення військової дисципліни...».

4.6 Міністерство юстиції України

Мін'юст у цій архітектурі відіграє роль не лише галузевого регулятора правовідносин, але й центрального верифікаційного вузла для інфраструктурних процесів інших ОВВ. Міністерство виступає головним арбітром у реєстраційних діях та забезпечує дотримання законності на технічному та організаційному рівнях.

Структурні підрозділи

1. Патронатна служба
2. Департамент державної реєстрації та нотаріату
3. Департамент виконання кримінальних покарань
4. Департамент судової роботи та експертного забезпечення
5. Власний орган юстиції та внутрішнього контролю (Генеральна інспекція)
    - Департамент внутрішніх розслідувань і комплаєнсу
    - Департамент аудиту реєстрів
6. Управління цифровізації та структурних підрозділів
    - Адміністрування державних реєстрів юстиції
    - Забезпечення кіберстандарту (спільно з ЦОВВ)

4.7 Міністерство фінансів України

Організаційна модель Міністерства фінансів відповідає за макрофінансову стабільність, податкову політику та бюджетування всіх державних ресурсів. Для ефективного управління вона суворо розмежовує блок аналітики, блок адміністрування надходжень та блок контролю.

Структурні підрозділи

1. Патронатна служба
2. Департамент державного бюджету та фінансового планування
3. Департамент податкової та митної політики
4. Департамент управління державним боргом
5. Управління юстиції
   - Департамент фінансового аудиту
   - Служба державного фінансового моніторингу
   - Юридичний департамент
6. Управління політиками та структурними підрозділами (інтеграційне планування з Казначейством)

4.8 Міністерство економіки України

Архітектура цього міністерства сконцентрована на прогнозуванні макропоказників, регулюванні промислового та аграрного секторів, а також на питаннях захисту вільної конкуренції та залучення інвестицій.

Структурні підрозділи

1. Патронатна служба
2. Департамент макроекономічного прогнозування та аналітики
3. Департамент розвитку реального сектору економіки
4. Департамент регуляції зовнішньоекономічної діяльності
5. Управління юстиції
   - Департамент антимонопольного контролю та нагляду
   - Департамент внутрішнього аудиту
6. Управління політиками та структурними підрозділами
   - Департамент методології закупівель (Prozorro)

4.9 Міністерство енергетики України

В основу структурної моделі Міністерства енергетики покладено розподіл за ключовими сировинними та енергогенеруючими напрямами, з виділенням сильного блоку екологічної безпеки та стратегічної синхронізації з європейськими енергомережами.

Структурні підрозділи

1. Патронатна служба
2. Департамент ядерної енергетики та атомно-промислового комплексу
3. Департамент нафтогазового комплексу
4. Департамент електроенергетичного комплексу та зеленої енергії
5. Управління юстиції
   - Інспекція з енергетичного нагляду
   - Департамент внутрішнього аудиту та комплаєнсу
6. Управління політиками та структурними підрозділами (агенція переходу до ENTSO-E)

4.10 Міністерство соціальної політики України

Структура цього органу сфокусована на соціальному захисті, регулюванні ринку праці та пенсійному забезпеченні. Вона спирається на Єдину інформаційну систему соціальної сфери (ЄІССС) як інструмент діджиталізації соціальних послуг.

Структурні підрозділи

1. Патронатна служба
2. Департамент пенсійного забезпечення та соціального страхування
3. Департамент захисту прав дітей та піклування
4. Департамент праці та зайнятості
5. Управління юстиції (внутрішній контроль, координація інспекції праці)
6. Управління політиками та структурними підрозділами (Трансформація ЄІССС)

4.11 Міністерство регіональної політики України

Таксономія спрямована на ефективне управління магістральними інфраструктурними проєктами, процесами післявоєнного відновлення та завершенням комплексної децентралізації.

Структурні підрозділи

1. Патронатна служба
2. Департамент інфраструктури та житлово-комунального господарства
3. Департамент регіонального розвитку та децентралізації
4. Департамент містобудування, урбаністики та архітектури
5. Управління юстиції
   - Аудит будівельних і транспортних інспекцій
   - Внутрішня безпека
6. Управління політиками та інституційного відновлення (ЄДЕССБ, логістичні системи)

4.12 Міністерство цифрової трансформації України

Мінцифра виступає головним ідеолог-архітектором всієї урядової мета-системи електронного врядування. Структура побудована як гнучка IT-агенція, поділена за продуктовими напрямами (додаток Дія, ШСД) та проєктними екосистемами (Дія.City).

Структурні підрозділи

1. Патронатна служба
2. Департамент розвитку електронних послуг (екосистема «Дія»)
3. Департамент розвитку IT-індустрії (Спеціальний режим «Дія.City»)
4. Департамент розвитку цифрової інфраструктури (широкосмуговий доступ)
5. Управління юстиції
   - Департамент захисту персональних даних та криптографічного супроводу
   - Департамент внутрішнього IT-аудиту
6. Департамент системної інституційної трансформації ОВВ (методична координація IT-директорів усіх міністерств)

4.13 Міністерство захисту довкілля та природних ресурсів України

Архітектура цього відомства забезпечує контроль за раціональним використанням природних ресурсів і впровадженням політик сталого розвитку згідно з екологічними директивами ЄС.

Структурні підрозділи

1. Патронатна служба
2. Департамент управління відходами та екологічної безпеки
3. Департамент лісового та водного господарства
4. Департамент природно-заповідного фонду та надрокористування
5. Управління юстиції
    - Державна екологічна інспекція (наглядовий орган)
    - Департамент внутрішнього аудиту
6. Управління політиками та структурними підрозділами (впровадження екологічних реєстрів)

Розділ **5**

Специфікація та сертифікація

5.1 Законодавча база

5.1.1 Загальні положення

Базова версія «ERP/1: Документи» керується наступними загальними положеннями які виражені законами України:

— 2657-XII, Про інформацію[1],
— 7498-ВР, Про Національну програму інформатизації[2],
— 39396-ВР, Про звернення громадян[3],
— 2939-VI, Про доступ до публічної інформації[4]
— 2155-VIII, Про електронні довірчі послуги[5],
— 851-IV, Про електронні документи та електронний документообіг[6],

та розпорядженнями і постановами Кабінету Міністів України:

— 386-2013-р, Розпорядження КМУ #3860-Р [7],
— 373-2006-п, Постанова КМУ #373 [8].

[1] https://zakon.rada.gov.ua/laws/show/2657-XII
[2] https://zakon.rada.gov.ua/laws/show/74/98-вр
[3] https://zakon.rada.gov.ua/laws/show/393/96-вр
[4] https://zakon.rada.gov.ua/laws/show/2939-17
[5] https://zakon.rada.gov.ua/laws/show/2155-19
[6] https://zakon.rada.gov.ua/laws/show/851-15
[7] https://zakon.rada.gov.ua/laws/show/386-2013-р
[8] https://zakon.rada.gov.ua/laws/show/373-2006-п

5.1.2 Базова версія «ERP/1: Документи»

Продукт «ERP/1: Документи» в основному базується на Поставі #55 Кабінету Міністрів України та інших постановах КМУ:

— 55-2018-п, КМУ. Постанова #55 Деякі питання документування управлінської діяльності [9],
— 749-2018-п, КМУ. Постанова #749 Про затвердження Порядку використання електронних довірчих послуг в органах державної влади, органах місцевого самоврядування, підприємствах, установах та організаціях державної форми власності[10],
— v0144774-20, ДП «Український науково-дослідний і навчальний центр проблем стандартизації, сертифікації та якості". Наказ #144 Про прийняття та скасування національних стандартів ДСТУ 4163:2020 та ДСТУ 9031:2020[11],

але платформа продукту базується на наказаї Міністерства юстиції України, Міністерства цифрової трансформації, Міністерства освіти і науки, та Законами України:

— z1854-12, Міністерство юстиції України. Наказ #16005 Про затвердження Порядку роботи з електронними документами через систему електронної взаємодії органів виконавчої влади з використанням електронного цифрового підпису[12],
— z1039-20, Міністерство цифрової трансформації України. Адміністрація державної служюи спеціального зв'язку та захисту інформації України. Наказ #140614[13],
— z1306-11, Міністерство освіти і науки, молоді та спорту України. Наказ #1207 Про вимоги до форматів даних електронного документообігу в органах державної влади. Формат електронного повідомлення[14],
— z1421-14, Міністерство юстиції України. Наказ #1886/5 Про затвердження Порядку роботи з електронними документами у діловодстві та їх підготовки до передавання на архівне зберігання[15], — 851-IV, Про електронні документи та електронний документообіг[16],
— 8094-ВР, Про захист інформації в інформаційно-телекомунікаційних системах[17],

[9] https://zakon.rada.gov.ua/laws/show/55-2018-п
[10] https://zakon.rada.gov.ua/laws/show/749-2018-п
[11] https://zakon.rada.gov.ua/rada/show/v0144774-20
[12] https://zakon.rada.gov.ua/laws/show/z1854-12
[13] https://zakon.rada.gov.ua/laws/show/z1039-20
[14] https://zakon.rada.gov.ua/laws/show/z1306-11
[15] https://zakon.rada.gov.ua/laws/show/z1421-14
[16] https://zakon.rada.gov.ua/laws/show/851-15
[17] https://zakon.rada.gov.ua/laws/show/80/94-вр

5.1.3 Розширення та додаткові модулі

Розширення «ERP/1: Судопис і Суддівство»

— 4651-VI, Кримінальний процесуальний кодекс України[18],
— v0298905-20, Офіс генерального прокурора. Наказ #298 Про затвердження Положення про Єдиний реєстр досудових розслідувань, порядок його формування та ведення[19],

Розширення «ERP/1: Закупівлі»

— 922-19, Закон України про публічні закупівлі[20], — 169, Постанова КМУ #169,
— 1178 Постанова КМУ #1178,
— 808-20 Закон України про оборонні закупівлі,
— 1275-2022-п Постанова КМУ #1275,
— 1070-2019-п Постанова КМУ #1070,
— 224-2020-п Постанова КМУ #224,
— 710-2016-п Постанова КМУ #710,
— z0500-20 Наказ Мінекономіки #708,
— 544-2016-п Постанова КМУ #544,
— v1749731-15 Наказ Мінекономіки #1749,
— 1495-п Постанова КМУ #1495.

Розширення «ERP/1: Зброя»

— 5708, Проект Закону про право на цивільну вогнепальну зброю[21], — 5709, Проект Закону про внесення змін до Кодексу України про адміністративні правопорушення та Кримінального кодексу України для реалізації положень Закону України ”Про право на цивільну вогнепальну зброю”[22],

[18] https://zakon.rada.gov.ua/laws/show/4651-17
[19] https://zakon.rada.gov.ua/laws/show/v0298905-20
[20] https://zakon.rada.gov.ua/laws/show/922-19
[21] http://w1.c1.rada.gov.ua/pls/zweb2/webproc4_2pf3516=5708skl=10
[22] http://w1.c1.rada.gov.ua/pls/zweb2/webproc4_2pf3516=5709skl=10

Розширення «ERP/1: Військова частина»

— ДСТУ 2732-94, Діловодство і архівна справа.
— 26.05.2014 #333, Інструкція з ведення обліку особового складу.
— #608 Порядок проведення службового розслідування.
— 26.07.2018 #370, Інструкція з діловодства.
— 07.04.2017 #124, Інструкція з діловодства.
— 07.10.2015 #393, Положення Про Юридичну Службу.
— 29.11.2018 #604, Інструкція з надання доповідей і донесень про події, кримінальні правопорушення, військові адміністративні правопорушення та адміністративні правопорушення, пов'язані з корупцією, порушення військової дисципліни та їх облік.

5.2 Функціональні модулі та вимоги

5.2.1 Призначення та цілі впровадження

5.2.2 Ревізія поточних систем

Розділ **6**

Політики

6.1 Політики

6.1.1 Загальні принципи

6.2 Технічні вимоги

6.2.1 Перелік технічних вимог наведених в політиках

6.2.2 Повний зміст технічних вимог

6.3 Технічне завдання

6.3.1 Зміст технічного завдання

6.4 Посилання на стандарти ДСТУ

6.5 Функціональні вимоги

6.5.1 Вимоги до процесів системи

6.5.2 Вимоги до викликів API

6.5.3 Вимоги до інтерфейсу користувача

6.5.3.1 Обробка помилок

6.5.3.2 Загальні вимоги

6.6 Нефункціональні вимоги

6.6.1 Вимоги до архітектури

6.6.1.1 Визначення термінів і призначення документа

6.6.1.2 Рівні архітектури компонентів системи

6.6.1.3 Маніфест відповідності архітектурі

6.6.1.4 Таксономія системних компонент

6.6.2 Вимоги до безпеки

6.6.2.1 Загальні вимоги до інформаційної безпеки

6.6.2.2 Технічні вимоги до інформаційної безпеки

6.6.3 Вимоги до потужності та ємності

6.6.3.1 Таблиця характеристик продуктивності

6.6.4 Вимоги до системи логування

6.6.4.1 Загальні вимоги

6.6.4.2 Рівні логування

6.6.4.3 Інформація для вендорів про інструментарії логування

6.6.4.4 Події логування

6.6.4.5 Структура журналу логування

6.6.4.6 Вимоги до структурних логів ELK стеку

6.6.5 Вимоги до контролю якості

6.6.5.1 Загальні вимоги до обліку і контролю якості

6.6.5.2 Вимоги до ручного тестування

6.6.5.3 Вимоги до формату сценаріїв та протоколів тестування

6.6.5.4 Вимоги до плану тестування

6.6.5.5 Вимоги до автоматичного тестування

6.6.5.6 Вимоги до результатів приймального (UAT) тестування

6.6.6 Вимоги до інтерфейсів

6.6.6.1 Загальні вимоги

6.6.6.2 Вимоги до відображення інформації

6.6.6.3 Вимоги до швидкодії додатка

6.6.6.4 Вимоги до технології реалізації додатку

6.6.7 Юридичні вимоги

Розділ **7**

Структура ERP/1

7.1 Бюджетне планування і контроль видатків

7.1.1 Вимоги до сутностей BUD

7.1.1.1 Фінансовий план

7.1.1.2 Коригування плану

7.1.1.3 Реєстр закупівель

7.1.1.4 Реєстр погоджень

7.1.1.5 Реєстр кошторисів і асигнувань

7.1.1.6 Реєстр казначейських рахунків і зобов’язань

7.1.2 Вимоги до процесів BUD

7.1.2.1 Планування бюджету

7.1.2.2 Коригування та контроль

7.1.2.3 Погодження видатків

7.1.2.4 Формування бюджетних запитів і декларацій

7.1.2.5 Казначейський контроль платежів

7.1.2.6 Інтеграція з Державним казначейством

7.2 Фінансовий і бухгалтерський облік

7.2.1 Вимоги до сутностей FIN

7.2.1.1 Головна книга

7.2.1.2 Первинні документи

7.2.1.3 Журнал проводок

7.2.1.4 Звіти та аналітика

7.2.1.5 Картотека основних засобів і НМА

7.2.1.6 Податкові регістри та ПДВ

7.2.1.7 Форми фінансової звітності

7.2.2 Вимоги до процесів FIN

7.2.2.1 Проведення операцій

7.2.2.2 Закриття періоду

7.2.2.3 Автоматична звітність

7.2.2.4 Подвійний запис за П(С)БО

7.2.2.5 Розрахунок амортизації та переоцінка

7.2.2.6 Формування та подання Є-звітності

7.2.3 Вимоги до сховища та серіалізації FIN

7.3 Кадрова система

7.3.1 Вимоги до сутностей АСС

7.3.1.1 Кадровий реєстр

7.3.1.2 Розрахунок заробітної плати

7.3.1.3 Табелі обліку часу

7.3.1.4 Звіти з кадрів

7.3.1.5 Штатний розпис і накази

7.3.1.6 Табелі та розрахунок ЗП з нарахуваннями

7.3.2 Вимоги до процесів АСС

7.3.2.1 Прийом та переміщення

7.3.2.2 Розрахунок та виплата

7.3.2.3 Формування звітності

7.3.2.4 Автоматизований кадровий документообіг

7.3.2.5 Інтеграція з автоматизованим розподілом справ

7.3.3 Вимоги до сховища та серіалізації АСС

7.4 Публічний документообіг і сервіс-деск

7.4.1 Вимоги до сутностей CRM

7.4.1.1 Реєстр договорів

7.4.1.2 Реєстр ТМЦ

7.4.1.3 Реєстр постачальників

7.4.1.4 Первинні документи CRM постанови КМУ №55

7.4.2 Вимоги до процесів CRM

7.4.2.1 Обробка документів за процесами постанови КМУ №55

7.4.2.2 Закупівлі та постачання

7.4.2.3 Сервісне обслуговування

7.4.3 Вимоги до сховища та серіалізації CRM

7.5 Шаблони документів

7.5.1 Вимоги до сутностей

7.5.1.1 Шаблон DOCX/XLSX

7.5.1.2 Набір змінних

7.5.2 Вимоги до процесів

7.5.2.1 Заповнення шаблону

7.5.2.2 Версіонування шаблонів

7.6 Генерація документів

7.6.1 Вимоги до сутностей

7.6.1.1 Заповнений документ

7.6.1.2 PDF-документ

7.6.2 Вимоги до процесів

7.6.2.1 Генерація PDF

7.6.2.2 Накладання КЕП

7.6.2.3 Архівація

7.7 Генерація бар- та штрих-кодів

7.7.1 Вимоги до сутностей

7.7.1.1 Код

7.7.1.2 Накладання

7.7.2 Вимоги до процесів

7.7.2.1 Генерація коду

7.7.2.2 Накладання на документ

7.7.2.3 Верифікація

7.8 Розпізнавання Tesseract

7.8.1 Вимоги до сутностей

7.8.1.1 Документ PDF

7.8.1.2 Результати OCR

7.8.2 Вимоги до процесів

7.8.2.1 Інгестія документа

7.8.2.2 Виконання OCR

7.8.2.3 Повнотекстовий пошук

7.8.2.4 Отримання підсвічування

7.9 Система сканування

7.9.1 Вимоги до сутностей

7.9.1.1 Профіль сканера

7.9.1.2 Сканований документ

7.9.2 Вимоги до процесів

7.9.2.1 Ініціація сканування

7.9.2.2 Доставка результату

7.10 Багатокористувацький редактор

7.10.1 Вимоги до сутностей

7.10.1.1 Редакційний документ

7.10.1.2 Операція редагування

7.10.2 Вимоги до процесів

7.10.2.1 Синхронізація в реальному часі

7.10.2.2 Історія змін

7.10.2.3 Фіксація версії

7.10.2.4 Транзакційність

7.10.3 Призначення та цілі впровадження

7.11 Інфраструктура безпеки

7.11.1 Провайдери української криптографії

7.11.2 Вимоги до сутностей

7.11.2.1 Директорія (LDAP)

7.11.2.2 Сертифікати (X.509)

7.11.2.3 Користувачі (inetOrgPerson)

7.11.2.4 Організації (organization)

7.11.2.5 Криптографічне повідомлення (CMS X.894)

7.11.2.6 Кваліфікований електронний підпис (ДСТУ 4145)

7.11.2.7 Відкликання і перевірка (OCSP)

7.11.2.8 Мітка часу (TSP)

7.11.2.9 Ключі шифрування та сервісні креденціали

7.11.3 Вимоги до процесів

7.11.3.1 Видача сертифікату (enroll)

7.11.3.2 Автентифікація сертифікатом (check)

7.11.3.3 Відкликання сертифікату (revoke)

7.11.3.4 Оновлення сертифікату (renew)

7.11.3.5 Підпис і верифікація (sign/verify)

7.11.3.6 Шифрування і розшифрування (encrypt/decrypt)

7.11.3.7 Управління секретами (store/retrieve)

7.11.4 Вимоги до серіалізації і сховища

7.12 Рольова модель і система доступів

7.12.1 Вимоги до рольової моделі ABAC

7.12.1.1 Об’єкт доступу

7.12.1.2 Суб’єкт доступу

7.12.1.3 Правило доступу

7.12.2 Вимоги до сутностей ABAC

7.12.2.1 Політики доступу

7.12.3 Вимоги до процесів ABAC

7.12.3.1 Життєвий цикл Політики доступу

7.12.3.2 Точки доступу

7.12.4 Вимоги до сутностей IAM

7.12.4.1 Адміністратор організацій

7.12.4.2 Адміністратор структурних підрозділів

7.12.4.3 Адміністратор співробітників

7.12.4.4 Адміністратор користувачів

7.12.5 Вимоги до процесів IAM

7.12.5.1 Життєвий цикл Адміністратора

7.12.5.2 Життєвий цикл Організації

7.12.5.3 Життєвий цикл Користувача

7.12.5.4 Життєвий цикл Сесії

7.12.5.5 Життєвий цикл Ролі

7.12.6 Вимоги до рольової моделі IAM

7.12.6.1 Точки доступу

7.12.7 Вимоги до сховища

7.12.8 Вимоги до серіалізації і валідації

7.13 Оркестрація процесів

7.13.1 Вимоги до сутностей BPMN

7.13.1.1 Документ

7.13.1.2 Процес

7.13.1.3 Монітор

7.13.1.4 Задача користувача (APM)

7.13.1.5 Системна задача

7.13.1.6 Перехід

7.13.1.7 Подія

7.13.1.8 Таймаут

7.13.1.9 Асинхронне повідомлення

7.13.2 Вимоги до процесів

7.13.2.1 Створення

7.13.2.2 Зупинення

7.13.2.3 Відновлення

7.13.2.4 Архівація

7.13.2.5 Зміна контексту процесу

7.13.2.6 Перехід на стадію

7.13.3 Вимоги до сховища

7.14 Словникова підсистема

7.14.1 Загальні відомості

7.14.2 Вимоги до сутностей HL7

7.14.2.1 Системноутворюючі довідники

7.14.2.2 Похідні довідники

7.14.2.3 Система імен

7.14.2.4 Відображення довідників

7.14.2.5 Налаштування довідникової системи

7.14.3 Вимоги до процесів

7.14.3.1 Створення попереднього словника

7.14.3.2 Редагування попереднього словника

7.14.3.3 Публікація словника

7.14.3.4 Деактивація словника

7.14.3.5 Архівування словника

7.14.4 Вимоги до сховища

7.15 Мета сервіси та конструктор

7.15.1 Реєстр сутностей системи (API)

7.15.2 Біблітека валідації (JSON Schema)

7.15.3 Конструктор сутностей та форм

7.15.4 Конструктор бізнес процесів

7.15.5 Конструктор словників

7.15.6 Конструктор правил доступу

7.15.7 Конструктор шаблонів профілів користувачів

Розділ **8**

Державна система

По аналогії зі стандартом ISO 42010 «Фреймворку Закмана», фреймворк Максима Сохацького визначає та уточнює архітектурні рівні з яких складаються сучасні корпоративні інформаційні системи:

— Юридично-документальний рівень
— Обліково-реєстровий рівень
— Зв'язність людей та пристроїв
— Генерація, валідація і верифікація
— Телекомунікаційна платформа і безпека інтернету

8.1 Юридично-документальний рівень

Згідно фреймворку верхній шостий рівень визначає BPMN процеси згідно яких здійнюється відзеркалення юридично-правивих відносин електронного документообігу. Кожен крок такого процесу, та усі його документи підписуються особистим ключем КЕП посадової особи, що дає змогу проведення диспутів та розслідувань Міністерством юстиції України. Окрім того цей рівень системи орієнтований на аналітику у взаємодії з громадянами через СЕВ ОВВ.

Юридично-документальні системи ERP/1 будуються на сховищі з єдиним простором ключів Facebook RocksDB, що здатне працювати через Intel SPDK на NVMe дисках, наприклад у складі таких сховищ як CEPH. Обсяг обігу документів на великих підприємствах сягає 1ТБ на рік.

8.2 Обліково-реєстровий рівень

Обліково-реєстровий рівень пропонує низькорівневе масштабоване розподілене журнальне сховище даних та метаданих, яке може бути побудоване на реляційних базах даних, базах даних з єдиним простором ключів з гарантіями консистентності (chain-hash) або їх комбінаціях.

Класичні представники цього рівня в системах управління підприємствами: система управління людськими та матеріальними ресурсами, банківські системи PCI DSS, складські системи, медичні інформаційні системи, системи управління поставками та виробництвом, системи сервісних послуг, системи управління проектами, тощо.

8.3 Технологічний рівень зв'язності людей та пристроїв

8.3.1 Локальний

Рівень зв'язності людей та пристроїв визначає комунікаційні протоколи та технології, які об'єднують головні ресурси підприємства (пристрої та людей) у одну телекомунікаційну мережу. Як правило виробництво складається з багатьох пристроїв що підключаються до промислових шин як MQTT, та робочих місць користувачів, каналів зв'язку з інформаційними системами, корпоративні та національні шини, тощо.

Цей рівень також визначає засоби мастшабування пам'яті (персистентної та волатильної) та обчислювальних ресурсів (за допомогою процесінгових брокерів доставки повідомлень). Це рівень визначає реляційні бази даних та бази даних з єдиним простором ключів, а також стандарти та протоколи передачі інформації у промислових ERP системах, такі як CSV, JSON, SOAP, BERT, ASN.1, тощо.

8.3.2 Крос-системний

Крім того Мінцифра підтримує два середовища інтеграційної взаємодії національного рівня, учасниками яких є суб'єкти господарювання індексовані ЄДРПО підприємства:

1) національна система електронної взаємодії органів виконавчої влади (СЕВ ОВВ) з відкритим ринком клієнтів[1] і широким охопленням органів виконавчої влади[2]. Ця шина діє на рівні юридично-документального рівня і безпосередньо пов'язана з серверами документообігу, які є учасниками цієї взаємоії;

2) національна система електронної взаємодії державних електронних інформаційних ресурсів[3] (СЕВДЕІР «Трембіта»), яка представляє собою спеціалізовану версію Ubuntu з пакетами X-ROAD, власною інфраструктурою CA, TSP, OCSP серверами і шифрованими каналами передачі конфіденційної інформації.

[1] https://se.diia.gov.ua/sedlist — Перелік сертифікованих систем документообігу

[2] https://se.diia.gov.ua/uploads/documents/45.xlsx — Перелік систем документообігу і їх ЄДРПО підключених до національної шини СЕВ ОВВ

[3] https://catalog.trembita.gov.ua/?env=SEVDEIR — Каталог сервісів «Трембіта»

8.4 Генерація, валідація і верифікація

Рівень схеми даних визначає модель зберігання даних як з точки зору об'єктів-сутностей так і з точки зору технологій та протоколів, які необхідні для їх опису. Головним чином це Фреймворк Закмана та сімейство стандартів які описують UML, System F та необхідні генератори SDK, верифікатори типів (валідатори), моделі процесів, тощо.

8.5 Безпека інтернету та інфраструктури

Рівень безпеки визначає схему функціонування основного центрального засвідчувального орнагу, акредитованих центрів сертифікації ключів, протоколи шифрування та підпису, директорію підприємства, інтернет протоколи найменування ресурсів, шифровані протоколи комунікації, диспетчерські системи трафіку повітряних суден, тощо. Усе визначено згідно ASN.1 специфікації і стандартів протоколів серії X[4].

[4] https://www.itu.int/itu-t/recommendations/index.aspx?ser=X

Розділ **9**

Юридично-документальний рівень

9.1 Вступ

Відповідно до фреймворку системи, верхній п'ятий рівень визначає BPMN-процеси, згідно з якими здійснюється точне відображення юридичних відносин у вигляді електронного документообігу. Кожен крок такого процесу та всі супровідні документи підписуються особистим ключем КЕП (кваліфікованого електронного підпису) посадової особи. Це створює надійне підґрунтя для проведення аудитів, диспутів та службових розслідувань Міністерством юстиції України, із залученням власного Засвідчувального центру в просторі інтернет-імен. Окрім того, цей рівень системи повністю орієнтований на аналітику та прозору взаємодію з громадянами через Систему електронної взаємодії органів виконавчої влади (СЕВ ОВВ).

Юридично-документальні системи класу ERP/1 будуються на базі надійного сховища з єдиним простором ключів (на кшталт Facebook RocksDB), що здатне працювати через інтерфейси Intel SPDK на швидкісних NVMe дисках, наприклад, у складі таких кластерних сховищ, як CEPH. Архітектура враховує той факт, що обсяг обігу документів на великих державних чи комерційних підприємствах може сягати 1 ТБ на рік.

9.1.1 Види документообігів

9.1.1.1 Постанова №55 КМУ

9.1.1.2 Наказ №124 МОУ

9.1.1.3 Закупівлі

9.1.1.4 Провадження

9.1.1.5 Зброя

9.1.2 Функціональні можливості

9.2 Модулі підприємства

ERP/1 є потужним комплексом інструментальних бібліотек (N2O.DEV) та підсистем додатків (ERP.UNO), який використовує загальну шину повідомлень і спільну розподілену базу даних для побудови швидкісних операційних вітрин.

ERP — Даний модуль обліково-реєстраційного рівня надійно зберігає основну ієрархічну структуру підприємства, її схему, метаінформацію про базові типи даних, а також безпосередньо саму інформацію: записи про персонал, інвентар, компанії-контрагенти та офіси підприємства.

CRM — Система функціонального управління зв'язками з громадськістю та органами виконавчої влади: являє собою базову вітально необхідну реалізацію вимог постанови №55 КМУ.

CART — Система гнучкого управління реєстрами: являє собою реалізацію надійного базового сервера для виконання широкого спектра реєстрових та класифікаторних задач.

9.3 Управління ресурсами

Головним чином інформаційна інфраструктура підприємства складається з активних обчислювальних ресурсів (додатків та сервісів, запущених у шині) та пасивних накопичувальних ресурсів (даних, збережених у розподіленій базі).

Для ефективного управління обчислювальними ресурсами SOA-архітектура як системна модель пропонує асинхронний протокол віддаленого виклику процедур безпосередньо на шинах. Разом з екосистемою N2O можна використовувати MQTT та інші транспортабельні шини, послуговуючись такими протоколами, як TCP та WebSocket. Ці асинхронні комунікації часто називають протоколами реального часу, оскільки функції відправлення повідомлень у них завжди миттєво повертають результат (fire-and-forget). Що ж стосується протоколів для традиційної публікації та доступу до даних, то у цьому контексті може виявитися особливо доречним застосування класичного синхронного протоколу HTTP.

9.4 Архітектура врядувальних CRM-систем

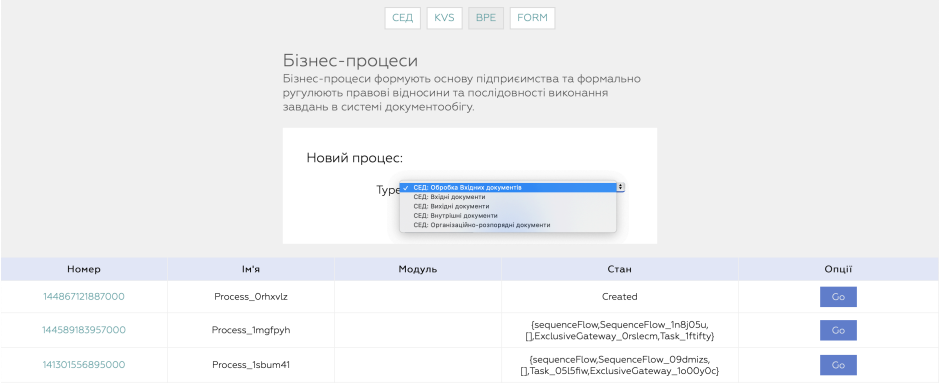
9.4.1 Сторінки

Нижче наведено базовий перелік маршрутів сторінок:

```
def route(<<"ldap", _::binary>>), do: LDAP.Index
def route(<<"crm",  _::binary>>), do: CRM.Index
def route(<<"rmk",  _::binary>>), do: RMK.Index
def route(<<"kvs",  _::binary>>), do: KVS.Index
def route(<<"act",  _::binary>>), do: BPE.Actor
def route(<<"help", _::binary>>), do: HELP.Index
```

9.4.1.1 LDAP

Сторінка безпечної авторизації та ідентифікації користувачів.



Сторінка авторизації

9.4.2 Комболукап

9.4.3 Сервіси

9.4.4 СЕВ ОВВ

9.4.5 Шаблони

9.4.6 Дерева

9.4.7 Процеси

Даний керівний модуль надійно інкапсулює визначення сутностей Схеми, Бізнес-процесів та Форм, які безперервно використовуються у системі Infotech-ERP згідно з академічною методологією фреймворку Закмана.

9.4.7.1 Формування нормативно-довідкової інформації

У цьому модулі автоматизації виділяються такі основні типи процесів для організаційно-розпорядчих документів: «Накази», «Протоколи», «Доручення керівництва».

9.4.7.2 Обробка вхідних документів

Усі вхідні документи автоматично надходять в УДСД (Управління документального забезпечення), ВОРЗГ (Відділ організації роботи зі зверненнями громадян) та ВОДПІ. При надходженні документа уповноважена посадова особа зазначених структурних підрозділів сумлінно реєструє його в системі, після чого стартує процес його подальшої маршрутизації та обробки.

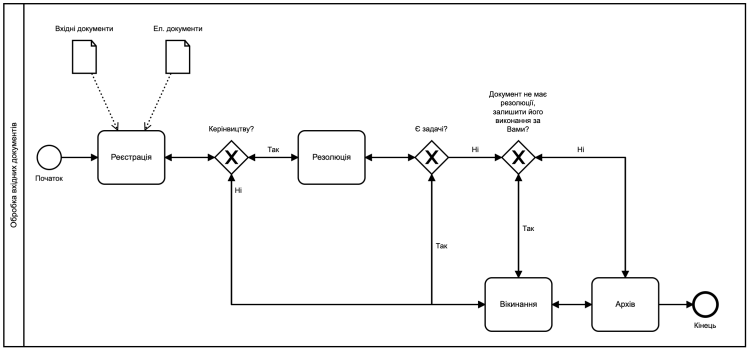


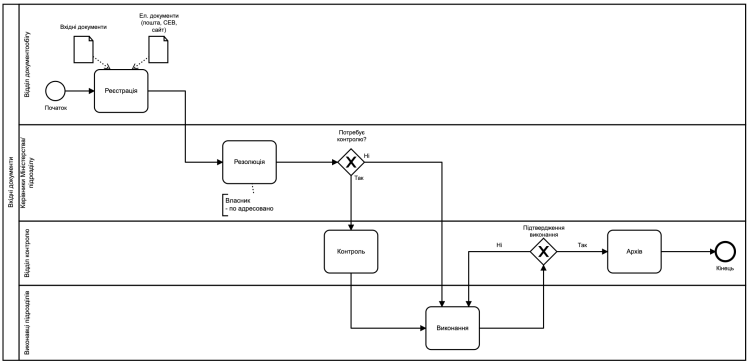
Рис. 9.2 Бізнес-процес обробки вхідних документів

Далі зареєстрований вхідний документ надходить або Міністру, Заступнику міністра, чи Державному секретарю для накладання управлінської резолюції, або безпосередньо спрямовується до профільного структурного підрозділу на виконання.

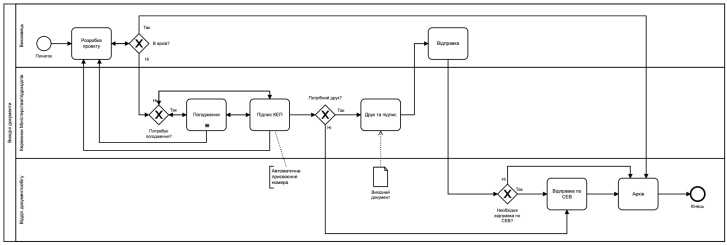
9.4.7.3 Вхідні документи

9.4.7.4 Вихідні документи

Вихідні документи формуються та створюються безпосередньо в підрозділах, які виступають ініціаторами документа. Вони можуть виникати як з власної ініціативи співробітників Міністерства, так і як прямий результат або відповідь на обробку вхідних документів. Якщо вихідний документ логічно пов'язаний із вхідним документом, то відповідальному працівнику система рекомендує обов'язково вказати семантичне посилання на пов'язаний



Бізнес-процес обробки вхідних документів



Бізнес-процес підготовки вихідних документів

документ-першоджерело. Регламентна обробка документів виконується за єдиним еталонним уніфікованим бізнес-процесом, незалежно від конкретного місця виникнення документа. Від початку в системі реєструється проект вихідного документа, який неодмінно повинен бути завізований і погоджений із визначеним кваліфікованим переліком погоджуючих посадових осіб. Щойно фінальний підписант накладає свій КЕП, документу автоматично присвоюється вихідний реєстраційний номер, і система ініціює його відправку.

9.4.7.5 Внутрішні документи

Внутрішні документи можуть створюватися та вводитися всіма авторизованими учасниками контуру документообігу. Серед них виділяються такі основні стандартизовані бізнес-процеси внутрішніх документів: «Доповідна записка», «Службовий лист».

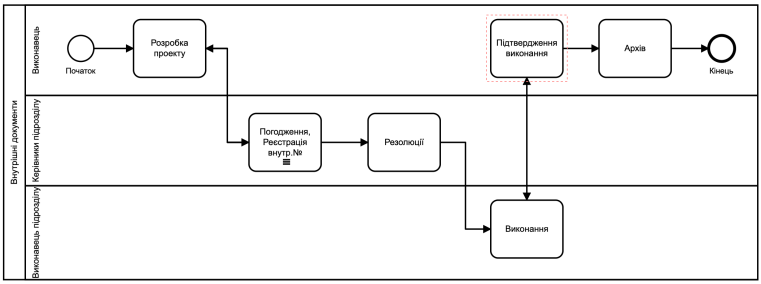


Рис. 9.5 Бізнес-процес обігу внутрішніх документів

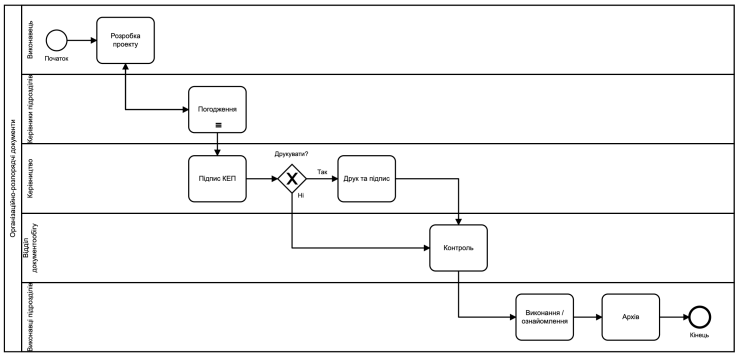


Рис. 9.6 Бізнес-процес просування організаційно-розпорядчих документів

9.4.7.6 Організаційно-розпорядчі документи

Розробка проєкту документа

На цьому початковому етапі розробляється базовий електронний проєкт документа. Спочатку уважно заповнюються всі необхідні мета-реквізити в електронній картці документа; після її збереження до картки автоматично прикріплюється візуальний шаблон документа, який відіграватиме роль оригіналу. Далі виконавець вносить безпосередній змістовий вміст документа в цей оригінал і зберігає його версію. Ініціатор у системі додає всіх виконавців, кому адресований майбутній наказ. Варто зазначити, що використовуючи поле «Адресовано», система автоматично згенерує та призначить задачі відповідним виконавцям. Тільки після цих процедур документ слід передати на наступний крок.

Погодження (візування)

На даному кроці у суворій послідовності виконується фахове погодження документа усіма особами, які були заздалегідь вказані виконавцем під час створення проєкту. Обов'язковою умовою автоматичної передачі документа на наступний етап є отримання позитивного погодження (візи) від **УСІХ**

погоджуючих осіб. В іншому разі просунути документ за маршрутом далі просто неможливо. Якщо принаймні один з візуючих відхилив документ (при цьому він зобов'язаний внести обґрунтований коментар із причинами відхилення і зауваженнями до тексту чи суті наказу) — система повертає документ на першу стадію, інформуючи виконавця про необхідність його доопрацювання. Якщо ж усі без винятку особи погодили проєкт, система автоматично переводить його на наступну стадію. Після остаточного візування є можливість згенерувати та вивантажити офіційний «Аркуш погодження» у вигляді друкованої форми.

Застосування КЕП (кваліфікованого електронного підпису)

На цій критичній стадії документ остаточно підписується в електронному вигляді уповноваженим керівництвом Міністерства. Саме в момент накладання валідного криптографічного КЕП документу присвоюється постійний інвентарний реєстраційний номер. У разі налаштування Системи електронного діловодства (СЕД) на використання візуального QR-коду, він генерується з розмірами 21 на 21 мм і надійно розміщується в нижньому лівому куті першої сторінки документа. Якщо ж політикою СЕД передбачено використання класичного лінійного штрихкоду, його алгоритмічно розміщують у правому кутку нижнього поля першої сторінки оригінального документа.

Підпис у паперовій формі

Після електронного підписання організаційно-розпорядчого документа, у разі суворої необхідності створення так званого «паперового оригіналу», уповноважена особа патронатної служби (помічник Міністра, заступника міністра чи державного секретаря) акуратно роздруковує документ і надає його на «мокрий» підпис безпосередньо керівнику. Якщо ж відповідний організаційно-розпорядчий документ був підписаний локальним керівником певного підрозділу, необхідний паперовий примірник, як правило, роздруковує сам виконавець.

Постановка на контроль

На даному етапі спеціалізований контролюючий структурний підрозділ аналізує завдання за документом і за необхідності здійснює його постановку на жорсткий контроль, вказуючи також періодичність звітування. Документи та задачі, що перебувають на контролі, завжди доступні уповноваженим особам за окремим фільтром, незалежно від поточного кроку свого опрацювання. Це дає змогу ефективно відстежувати їхній статус на будь-якій стадії життєвого циклу, чітко контролювати своєчасність виконання, проводити комплексну аналітику тощо, не втручаючись напролом в операційний перебіг.

Виконання та ознайомлення

На даному виконавчому етапі відповідальний за ознайомлення або контроль структурний підрозділ бере до уваги директиви, викладені в документі. За всіма документами на контролі в режимі реального часу відстежується їхній актуальний статус. Працівники можуть переглядати, аналізувати та закривати задачі паралельно з тим, як потік процесів продовжує свій рух інформаційними каналами міністерства.

Цифровий шифрований архів

Після виконання завдань та загального ознайомлення адресатів, документ офіційно переходить у довгостроковий цифровий Архів. Система архіву додатково накладає електронні підписи КЕП самого Архіву, надійно цементуючи файл. Крім того, задля гарантування непорушності, Архів навічно утримує повну криптографічну історію та ланцюг усіх застосованих проміжних сертифікатів: починаючи від локального АЦСК та закінчуючи ЦЗО.

9.4.7.7 Звернення громадян

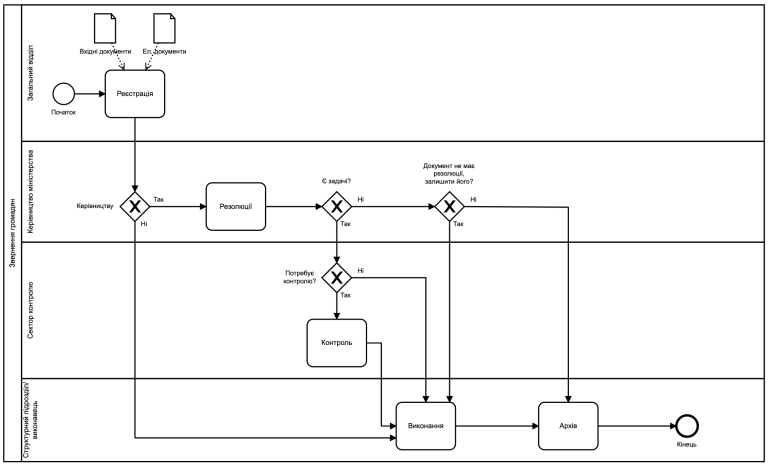


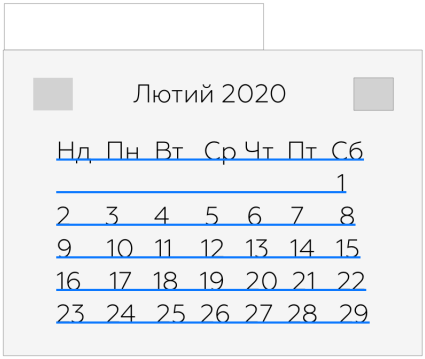
Рис. 9.7 Бізнес-процес опрацювання звернень громадян

9.4.8 Елементи інтерфейсу

У цьому розділі зібрано мінімально необхідну кількість бізнес-форм та UI-компонентів, розроблених спеціально для CRM-сегменту системи електронного діловодства (СЕД). Вони є індивідуальними та критично необхідними для виконання функціональних вимог, висунутих замовниками.

9.4.8.1 Календар

Базовий елемент інтерфейсу «Календар» взято з відкритої бібліотеки NITRO, проте він адаптований і потребує застосування додаткової кастомної стилізації.



Контрольний елемент «Календар»

9.4.8.2 Пошук за довільними атрибутами

Для забезпечення швидкого та релевантного пошуку за величезними загальнонаціональними словниками та бізнес-об'єктами в системі передбачено створення спеціалізованого скалярного комбо-пошуку. Він дає змогу виконувати віддалені запити за довільними полями в централізованому сховищі даних (наприклад, виконувати миттєвий пошук за довідниками співробітників, індексами населених пунктів КОАТУУ тощо).

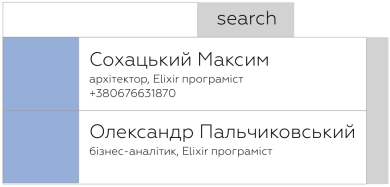
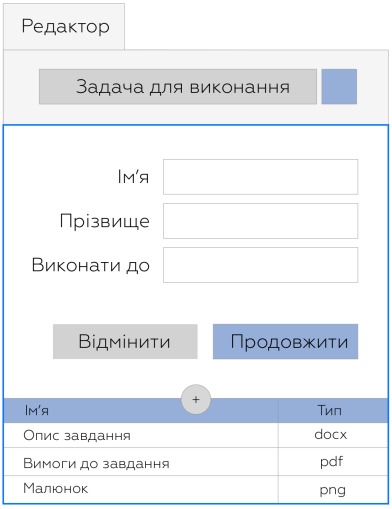


Рис. 9.9 Контрольний елемент віддаленого пошуку в базі даних

9.4.8.3 Форми редагування та пошуку

Для кожного зареєстрованого типу документа чи сутності в системі створюються дві паралельні форми: форма пошуку і форма редагування (яка одночасно виступає як форма створення нового об'єкта). Наявність цих двох різних сутностей суворо вмотивована відмінністю у логіці їхніх валідаторів. У формі пошуку валідатори апріорі повинні дозволяти залишати поля порожніми, тоді як для форми редагування валідатори зобов'язані безкомпромісно перевіряти сувору повноту та валідність заповнення полів критичних бізнес-об'єктів.



Контрольний елемент інтерактивного редагування документа та прикріплених підлеглих файлів Рис. 9.10

9.4.8.4 Управління бізнес-процесом

Для ефективного управління щоденними завданнями, миттєвого доступу до документів у розрізі певного процесу, створення дочірніх документів, а також для процесів візування, підпису або подальшої маршрутизації документів по бізнес-ланцюгах в інтерфейсі використовується стандартизований контрольний елемент управління бізнес-процесом.

Контрольний елемент управління бізнес-процесами Рис. 9.

9.4.8.5 Документи у виконавчих процесах

Під час навігації сторінками та документами окремого процесу інтерфейс гарантує миттєве відображення вмісту будь-якого підлеглого документа у

спеціальній лівій робочій панелі головної сторінки користувача, що значно прискорює аналіз та прийняття рішень.

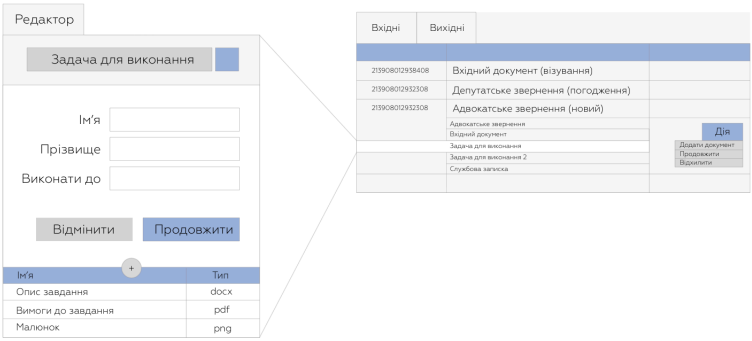
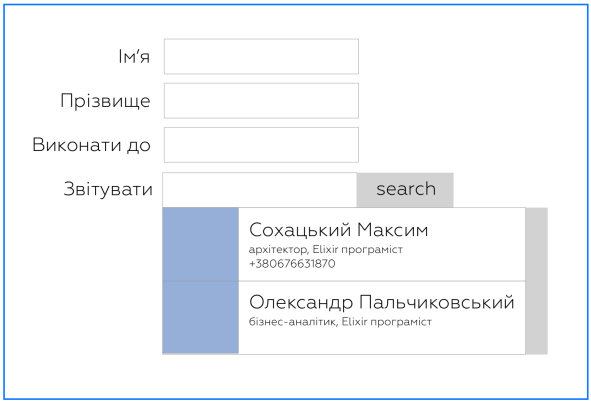


Рис. 9.12 Інструменти навігації за документами обраного бізнес-процесу

9.4.8.6 Використання вбудованих контролів у формах

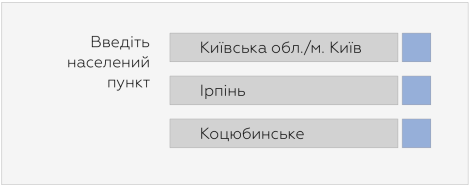
Приклад практичного використання універсального контрольного елемента довільного пошуку-комболукапу всередині складних комплексних форм.



Приклад безшовної інтеграції контрольних елементів у структурованих формах Рис. 9.13

9.4.8.7 Контрольний елемент КОАТУУ

Наочний приклад роботи спеціалізованого державного контрольного елемента пошуку за класифікатором КОАТУУ.



Приклад спеціалізованого використання контрольного елемента КОАТУУ Рис. 9.14

9.4.9 Редактори та додатки

Тут перелічено контролери основних сторінок, кожна з яких є повноцінним незалежним SPA (Single Page Application) вебдодатком, завантаженим в екосистему.

9.4.9.1 Вхід до системи

Головна сторінка входу та первинної авторизації користувачів з використанням ЕЦП (КЕП).

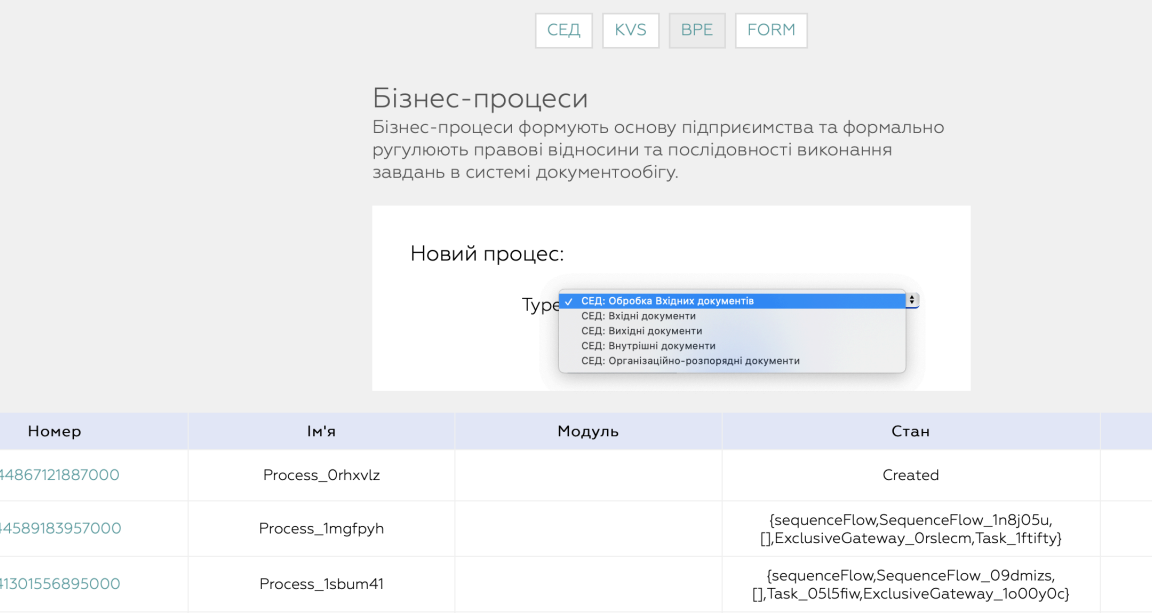
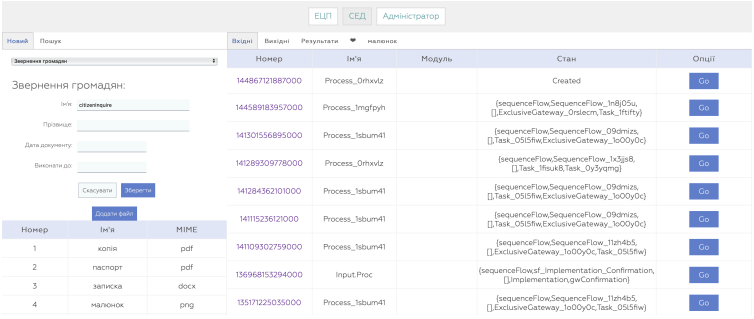


Рис. 9.15 Захищена сторінка входу в систему

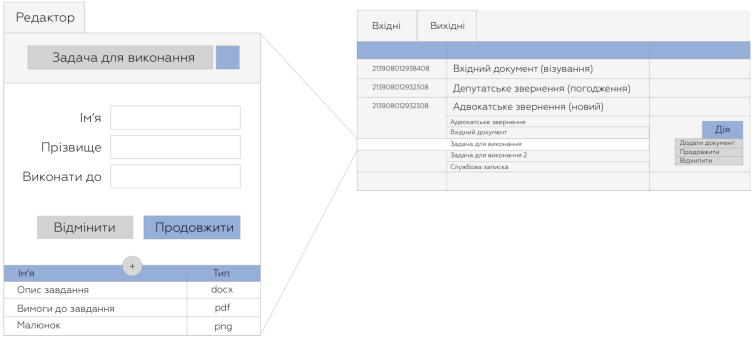
9.4.9.2 Робота з документами

Основна консоль системи та найголовніша робоча сторінка користувача, яка слугує хабом для безпосередньої обробки документів у робочих бізнес-процесах.



Функціональна сторінка роботи зі вхідними та вихідними документами Рис. 9.16

При навігації за документами процесу користувачеві доступне миттєве відображення змісту підлеглого документа або його вкладень на лівій інформаційній панелі головної сторінки, що оптимізує час опрацювання.



Мініатюри та швидка навігація за підлеглими документами Рис. 9.17

9.4.10 Конструктор процесів

Тут представлено спеціалізовані адміністративні сторінки управління системою, які призначені для системних архітекторів.

9.4.10.1 Бізнес-об'єкти

Глобальний централізований каталог усіх типів бізнес-об'єктів (документів, організацій, форм), які наразі існують у системі.

9.4.10.2 Бізнес-процеси

Вичерпний перелік усіх зареєстрованих та затверджених бізнес-процесів у системі, із забезпеченою можливістю гнучкого тестування, відлагоджування і проєктування маршрутів.

9.4.10.3 Бізнес-форми

Перелік усіх стандартизованих візуальних форм документів та кастомних бізнес-форм користувача, спеціально зареєстрованих для рендерингу в системі.

9.4.11 Мова програмування FormalTalk

Розділ **10**

Обліково-реєстраційний рівень

10.1 Вступ

Обліково-реєстровий рівень пропонує низькорівневе масштабоване розподілене журнальне сховище даних та метаданих, яке може бути побудоване на реляційних базах даних, базах даних з єдиним простором ключів з гарантіями консистентності (chain-hash) або їх комбінаціях. Класичні представники цього рівня в системах управління підприємствами: система управління людськими та матеріальними ресурсами, банківські системи PCI DSS, складські системи, системи управління поставками та виробництвом, системи сервісних послуг, системи управління проектами, тощо.

10.1.1 Види реєстрів

1) Реєстри орієнтовані на суб'єктів організаційних систем;
2) Реєстри орієнтовані на облік матеріальних ресурсів;
3) Реєстри орієнтовані на георафічні об'єкти;
4) Реєстри орієнтовані на події;
5) Реєстри орієнтовані на документи, накази, НПА;
5) Реєстри медичних систем (FHIR);
5) Реєстри предметно-орієнтованих словників для функціональних підсистем.

10.1.2 Функціональні можливості

10.2 Модулі підприємства

ERP/1 є комплексом бібліотек (N2O.DEV) та підсистем додатків (ERP.UNO), який використовує загальну шину і загальну розподілену базу даних для швидкісних операційних вітрин.

FIN — Фінансовий модуль підприємства для бухгалтерії, зберігає бізнес процеси, які представляють собою рахунки учасників системи: персонал (для нарахування зарплат), рахунки та субрахунки підприємства (для здійснення економічної діяльності) і зовнішні рахунки в платіжних системах.

ACC — Система управління персоналом: зарплатні відомості, календар підприємства, відпустки, декретні відпустки, інші календарі.

SCM — Система управління ланцюжком поставок: головний БП системи — експедиційний процес доставки товарів ланцюжку одержувачів за допомогою транспортних компаній.

PLM — Система управління життєвим циклом проектів і продуктів. Також містить CashFlow та P&L звіти.

PM — Система управління проектами підприємства з деталізацією часу і протоколів прийому-передачі (прийняті коміти в гитхабі).

WMS — Система управління складом, устаткуванням, деталями.

TMS — Система управління транспортом підприємства.

HL7 — Медична система, яка реалізує міжнародний FHIR стандарт.
МЕДИЧНА КАРТА стаціонарного хворого — №003/о[1]
КАРТА ПАЦІЄНТА, який вибув із стаціонару — №066/о[2]
МЕДИЧНА КАРТА амбулаторного хворого — №025/о[3]
КОНТРОЛЬНА КАРТА диспансерного нагляду — №030/о[4]
МЕДИЧНА КАРТА стоматологічного хворого — №043/о[5]
КАРТА хворого фізіотерапевтичного відділення — №044/о[6]
Медична карта новонародженого — №097/о[7]

[1] https://zakon.rada.gov.ua/laws/show/z0662-12n2
[2] https://zakon.rada.gov.ua/laws/show/z0668-12n2
[3] https://zakon.rada.gov.ua/laws/show/z0669-12n2
[4] https://zakon.rada.gov.ua/laws/show/z0671-12n2
[5] https://zakon.rada.gov.ua/laws/show/z0678-12n2
[6] https://zakon.rada.gov.ua/laws/show/z0689-12n2
[7] https://zakon.rada.gov.ua/laws/show/z0233-16n7

10.3 Архітектура облікових CART систем

10.3.1 Облік метаінформації

10.3.2 Облік ABAC правил

10.3.3 Облік інфраструктури і само-моніторинг

10.3.4 Облік словників і класифікаторів

10.3.5 Адміністративний облік

10.3.6 Облік таксономії предметної облісті

10.3.7 Облік процесів предметної області

Розділ **11**

Технологічний рівень зв'язності людей та пристроїв

11.1 Вступ

Ця глава визначає форамальну специфікацію на програмне забезпечення усіх рівнів моделі Закмана для підприємств ISO-42010, містить широкий спектр прикладів, розказує про складові компоненти та є вичерпним авторським стартовим посібником для курсу навчання розробки технологічних програм для платформи Erlang і публікації в системі електронної взаємодії державних електронних інформаційних ресурсів "Трембіта".

Трембіта — це система побудована на пакетах Ubuntu 18 LTS і пропонує головним чином інфраструктуру X.509, а також розгортання національного форку шини X-ROAD, який використовується як сереовище для гетерогенних сервісів, в якому всі 15 міністерств публікують свої сервіси і їх клієнтські адаптери. Каталог сервісів доступний публічно[1]. Сам інтерфейс управління X-ROAD написаний з використаннями SOAP/WSDL специфікацій. В цьому цифровому просторі відбувається взаємодія (передача конфіденційних даних в основному) між обліково-реєстраційними системами міністерств на основі безпосередніх захищеник каналів зв'язку між сервісами та їх споживачами, яка містить просту і фіксовану логіку.

СЕВ ОВВ, на відміну від системи Трембіта — це публічна шина даних для урядової кореспонденції і нормативно-правових актів, вона побудована згідно ISO/IEC 11756:2010 (MUMPS) на базі продукту InterSystems Caché. В цьому цифровому просторі відбувається робота систем врядування юридично-документального рівня

[1] https://catalog.trembita.gov.ua

11.2 Виробничий процес

11.3 Системи сховищ даних

11.3.1 Реляційні бази даних

11.3.2 Бази даних з єдиним простором ключів

11.3.3 Шини комунікації та брокери повідомлень

11.3.4 Розміщені в пам'яті гарячі дані

11.4 Обчислювальні ресурси

Концептуальна модель системи в рамках якої функціонує N2O визначаена як обчислювальне середовище, яке складається з процесору подій (N2O), операційного (ETS) та персистентного сховища (KVS). З точки зору обчислювального середовища, ресурси підприємства складаються з глобального сховища та обчислень, які розділяють глобальну адресацію та представляють собою Erlang-процеси (N2O протоколи). Кожен процес PI, може містити певний набір протоколів, будь-який з яких відповідає на певний набір повідомлень. Протоколи N2O визначені на точці підключення повинні не перетинатися, в іншому випадку протокольні модулі можуть перехоплювати та впливати на інші протокольні модулі, які повинні реагувати на той самий тип повідомлень.

Усі асинхронні процеси PI запускаються під головним супервізором n2o та індексуються URI ключем разом з типом реактивного каналу реального часу: ws або mqtt. N2O протоколи підключені безпосередньо до веб-сокет точок підключення виконуються в контексті TCP процесів, у даному випадку TCP-сервера бібліотеки RANCH, супервізор ranch_sup.

```
> :supervisor.which_children :n2o
[
  { {:ws, '/chat/ws/4'}, <0.985.0>, :worker, [:n2o_ws] },
  { {:ws, '/chat/ws/3'}, <0.984.0>, :worker, [:n2o_ws] },
  { {:ws, '/chat/ws/2'}, <0.983.0>, :worker, [:n2o_ws] },
  { {:ws, '/chat/ws/1'}, <0.982.0>, :worker, [:n2o_ws] },
  { {:mqtt, '/bpe/mqtt/4'}, <0.977.0>, :worker, [:n2o_mqtt] },
  { {:mqtt, '/bpe/mqtt/3'}, <0.976.0>, :worker, [:n2o_mqtt] },
  { {:mqtt, '/bpe/mqtt/2'}, <0.975.0>, :worker, [:n2o_mqtt] },
  { {:mqtt, '/bpe/mqtt/1'}, <0.974.0>, :worker, [:n2o_mqtt] },
  { {:caching, 'timer'}, <0.969.0>, :worker, [:n2o] }
]
```

11.4.1 Накопичувальні ресурси

Розподілені хеш-кільця використовуються не тільки для розподілених обчислень, але і для зберігання даних. Деякі бази даних, наприклад RocksDB та Cassandra, використовують глобальний простір ключів для даних (на відміну від таблично-орієнтованих баз). Саме для таких баз і створено библиотеку KVS, де в якості синхронного транзакційного інтерфейсу — API ланцюжків з гарантією консистентності. Нижче наведено приклад структури ланцюжків екземпляру системи PLM:

```
> :kvs.all :writer
[
  {:writer, '/bpe/proc', 2},
  {:writer, '/erp/group', 1},
  {:writer, '/erp/partners', 7},
  {:writer, '/acc/synrc/Kyiv', 3},
  {:writer, '/chat/5HT', 1},
  {:writer, '/bpe/hist/1562187187807717000', 8},
  {:writer, '/bpe/hist/1562192587632329000', 1}
]
```

В нашій моделі синхронні протоколи використовуються для управління накопичувальними ресурсами підприємства і транзакційного процесингу.

11.5 Типові специфікації

Протоколи визначаються типовими специфікаціями і генеруються для наступних мов: Java, Swift, JavaScript, Google Protobuf V3, ASN.1. Також ми генеруємо валідатори даних по цих типових анотаціях і вбудовуємо ці валідатори в тракт наших розподілених протоколів, тому ми ніколи не дозволимо клієнтам зіпсувати сторадж. Для веб додатків у нас развинута система валідації — як для JavaScript, так і на стороні сервера. Бізнес логіка повністю ізольована в нашій системі управління бізнес процесами, де кожен бізнес процесс є процесом віртуальної машини. Всі ланцюжки модифікуються атомарним чином, підтримують flake адресацію, і не вимагають додаткової ізоляції у своєму примітивному використанні. Тому ви можете трактувати базу як розподілений кеш і використовувати її з фронт додатків для примітивних випадків.

11.6 Середовище

Для забезпечення повного замкненого середовища пропонують наступні заміни бібліотек kernel та stdlib:

VM — віртуальна машина середовища виконання[2]
BASE — базова системна бібліотека як заміна stdlib[3]
RT — бібліотека середовища виконання як заміна kernel[4]
SYN — бібліотека PubSub для розподілених систем[5]
MAD — бібліотека управління пакетами та інстансами[6]

[2] vm.n2o.dev
[3] base.n2o.dev
[4] rt.n2o.dev
[5] syn.n2o.dev
[6] mad.n2o.dev

11.6.1 Бібліотеки

Для запезпечення повноцінної промислової специфікації ERP/1, ми розширили набір інструментальних засобів наступними бібліотеками: формальними представленнями презентаційного рівня FORM та системою управління бізнес-процесів BPE. FORM представляє собою декларативну бібліотеку побудови графічних інтерфейсів, а бібліотека BPE підтримує XML файли стандарту BPMN 2.0 та реалізує безпосередню інтерналізацію BPMN семантики у семантику віртуальної машини Erlang.

N2O — сервер протоколів для стандартів MQTT/WS/QUIC[7]
NITRO — UI веб-фреймворк Nitrogen[8]
KVS — бібліотека доступу до KV сховищ RocksDB[9]
FORM — бібліотека декларативного конструювання іформ[10]
BPE — сисема управління процесами стандарту BPMN 2.0[11]
RPC — бібліотека генерації SDK для мов JS, protobuf, Swift[12]

11.6.2 Приклади

Головні приклади фундації N2O.DEV присвячені наступним темам: MQTT та WebSocket чати для демонстрації веб-фреймворку NITRO, який працює як модуль N2O, приклад REST адаптер до бази даних KVS, та повністю чистий N2O додаток CHAT на основі бібліотеки SYN без використання NITRO:

SAMPLE — ідіоматичний приклад Nitrogen поверх WS[13]
REVIEW — ідіоматичний приклад Nitrogen поверх MQTT[14]
REST — бібліотека для побудови HTTP API[15]
CHAT — приклад системи доставки повідомлень[16]

[7] ws.n2o.dev
[8] nitro.n2o.dev
[9] kvs.n2o.dev
[10] form.n2o.dev
[11] bpe.n2o.dev
[12] rpc.n2o.dev
[13] sample.n2o.dev
[14] review.n2o.dev
[15] rest.n2o.dev
[16] chat.n2o.dev

11.7 Протоколи, схеми та мови їх опису

11.7.1 Мова опису протоколів ASN.1

11.7.2 Мова опису протоколів SOAP/XSD/XML

11.7.3 JSON валідатори draft-07 і JTD

11.8 Формати передачі даних

11.8.1 Бінарні формати ETF/BERT

11.8.2 Бінарні формати DER/BER/PER

11.8.3 Колоночний текстовий формат CSV/CSM

11.8.4 Текстові формати JSON і XML

11.9 Розробка Інтернет додатків

11.9.1 Erlang та сучасний веб

Erlang реалізує недосяжну мрію кожного обчислювального середовища для паралельної та узгодженої конкуретної обробки повідоблень. Так найбільш відомі бібліотеки акторів (Akka, Orleans), які реалізують основні примітиви: процесори та черги, копіюють модель акторів Erlang, зазвичай намагаються також реалізують додатково механізми перезапуску та супервізії процесів подібно до Erlang, проте тільки Erlang забезпечує soft real-time характеристики, завдяки керуванню латенсі з точністю до таймінгу команд віртуальної машини. А з виходом 24 версії в 2020 році, яка почала підтримувати JIT-компіляцію завдяки asmjit, продуктивність та чуттєвість віртуальної машини зрозла ще більше.

З формальної точки зору достатньо добре ізольоване середовище віртуальної машини Erlang не тільки забезпечує характерстики реального часу для SMP-планувальника легких зелених процесів, але і обмежує область видимості heap пам'яті виключно для процесів-власників, що унеможливлює вплив відмови певних процесів на глобальний стан віртуальної машини.

Erlang ідеально підходить для побудови високо-навантажених, простомасштабованих, подійно-орієнтованих, неблокуючих, надійних, постійнодоступних, високо-ефективних, швидких, безпечних та надійних систем обробки повідомлень та розподілених у просторі та часі систем.

11.9.2 DSL vs Шаблони

З технічної точки зору N2O успішно показує неперевершену досі якість DSL програмування, яку ви не зможете знайти в сучасних веб-фреймворках для мов Erlang та Elixir. За 7 років неперервної еволюції N2O ми переписали кожен з 700 рядків по 30 разів, якшо порахувати через коміти Github. Веб-фреймворк NITRO, сховище KVS, та BERT.JS кодування може забезпечити відображення в веб-браузері повноекранних вертикальних форм з усіма обчислюваними полями зі швидкістю 60 форм в секунду по веб-сокет каналу. А надзвичайно компакта JavaScript бібліотека-компаньйон вміщується в 4 MSS/MTU вікна — саме такий розмір мінімального веб-клієнта з BERT кодуванням, який повністю управляється зі сторони сервера.

N2O сервер та веб-фреймворк NITRO реалізують концепцію не тільки управління сесіями та каналами, але і усім стеком побудови додатків включаючи UI частину, як це відбувається у таких веб-фреймворках як Erlang Nitrogen, OCaml Ocsigen, Scala Lift, F# WebSharper, а завдяки таким розширенням як FORM та BPE ідеально підходять і для побудови автоматизованих CRM систем.

Це не означає, що за допомогою N2O ви не можете створювати більш класичні та архаїчні додатки у стилі DTL шаблонізаторів, або як це відбувається у таких фреймворках як PHP, ASP, JSP, Rails, тощо. Перші версії NITRO містили в прикладах використання Django Template Library (DTL),

проте задля чистоти стеку були прийнято не включати в N2O додаткові шаблонізатори крім NITRO DSL.

11.9.3 Історія

N2O сервер, а також NITRO веб-фреймворк були спроектовані як інструментільні засоби для створення промислових ERP модулів підприємства у складі відкритої платформи ERP/1. Напочатку, N2O був відгалужений, як оптимізована версія веб-фреймворку Nitrogen, створеного Расті Клопгаузом. Хотілося оптимізувати та вдосконалити мінімізований WebSocket-тракт, який не містить синхронного протоколу HTTP взагалі та дозволяє створювати повноцінні асинхронні веб-додатки реального часу. На ньому була створена система управління депозитами в національному банку ПриватБанк. Пізніше N2O був розділений на бібліотеку-фреймворк процесів та протоколів (власне N2O) та бібліотеку-веб-фреймворк NITRO. Бібліотеки N2O та NITRO також отримали можливість роботи не тільки через WebSocket але і через MQTT та через чисті TCP або UDP. Така оновлена версія 5.10 була впроваджена як ядро системи повідомлень для додатку NYNJA з відкритим open-source протоколом і саме ій присвячений друга версія підручника.

11.9.4 Інтерфейс NITRO

11.9.5 Сховище KVS

11.9.6 Логіка BPMN

11.9.7 Додатки MQTT та WebSocket

Розділ **12**

Генерація, валідація і верифікація

Комплексна інфраструктура рівня ERP вимагає бездоганного теоретичного фундаменту для забезпечення безвідмовної роботи алгоритмів. В рамках архітектурного підходу (відомого як *Volume V: Verification* або *Computational Layer*) ми визначаємо набір математичних та інженерних дисциплін. Програмне забезпечення повинно бути не лише емпірично протестованим, але й строго доведеним за допомогою формальних методів. Розділ закладає теоретичний фундамент, оглядаючи сучасний стан формальної верифікації та визначаючи місце пропонованої уніфікованої системи формальних мов у цій галузі.

Формальна верифікація як дисципліна бере початок із робіт Ніколаса де Брейна, чия система AUTOMATH (1967) стала першим механізмом комп'ютеризованого доведення теорем у фіброваній моделі. Хенк Барендрегт розвинув цю ідею, створивши лямбда-куб — класифікацію типових систем, де найвища вершина (Calculus of Constructions, CoC) уможливила вищі формальні мови. Ми використовуємо CoC як базис для теоретичного ядра, доповнюючи його вищими мовами для поглинання машинною верифікацією усієї доступної математики підприємства.

12.1 Формальна верифікація та валідація

Для унеможливлення помилок на виробництві застосовуються різні методи формальної верифікації. Формальна верифікація — доказ, або заперечення відповідності системи у відношенні до певної формальної специфікації або характеристики, із використанням формальних методів математики.

Згідно з міжнародними нормами (IEEE, ANSI)[1] та у відповідності до вимог Європейського Аерокосмічного Агенства[2], процес валідації включає в себе перегляд (code review), тестування (модульне, інтеграційне, властивостей), перевірку моделей, аудит — увесь комплекс необхідний для доведення, що продукт відповідає вимогам висунутим при розробці.

Огляд сучасних засобів формальної верифікації показує, що найкращі результати досягаються тоді, коли мова програмування сама слугує інстру-

[1] IEEE Std 1012-2016 — V&V Software verification and validation
[2] ESA PSS-05-10 1-1 1995 – Guide to software verification and validation

ментом математичного доведення. Класифікація цих підходів виділяє окрему *спектральну категорію мов формальної верифікації* (Dependently Typed Systems). Процес розподіляється на валідацію (чи правильну систему ми будуємо?) та верифікацію (чи правильно ми її побудували?).

12.2 Формальна специфікація

Для спрощення процесу верифікації та валідації застосовується математична техніка формалізації постановки задачі — формальна специфікація. Це математична модель, створена для опису систем, визначення їх основних властивостей, та інструментарій для перевірки цих систем.

Існують два фундаментальні підходи: 1) Алгебраїчний підхід, де система описується в термінах операцій та відношень між ними (аналітичний метод); 2) Модельно-орієнтований підхід, де модель створена конструктивними побудовами на базі теорії множин, а системні операції визначаються тим, як вони змінюють стан системи (синтетичний метод).

12.2.1 Програмне забезпечення та логіка

Найбільш стандартизована та прийнята в області формальної верификації — це нотація Z[3] (Spivey, 1992), приклад модельно-орієнтованої мови, названої на честь Ернеста Цермело.

Інша відома мова формальної специфікації як стандарт для моделювання розподілених систем, таких як телефонні мережі та протоколи, це LOTOS[4] як приклад алгебраїчного підходу. Ця мова побудована на темпоральних логіках та поведінках залежниих від спостережень. Інші мови специфікацій, які можна відзначити тут — це TLA+, CSP, CCS (Milner, 1971), Actor Model, BPMN, тощо.

12.2.2 Математичні компоненти

Перші системи комп'ютерної алгебри розроблялися ще під PDP-6: MATHLAB, MACSYMA, SCRATCHPAD, REDUCE, SACLIB, MUMATH. Сучасні системи включають AXIOM, MAGMA, MUPAD, GAP.

12.3 Формальні методи верифікації

Можна виділити три підходи до верифікації: 1) спеціалізовані верифікатори моделей, або системи моделювання (VST, NuPRL, TLA+, Twelf, SystemVerilog); 2) алгебраїчні мови для синтетичних моделей і глибокого вбудовування (Coq, Agda, Lean, F*, cubicaltt, RedPRL); 3) системи автоматичного доведення теорем і синтезу програм (HOL/Isabelle, ACL2).

[3] ISO/IEC 13568:2002 — Z formal specification notation
[4] ISO 8807:1989 — LOTOS

12.3.1 Алгебраїчні мови та System F

Вступ у теорію мов програмування неможливий без розуміння фундаменту типізації, побудованого на поліморфному лямбда-численні **System F**. Завдяки System F, програми автоматично гарантують відсутність цілих класів помилок періоду виконання. Компілятор розв'язує рівняння рівності типів через механізми уніфікації, працюючи у безпечній та математично обґрунтованій категорії.

12.4 Моделі процесів

Для моделювання складних бізнес-процесів (BPMN) та високонавантажених мережевих протоколів ми спираємося на математичну формалізацію, здатну автоматизувати генерування тестів. Для цього розроблено спеціалізовану мову специфікацій **Zen Crypted Dharma DSL (Z.180)**, яка слугує виконавчим доповненням до міжнародного стандарту **ITU-T Z.120 Message Sequence Chart (MSC)**.

Мова Z.180 абстрагує транспортний рівень та криптографічні кодування, природним чином маплячись на конструкти Z.120: сесії (Sessions) стають інстансами (lifelines), дії (Actions) — вихідними/вхідними повідомленнями, а експектації (Expectations/Predicates) — умовами перевірки. Завдяки цьому забезпечується строга верифікація міжшарових інваріантів (консистентність курсорів читання, механізми ABAC, модерації) без втручання в імутабельну історію подій. Цей підхід безпосередньо сумісний з TTCN-3 для генерації тест-сьютів і формальною семантикою процес-алгебр з Annex B. Процес генерації інтеграційних шлюзів та SDK на основі цих специфікацій стає рутинною механічною трансформацією з AST у клієнти Swift, Kotlin, Erlang, C# та JavaScript.

12.5 Формальні мови та середовища виконання

Усі середовища виконання можна умовно розділити на два класи: 1) інтерпретатори нетипізованого або просто типізованого лямбда числення (Erlang/BEAM, V8, HotSpot, Kx, PyPy); 2) безпосередня генерація інструкцій процессора і лінкування (OCaml, Rust, Haskell, Pony).

Найбільш цікаві цільові платформи для виконання программ які побудовані на основі формальних доведень для нас є OCaml (основна мова екстракту Coq), Rust (відсутність сміттєзбірника), Erlang (неблокована семантика -числення) та Pony.

12.5.1 Формальні інтерпретатори та екстракція

Рантайм може будуватися як інтерпретатор нетипізованої системи. У рамках розвитку проєкту використовується PTS тайпчекер Henk (на базі Erlang/OCaml) у якості проміжної мови для повної нормалізації лямбда термів. Тотальна програма виступає способом лінкування з підсистемою

вводу-виводу віртуальної машини Erlang, поєднуючи функціональну довершеність (CoC) з реальною неблокованою роботою.

12.6 Базова схема підприємства ERP/1

Генерація, валідація та верифікація зливаються в єдиний обчислювальний конвеєр для бази підприємства. Та встановлено, що усі системи доведення і середовища виконання є носіями мовних елементів, які згідно теорії типів Мартіна-Льофа можна кластеризувати так:

- — нетипизоване -числення Чорча;
- — числення процесів, CCS, CSP або -числення Мілнера;
- — тензорне числення та векторизація;
- — числення конструкцій (функціональна повнота);
- — числення контекстів (контекстуальна повнота);
- $_{=}$ — теорія типів Мартіна-Льофа (логіка);
- — числення індуктивних конструкцій (матіндукція);
- — гомотопічна система типів (формальна математика).

Усі бізнес-правила (BPMN/BPE), протоколи міжсервісної взаємодії та політики доступу (ABAC) проходять етап символьного виконання, тестування за моделлю MSC та типового розв’язання. Лише код, який успішно формує доведення власної консистентності у термінах цих вищих мов програмування, допускається до середовища виконання підприємства.

Розділ **13**

Інфраструктурний рівень безпеки інтернету

13.1 Електронний підпис і цифрова печатка

Кваліфікований Електронний Підпис, або Кваліфіклована Електронна Печатка — це набір стандартів криптографічного захисту ДСТУ 4145, та міжнародних стандартів які визначають його конверт: X.501, X.509, X.511, X.520.

Серія міжнародних стандартів X.500, групується по категоріям, кожна з яких має свій перелік ASN.1 файлів. Аби підключити усі визначення необхідні для КЕП використані наступі компоненти стандартів (виділені болдом): X.501 — BasicAccessControl [1], InformationFramework [2], UsefulDefinitions [3]; X.509 — SpkmGssTokens [4], PkiPmiExternalDataTypes [5], AttributeCertificateDefinitions [6],
AlgorithmObjectIdentifiers [7],
AuthenticationFramework [8], CertificateExtensions [9];
X.511 — SpkmGssTokens [10], DirectoryAbstractService [11]; X.520 — PasswordPolicy [12], UpperBounds [13], SelectedAttributeTypes [14].

Можно було би винести необхідні визначення одразу в KEP.asn1, однак цим хотілося підкреслити сумісність з міжнародними стандартами. Окрім серії протоколів X.500, КЕП ще визначає також запити та відповіді OCSP, також у ASN.1 форматі.

---

[1] https://www.itu.int/ITU-T/formal-language/itu-t/x/x501/2019/BasicAccessControl.html
[2] https://www.itu.int/ITU-T/formal-language/itu-t/x/x501/2019/InformationFramework.html
[3] https://www.itu.int/ITU-T/formal-language/itu-t/x/x501/2019/UsefulDefinitions.html
[4] https://www.itu.int/ITU-T/formal-language/itu-t/x/x509/2019/ExtensionAttributes.html
[5] https://www.itu.int/ITU-T/formal-language/itu-t/x/x509/2019/PkiPmiExternalDataTypes.html
[6] https://www.itu.int/ITU-T/formal-language/itu-t/x/x509/2019/AttributeCertificateDefinitions.html
[7] https://www.itu.int/ITU-T/formal-language/itu-t/x/x509/2019/AlgorithmObjectIdentifiers.html
[8] https://www.itu.int/ITU-T/formal-language/itu-t/x/x509/2019/AuthenticationFramework.html
[9] https://www.itu.int/ITU-T/formal-language/itu-t/x/x509/2019/CertificateExtensions.html
[10] https://www.itu.int/ITU-T/formal-language/itu-t/x/x511/2019/SpkmGssTokens.html
[11] https://www.itu.int/ITU-T/formal-language/itu-t/x/x511/2019/DirectoryAbstractService.html
[12] https://www.itu.int/ITU-T/formal-language/itu-t/x/x520/2019/PasswordPolicy.html
[13] https://www.itu.int/ITU-T/formal-language/itu-t/x/x520/2019/UpperBounds.html
[14] https://www.itu.int/ITU-T/formal-language/itu-t/x/x520/2019/SelectedAttributeTypes.html

На відміну від самого алгоритму КЕП, який визначено ДСТУ 4145, конверти визначаються не стандартами, а наказами міністерства юстиції: Проект наказу Адміністрації Держспецзв'язку та Держкомінфоматизації (2009) [15], Наказ Міністерства юстиції України 1236/5/453 [16]. Керуючись цими нормативними документами було створено файл KEP.asn1 [17], який є одним з трьох top-level файлів необхідниї для компіляції ASN.1 компілятором[18].

Існує небагато безкоштовних та повних компіляторів (генераторів парсерів) ASN.1 специфікацій. Erlang є прикладом системи, до складу якої входить першокласний безкоштовний з відкритою ліценцією ASN.1 компілятор, де файли в ASN.1 нотації можуть бути зкомпільовані безпосередньо Erlang компілятором:

```
> erlc AuthenticationFramework.asn1
> erlc InformationFramework.asn1
> erlc KEP.asn1
```

Створити файл підпису PKCS-7 можна за допомогою будь якої програми сертифікованої в Україні. Найпростіше отримати свою КЕП печатку будучи клієнтом ПриватБанку. За допомогою "Користувача ЦСК"компанії ІІТ ви можете підписувати файли використовуючи безкоштовну форму приватного ключа у вигляді звичайного файлу.

[15] http://www.dsszzi.gov.ua/dsszzi/control/uk/publish/article?art_id=77726
[16] https://zakon.rada.gov.ua/laws/show/z1401-12
[17] https://github.com/synrc/ca/blob/master/priv/kep/KEP.asn1
[18] https://asn1.erp.uno

13.1.1 Приклад використання

Щоб показати як користуватися КЕП, та прочитати атрибутивну інформацію з сертифікату, який вшитий в PCKS-7 повідомлення з криптографічним підписом, покажемо 5 функцій:

```
> CA.CAdES.readSignature
[
  {:certinfo, ~c"TINUA-2955020254",
   "СОХАЦЬКИЙ МАКСИМ ЕРОТЕЙОВИЧ",
   "МАКСИМ ЕРОТЕЙОВИЧ", "СОХАЦЬКИЙ",
   "СОХАЦЬКИЙ МАКСИМ ЕРОТЕЙОВИЧ",
   [
     subjectKeyIdentifier: "VNXfTvJQccGtPgNhUftIQZV+mUROTgroLotsbtYZsFE=",
     authorityKeyIdentifier: "XphNUm+C84/0vi5ABGgN/rOvysLkBHVNB9CuTISwfB0=",
     keyUsage: [<<6, 192>>],
     certificatePolicies: {"https://acsk.privatbank.ua/acskdoc",
      ["1.2.804.2.1.1.1.2.2", "1.3.6.1.5.5.7.2.1"]},
     basicConstraints: [],
     qcStatements: {"https://acsk.privatbank.ua",
      ["0.4.0.1862.1.1", "0.4.0.1862.1.5", "1.3.6.1.5.5.7.11.2",
       "0.4.0.194121.1.1", "1.2.804.2.1.1.1.2.1"]},
     cRLDistributionPoints: ["http://acsk.privatbank.ua/crl/PB-2023-S6.crl"],
     freshestCRL: ["http://acsk.privatbank.ua/crldelta/PB-Delta-2023-S6.crl"],
     authorityInfoAccess: [
       {"1.3.6.1.5.5.7.48.2",
        "http://acsk.privatbank.ua/arch/download/PB-2023.p7b"},
       {"1.3.6.1.5.5.7.48.1", "http://acsk.privatbank.ua/services/ocsp/"}
     ],
     subjectInfoAccess: [
       {"1.3.6.1.5.5.7.48.3", "http://acsk.privatbank.ua/services/tsp/"}
     ],
     subjectDirectoryAttributes: [
       {"1.2.804.2.1.1.1.11.1.4.7.1", "0"},
       {"1.2.804.2.1.1.1.11.1.4.1.1", "2955020254"}
     ]
   ], "ФІЗИЧНА ОСОБА", "", "", ~c"UA", "КИЇВ"},
  {:certinfo, ~c"UA-14360570-2310",
   "КНЕДП АЦСК АТ КБ ПРИВАТБАНК\"\"", "", "",
   "КНЕДП АЦСК АТ КБ ПРИВАТБАНК\"\"",
   [
     contentType: "0.6.9.42.840.113549.1.7.1",
     signingTime: "240221110356Z",
     messageDigest: "MfvlhoDVCPkptQRN+S2zNGp0nrOsS93mLdbcz/kZ9GI=",
     signingCertificateV2: 540041581425012649131508804155871837613877419268,
     contentTimestamp: {"1.2.840.113549.1.7.2",
      36995253346304402407284752111874897026, "20240221110626Z",
      "MfvlhoDVCPkptQRN+S2zNGp0nrOsS93mLdbcz/kZ9GI="}
   ], "АТ КБ ПРИВАТБАНК\"\"", "", "", ~c"UA", "Київ"}
]
```

13.2 Криптографічні інформаційні повідомлення

Реалізації повинні підтримувати транспортування ключів, узгодження ключів і раніше розподілені симетричні ключі шифрування ключів, представлені ktri, kari та kekri відповідно.

Реалізації можуть підтримувати керування ключами на основі пароля, представлене pwri. Реалізації МОЖУТЬ підтримувати будь-які інші методи керування ключами, такі як шифрування на основі ідентифікації Боне-Франкліна та Боне-Бойєна (RFC 5409) або інші методи шифрування SYNRC, такі як варіанти KYBER Key Transport (LAMPS-WG) для постквантової криптографії (PQC).

IETF (SMIME-WG) стандарти: 5990, 5911, 5750–5754, 5652, 5408, 5409, 5275, 5126, 5035, 4853, 4490, 4262, 4134, 4056, 4010, 3850, 3851, 3852, 3854, 3855, 3657, 3560, 3565, 3537, 3394, 3369, 3370, 3274, 3114, 3278, 3218, 3211, 3217, 3183, 3185, 3125–3126, 3058, 2984, 2876, 2785, 2630, 2631, 2632 , 2633, 5083, 5084, 2634.

Сумісність: Erlang SSL, LibreSSL CMS, OpenSSL CMS, GnuPG S/MIME.

13.2.1 Головна функція

Специфікація синтаксису криптографічних повідомлень CMS X.509 для дисципліни RSA (Key Transport), ECC (Key Agreement), KEK (Key Encryption Key) для додатків Erlang/OTP, які ніколи не переживала heartbleed (!) CRYPTO та SSL. Реалізовано як модуль CMS програми CA.

```
defmodule CMS do
    def decrypt(cms, {schemeOID, privateKeyBin}) do
        {_,{:ContentInfo,_,{:EnvelopedData,_,_,x,y,_}}} = cms
        {:EncryptedContentInfo,_,{_,encOID,{_,<<_::16,iv::binary>>}},data} = y
              case :proplists.get_value(:kari,  x, []) do
        [] -> case :proplists.get_value(:ktri,  x, []) do
        [] -> case :proplists.get_value(:kekri, x, []) do
        [] -> case :proplists.get_value(:pwri,  x, []) do
        [] -> {:error, "Unknown Other Recepient Info"}
              pwri  -> pwri(pwri,   privateKeyBin, encOID, data, iv) end
              kekri -> kekri(kekri, privateKeyBin, encOID, data, iv) end
              ktri  -> ktri(ktri,   privateKeyBin, encOID, data, iv) end
              kari  -> kari(kari,   privateKeyBin, schemeOID, encOID, data, iv)
     end
    end
end
```

13.2.2 CMS-KARI-ECC

IETF 3278:2002 Використання алгоритмів криптографії еліптичних кривих (ECC) у синтаксисі криптографічних повідомлень (CMS) із підтримкою Suite B IETF 5008:2007, 6318:2011.

```
# openssl cms -decrypt -in encrypted.txt -inkey client.key -recip client.pem
# openssl cms -encrypt -aes256 -in message.txt -out encrypted.txt \
                       -recip client.pem -keyopt ecdh_kdf_md:sha256
```

CMS Codec KARI: ECC+KDF/ECB+AES/KW+256/CBC:

```
def map(:'dhSinglePass-stdDH-sha512kdf-scheme'), do: :sha512
def map(:'dhSinglePass-stdDH-sha384kdf-scheme'), do: :sha384
def map(:'dhSinglePass-stdDH-sha256kdf-scheme'), do: :sha256
def eccCMS(ukm, bit), do:
    :'CMSECCAlgs-2009-02'.encode(:'ECC-CMS-SharedInfo', sharedInfo(ukm, bit))
def sharedInfo(ukm, len), do: {:'ECC-CMS-SharedInfo',
    {:'KeyWrapAlgorithm',{2,16,840,1,101,3,4,1,45},:asn1_NOVALUE}, ukm, <>}

def kari(kari, privateKeyBin, schemeOID, encOID, data, iv) do
    {_,:v3,{_,{_,_,publicKey}},ukm,{_,kdfOID,_},[{_,_,encryptedKey}]} = kari
    {scheme,_}  = CA.ALG.lookup(schemeOID)
    {kdf,_}     = CA.ALG.lookup(kdfOID)
    {enc,_}     = CA.ALG.lookup(encOID)
    sharedKey   = :crypto.compute_key(:ecdh,publicKey,privateKeyBin,scheme)
    {_,payload} = eccCMS(ukm, 256)
    derived     = KDF.derive(map(kdf), sharedKey, 32, payload)
    unwrap      = CA.AES.KW.unwrap(encryptedKey, derived)
    res         = CA.AES.decrypt(enc, data, unwrap, iv)
    {:ok, res}
end
def testDecryptECC(), do: CA.CMS.decrypt(testECC(), testPrivateKeyECC())

def testECC() do
    {:ok,base} = :file.read_file "priv/certs/encrypted.txt"
    [_,s] = :string.split base, "\n\n"
    x = :base64.decode s
    :'CryptographicMessageSyntax-2010'.decode(:ContentInfo, x)
end

def testPrivateKeyECC() do
    privateKey = :public_key.pem_entry_decode(pem("priv/certs/client.key"))
    {:'ECPrivateKey',_,privateKeyBin,{:namedCurve,schemeOID},_,_} = privateKey
    {schemeOID,privateKeyBin}
end
```

13.2.3 CMS-KEKRI-KEK

Інформація про одержувача ключа шифрування ключа, як визначено CMS IETF 5652:2009, 3852:2004, 3369:2002, 2630:1999.

```
# openssl cms -encrypt -secretkeyid 07 \
              -secretkey 0123456789ABCDEF0123456789ABCDEF \
              -aes256 -in message.txt -out encrypted2.txt

# openssl cms -decrypt -in encrypted2.txt -secretkeyid 07 \
              -secretkey 0123456789ABCDEF0123456789ABCDEF
```

CMS Codec KEKRI: KEK+AES-KW+CBC:

```
def kekri(kekri, privateKeyBin, encOID, data, iv) do
    {:'KEKRecipientInfo',_vsn,_,{_,kea,_},encryptedKey} = kekri
    _ = CA.ALG.lookup(kea)
    {enc,_} = CA.ALG.lookup(encOID)
    unwrap = CA.AES.KW.unwrap(encryptedKey,privateKeyBin)
    res = CA.AES.decrypt(enc, data, unwrap, iv)
    {:ok, res}
end
def testDecryptKEK(), do: CA.CMS.decrypt(testKEK(), testPrivateKeyKEK())

def testPrivateKeyKEK() do
    {:kek, :binary.decode_hex("0123456789ABCDEF0123456789ABCDEF")}
end

def testKEK() do
    {:ok,base} = :file.read_file "priv/certs/encrypted2.txt"
    [_,s] = :string.split base, "\n\n"
    x = :base64.decode s
    :'CryptographicMessageSyntax-2010'.decode(:ContentInfo, x)
end
```

13.2.4 CMS-KTRI-RSA

The very first CMS IETF 3852:1999:

```
# gpgsm --list-keys
# gpgsm --list-secret-keys
# gpgsm -r 0xD3C8F78A -e CNAME > cms.bin
# gpgsm -u 0xD3C8F78A -d cms.bin
# gpgsm --export-secret-key-p12 0xD3C8F78A > key.bin
# openssl pkcs12 -in key.bin -nokeys -out public.pem
# openssl pkcs12 -in key.bin -nocerts -nodes -out private.pem
```

CMS Codec KTRI: RSA+RSAES-OAEP:

```
def ktri(ktri, privateKeyBin, encOID, data, iv) do
    {:'KeyTransRecipientInfo',_vsn,_,{_,schemeOID,_},key} = ktri
    {:rsaEncryption,_} = CA.ALG.lookup schemeOID
    {enc,_} = CA.ALG.lookup(encOID)
    sessionKey = :public_key.decrypt_private(key, privateKeyBin)
    res = CA.AES.decrypt(enc, data, sessionKey, iv)
    {:ok, res}
end
def testDecryptRSA(), do: CA.CMS.decrypt(testRSA(), testPrivateKeyRSA())

def testPrivateKeyRSA() do
    {:ok,bin} = :file.read_file("priv/rsa-cms.key")
    pki = :public_key.pem_decode(bin)
    [{:PrivateKeyInfo,_,_}] = pki
    rsa = :public_key.pem_entry_decode(hd(pki))
    {:'RSAPrivateKey',:'two-prime',_n,_e,_d,_,_,_,_,_,_} = rsa
    {:rsaEncryption,rsa}
end

def testRSA() do
    {:ok,x} = :file.read_file "priv/rsa-cms.bin"
    :'CryptographicMessageSyntax-2010'.decode(:ContentInfo, x)
end
```

13.2.5 KDF

KDF (MD5: 128, SHA: 160—512) and HKDF (HMAC) Key Derive functions used in ECC CMS schemes as of NIST SP 800-108r1.

```
defmodule KDF do
    def hl(:md5),    do: 16
    def hl(:sha),    do: 20
    def hl(:sha224), do: 28
    def hl(:sha256), do: 32
    def hl(:sha384), do: 48
    def hl(:sha512), do: 64

    def derive(h, d, len, x) do
        :binary.part(:lists.foldr(fn i, a ->
            :crypto.hash(h, d <> <> <> x) <> a
        end, <<>>, :lists.seq(1,round(Float.ceil(len/hl(h))))), 0, len)
    end
end
```

13.2.6 AES-KW

AES Key Wrap function is applicable to keys of 128/192/256 bit using AES-ECB encoding as of RFC 5649:2009 Advanced Encryption Standard (AES) Key Wrap with Padding Algorithm.

```
-define(MSB64,      1/unsigned-big-integer-unit:64).
-define(DEFAULT_IV, << 16#A6A6A6A6A6A6A6A6:?MSB64 >>).

unwrap(CipherText, KEK) -> unwrap(CipherText, KEK, ?DEFAULT_IV).
unwrap(CipherText, KEK, IV)
        when (byte_size(CipherText) rem 8) =:= 0
        andalso (bit_size(KEK) =:= 128
            orelse bit_size(KEK) =:= 192
            orelse bit_size(KEK) =:= 256) ->
    BlockCount = (byte_size(CipherText) div 8) - 1,
    IVSize = byte_size(IV),
    case do_unwrap(CipherText, 5, BlockCount, KEK) of
        << IV:IVSize/binary, PlainText/binary >> ->
            PlainText;
        _ ->
            erlang:error({badarg, [CipherText, KEK, IV]})
    end.

codec(128) -> aes_128_ecb;
codec(192) -> aes_192_ecb;
codec(256) -> aes_256_ecb.

do_unwrap(Buffer, J, _BlockCount, _KEK) when J < 0 -> Buffer;
do_unwrap(Buffer, J, BlockCount, KEK) ->
    do_unwrap(do_unwrap(Buffer, J, BlockCount, BlockCount, KEK),
              J - 1, BlockCount, KEK).
do_unwrap(Buffer, _J, I, _BlockCount, _KEK) when I < 1 -> Buffer;
do_unwrap(<< A0:?MSB64, Rest/binary >>, J, I, BlockCount, KEK) ->
    HeadSize = (I - 1) * 8,
    << Head:HeadSize/binary, B0:8/binary, Tail/binary >> = Rest,
    Round = (BlockCount * J) + I,
    A1 = A0 bxor Round,
    Data = << A1:?MSB64, B0/binary >>,
    << A2:8/binary, B1/binary >>
        = crypto:crypto_one_time(codec(bit_size(KEK)),
          KEK, ?DEFAULT_IV, Data, [{encrypt,false}]),
    do_unwrap(<< A2/binary, Head/binary, B1/binary,
          Tail/binary >>, J, I - 1, BlockCount, KEK).
```

13.2.7 AES-256

All AES-256 flavours are implemented for a wide range of ECC Key Agreement schemes.

```
def decrypt(crypto_codec, data, key, iv \\ :crypto.strong_rand_bytes(16))
def decrypt(:'id-aes256-ECB',data,key,iv), do: decryptAES256ECB(data,key,iv)
def decrypt(:'id-aes256-CBC',data,key,iv), do: decryptAES256CBC(data,key,iv)
def decrypt(:'id-aes256-GCM',data,key,iv), do: decryptAES256GCM(data,key,iv)
def decrypt(:'id-aes256-CCM',data,key,iv), do: decryptAES256CCM(data,key,iv)
def test() do
  [
    check_SECP384R1_GCM256(),
    check_X25519_GCM256(),
    check_C2PNB368w1_GCM256(),
    check_BrainPoolP512t1_GCM256(),
    check_BrainPoolP512t1_GCM256(),
    check_SECT571_GCM256(),
    check_X448_GCM256(),
    check_X448_CBC256(),
    check_X448_ECB256(),
  ]
end
```

13.3 Імплементація CMP сервера у складі АЦСК

IETF follow up (PKIX): 7030, 6960, 6818, 6844, 6712, 6664, 6402, 6277, 6170, 6024, 6025, 5934, 5912–5914, 5877, 5816, 5755, 5756, 5758, 5697, 5636, 5480, 5272–5274, 5280, 5055, 5019, 4985, 4683, 4630, 4476, 4387, 4325, 4158, 4210, 4211, 4055, 4043, 3874, 3779, 3820, 3739, 3709, 3628, 3161, 3029, 2797, 2559, 2587, 3039, 3029, 2511, 2510.

Compatibility: OpenSSL, Cisco, Red Hat, Siemens, Nokia, IBM.

Ця стаття могла би називати «Як написати CMP сервер за 30 хвилин», але на відміну від попередньої статті про LDAP, ця вже покриває більше ніж тузінь ASN.1 файлів, добре шо ми вже познайомилися з CMS та LDAP бібіліотеками та їх ASN.1 файлами. В цій статті про CMP нас в основному цікавитимуть PKIXCMP-2009, PKIXCRMF-2009 та EnrollmentMessageSyntax-2009 для CMC.

```
CMS-AES-CCM-and-AES-GCM-2009.asn1
CMSAesRsaesOaep-2009.asn1
CMSECCAlgs-2009-02.asn1
CMSECDHAlgs-2017.asn1
CryptographicMessageSyntax-2009.asn1
CryptographicMessageSyntax-2010.asn1
CryptographicMessageSyntaxAlgorithms-2009.asn1
EnrollmentMessageSyntax-2009.asn1
PKCS-10.asn1
PKCS-7.asn1
PKIX1Explicit-2009.asn1
PKIX1Implicit-2009.asn1
PKIXAlgs-2009.asn1
PKIXCMP-2009.asn1
PKIXCRMF-2009.asn1
```

13.3.1 CSR

Отже починається написання CMP серверу з найголовнішої його функції: видачі сертифікату по PCKS-10 CSR реквесту. Схема наступна: Клієнт генерує приватний ключ, конвертує його в PEM файл, відсилає як P10CR повідомлення у складі payload PKIMessage, отримує відповідь CP, після чого клієнт шле ще одне повідомлення CERTCONF, після якого CMP сервер повинен відповисти PKICONF повідомленням.

```
def csr(user) do
     {ca_key, ca} = read_ca()
     priv = X509.PrivateKey.new_ec(:secp384r1)
     der = :public_key.der_encode(:ECPrivateKey, priv)
     pem = :public_key.pem_encode([{:ECPrivateKey, der, :not_encrypted}])
     :file.write_file(user <> ".key", pem)
     :io.format '~p~n', [priv]
     csr = X509.CSR.new(priv, "/C=UA/L=Kyiv/O=SYNRC/CN=" <> user,
        extension_request: [
           X509.Certificate.Extension.subject_alt_name(["n2o.dev"])])
     :io.format 'CSR: ~p~n', [csr]
     :file.write_file(user <> ".csr", X509.CSR.to_pem(csr))
     true = X509.CSR.valid?(csr)
     subject = X509.CSR.subject(csr)
     :io.format 'Subject ~p~n', [subject]
     :io.format 'CSR ~p~n', [csr]
     X509.Certificate.new(X509.CSR.public_key(csr), subject, ca, ca_key,
        extensions: [subject_alt_name:
          X509.Certificate.Extension.subject_alt_name(["n2o.dev", "erp.uno"])
     ])
     csr
 end
```

Перед початком роботи CMP сервера повинен бути згенерований рутовий CA сертифікат з приватним ключем, ці два файла ми зберігаємо на диск, і у всіх подальших операціях користуємося ними. Для генерації файлів використовуємо функцію CA.CSR.ca.

```
def ca() do
     ca_key = X509.PrivateKey.new_ec(:secp384r1)
     ca = X509.Certificate.self_signed(ca_key,
           "/C=UA/L=Kyiv/O=SYNRC/CN=CSR-CMP", template: :root_ca)
     der = :public_key.der_encode(:ECPrivateKey, ca_key)
     pem = :public_key.pem_encode([{:ECPrivateKey, der, :not_encrypted}])
     :file.write_file "ca.key", pem
     :file.write_file "ca.pem", X509.Certificate.to_pem(ca)
     {ca_key, ca}
 end

 def read_ca() do
     {:ok, ca_key_bin} = :file.read_file "ca.key"
     {:ok, ca_bin} = :file.read_file "ca.pem"
     {:ok, ca_key} = X509.PrivateKey.from_pem ca_key_bin
     {:ok, ca} = X509.Certificate.from_pem ca_bin
     {ca_key, ca}
 end
```

Для одноразовоїї генерації серверних сертифікатів які обсуговують клієнтські TLS сесії можна використати наступний код.

```
 def server(name) do
     {ca_key, ca} = read_ca()
     server_key = X509.PrivateKey.new_ec(:secp384r1)
       X509.Certificate.new(X509.PublicKey.derive(server_key),
          "/C=UA/L=Kyiv/O=SYNRC/CN=" <> name, ca, ca_key,
          extensions: [subject_alt_name:
             X509.Certificate.Extension.subject_alt_name(["n2o.dev", "erp.uno"
     ]) ])
 end
```

13.3.2 CMS

Детально сімейство протоколів і CMS кодування описано в окремій статті присвяченій CMS Compliance. CMS кодування використовується тільки для CMC сервера, тому ми це поки висвітлювати не будемо.

13.3.2.1 CMP/CSR/TCP

RFC 6712, 4210. Для початку напишемо простий PKIMessage сервер.

```
defmodule CA.CMP do
  @moduledoc "CA/CMP TCP server."
  require CA

  def start(), do: :erlang.spawn(fn -> listen(1829) end)
  def listen(port) do
      {:ok, socket} = :gen_tcp.listen(port,
        [:binary, {:packet, 0}, {:active, false}, {:reuseaddr, true}])
      accept(socket)
  end

  def accept(socket) do
      {:ok, fd} = :gen_tcp.accept(socket)
      :erlang.spawn(fn -> __MODULE__.loop(fd) end)
      accept(socket)
  end

  def loop(socket) do
      case :gen_tcp.recv(socket, 0) do
           {:ok, data} ->
               [headers,body] = :string.split data, "\r\n\r\n", :all
               {:ok,dec} = :'PKIXCMP-2009'.decode(:'PKIMessage', body)
               {:PKIMessage, header, body, code, _extra} = dec
               __MODULE__.message(socket, header, body, code)
               loop(socket)
          {:error, :closed} -> :exit
      end
  end
```

13.3.2.2 PKIMessage.protection

Розберемося з полем PKIMessage.protection, в якому зберігається результат PBKDF2 алгоритма. Майте на увазі шо OpenSSL за замовчування використовує 20-байтні ключі та HMAC/SHA-1 у якості MAC функції, хоча OWF в 500 ітераціях обчислюється за допомогою OWF функції SHA-256.

13.3.2.3 ANSWER

Оскільки CMP сервер повинен працювати по HTTP/1.0 згідно стандартів додаємо необхідні HTTP заголовки.

```
def answer(socket, header, body, code) do
     message = CA."PKIMessage"(header: header, body: body, protection: code)
     {:ok, bytes} = :'PKIXCMP-2009'.encode(:'PKIMessage', message)
     res =  "HTTP/1.0 200 OK\r\n"
         <> "Server: SYNRC CA/CMP\r\n"
         <> "Content-Type: application/pkixcmp\r\n\r\n"
         <> :erlang.iolist_to_binary(bytes)
     :gen_tcp.send(socket, res)
 end
```

13.3.2.4 P10CR/CP

Запускаємо сервер та генеруємо сертифікати CA та CSR користувача:

```
. iex -S mix
> CA.CSR.ca
> CA.CSR.csr "maxim"
```

Запускаємо клієнтський запит за допомогою OpenSSL:

```
# openssl cmp -cmd p10cr -server localhost:1829 \
#             -path . -srvcert ca.pem -ref cmptestp10cr \
#             -secret pass:0000 -certout . client.csr
```

Пишемо функцію видачі сертифікату:

```
def message(socket, header, {:p10cr, csr} = body, code) do
      {:PKIHeader, pvno, from, to, messageTime, protectionAlg,
         _senderKID, _recipKID, transactionID, senderNonce,
         _recipNonce, _freeText, _generalInfo} = header
      true = code == validateProtection(header, body, code)

      {ca_key, ca} = CA.CSR.read_ca()
      subject = X509.CSR.subject(csr)
      :io.format '~p~n',[subject]
      true = X509.CSR.valid?(CA.parseSubj(csr))
      cert = X509.Certificate.new(X509.CSR.public_key(csr),
         CA.CAdES.subj(subject), ca, ca_key,
         extensions: [subject_alt_name:
            X509.Certificate.Extension.subject_alt_name(["synrc.com"]) ])

      reply = CA."CertRepMessage"(response:
            [ CA."CertResponse"(certReqId: 0,
              certifiedKeyPair: CA."CertifiedKeyPair"(certOrEncCert:
                {:certificate, {:x509v3PKCert, CA.convertOTPtoPKIX(cert)}}),
              status: CA."PKIStatusInfo"(status: 0))])

      pkibody = {:cp, reply}
      pkiheader = CA."PKIHeader"(sender: to, recipient: from, pvno: pvno,
          recipNonce: senderNonce, transactionID: transactionID,
          protectionAlg: protectionAlg, messageTime: messageTime)
      answer(socket, pkiheader, pkibody,
          validateProtection(pkiheader, pkibody, code))
  end
```

13.3.2.5 CERTCONF/PKICONF

```
def message(socket, header, {:certConf, statuses}, code) do
      {:PKIHeader, _, from, to, _, _, _, _, _, senderNonce, _, _, _} = header

      :lists.map(fn {:CertStatus,bin,no,{:PKIStatusInfo, :accepted, _, _}} ->
          :logger.info 'CERTCONF ~p request ~p~n', [no,:binary.part(bin,0,8)]
      end, statuses)

      pkibody = {:pkiconf, :asn1_NOVALUE}
      pkiheader = CA."PKIHeader"(header, sender: to, recipient: from,
          recipNonce: senderNonce)
      answer(socket, pkiheader, pkibody,
          validateProtection(pkiheader, pkibody, code))
  end
```

В результаті в консолі повинні спостерігати:

```
CMP info: sending P10CR
CMP info: received CP
CMP info: sending CERTCONF
CMP info: received PKICONF
CMP info: received 1 enrolled certificate(s), saving to file 'maxim.pem'
```

13.3.2.6 GENM/GENP

Далі можете написати інші функції:

```
# openssl cmp -cmd genm -server 127.0.0.1:1829 \
#             -recipient "/CN=CMPserver" -ref 1234 -secret pass:0000


def message(_socket, _header, {:genm, req} = _body, _code) do
      :io.format 'generalMessage: ~p~n', [req]
  end
```

13.3.2.7 IR/IP

```
# openssl cmp -cmd ir -server 127.0.0.1:1829 \
#             -path . -srvcert ca.pem -ref NewUser \
#             -secret pass:0000 -certout maxim.pem \
#             -newkey maxim.key -subject "/CN=maxim/O=SYNRC/ST=Kyiv/C=UA"
```

```
 def message(_socket, _header, {:ir, req}, _) do
      :lists.map(fn {:CertReqMsg, req, sig, code} ->
         :io.format 'request: ~p~n', [req]
         :io.format 'signature: ~p~n', [sig]
         :io.format 'code: ~p~n', [code]
      end, req)
  end
```

13.3.2.8 CR/CP

```
# openssl cmp -cmd cr -server 127.0.0.1:1829 \
#             -path . -srvcert ca.pem -ref NewUser \
#             -secret pass:0000 -certout maxim.pem \
#             -newkey maxim.key -subject "/CN=maxim/O=SYNRC/ST=Kyiv/C=UA"
```

13.3.2.9 Висновки

```
defmodule CA do
  use Application
  use Supervisor

  require Record
  Enum.each(Record.extract_all(from_lib: "ca/include/PKIXCMP-2009.hrl"),
            fn {name, definition} -> Record.defrecord(name, definition) end)

  Enum.each(Record.extract_all(from_lib: "public_key/include/public_key.hrl"),
            fn {name, definition} -> Record.defrecord(name, definition) end)

  def init([]), do: {:ok, { {:one_for_one, 5, 10}, []} }
  def start(_type, _args) do
      :logger.add_handlers(:ldap)
      CA.CMP.start
      CA.CMC.start
      :supervisor.start_link({:local, __MODULE__}, __MODULE__, [])
  end
```

Ну PasswordBasedMac в нас є, тепер треба DHMac, але shared secret можна і в PBM засунути. Є ше Proof Of Posession (POP) — там зразу ECDSA verify. Я до речі думаю в CA тримати ключі для всіх кривих, і коли я виставлятиму сервіс то я буду виставляти його на N портах і N ключах, щоб будь який клієнтський TLS сертифікат приймався як рідний! Chat □ X.509 дає можливість вибирати TLS сертифікати автоматично по обраних кривих. Ви вибираєте під якими ключами сьогодні заходити. LDAP, MQTT, NS, CA — в кожного сервісу свої N портів і N серверних TLS сертифікатів. Передбачається що перший сертифікат видається DH по TCP а потім зразу всьо переходить в TLS режим і всі наступні сертифікати вже видаються всередині клієнтського TLS. При реєстрації користувач зразу доступний в LDAP якшо захотів зробити себе відкритим для пошуку. Після реєстрації пошук в директорії і френдування (обмін ключами) і поїхали чат в обгортках CAdES, CMS, ECDSA/AES — лейби біля повідомлень ПІДПИС/ШИФР.

13.3.3 CA, АЦСК, ЦЗО та ОЗО

Акредитований центр сертифікації ключів (АЦСК) в українській термінології та Certificate Authority (CA) в міжнародній виконують спільну задачу — управління життєвим циклом інфраструктури відкритих ключів. В Україні розбудова цієї інфраструктури спирається на Центральний засвідчувальний орган (ЦЗО), який відповідає за адміністрування кореневого сертифіката держави, та Окремі засвідчувальні органи (ОЗО), що є підлеглими сертифікаційними вузлами. ЦЗО формує довіру на національному рівні та інтегрується з міжнародними реєстрами, тоді як ОЗО забезпечують функціонування конкретних галузевих доменів.

Для забезпечення цих процесів розроблено рішення `synrc/ca` — компактний та потужний засвідчувальний центр розміром усього 2000 рядків коду (*Pure Elixir*). Він комплексно покриває потреби інфраструктури:

- **Підтримка PKI-сутностей:** CA, RA, SERVER, CLIENT, HUMAN, PROGRAM.

- **Адаптація державних стандартів:** Повна підтримка операцій у поліноміальному базисі Галуа (2) для криптографії ДСТУ 4145-2002.
- **Широкий спектр еліптичних кривих та схем:** `secp256k1`, `secp384r1`, `secp521r1` та підтримка гібридних механізмів Діффі-Геллмана (`RSA`, (), (2)).
- **Документи розширеної ідентифікації (EUID):** TAXID (РНОКПП), PID (паспорт), IBAN, HIID, LOYAL.
- **Мережева взаємодія:** Обслуговування запитів за протоколами CMP (TCP 8829), EST (HTTP 8047) та CMC (TCP 5318) згідно з RFC.

13.4 Безпечна система доменних імен DNSSEC

Безпека маршрутизації та ідентифікації ресурсів в інтернеті критично залежить від цілісності доменної системи імен. Протокол DNSSEC (Domain Name System Security Extensions) додає криптографічні підписи до записів DNS, що унеможливлює підміну IP-адрес (cache poisoning). В інфраструктурі державних інформаційних систем впровадження DNSSEC є обов'язковим для захисту урядових доменів (`gov.ua`), гарантуючи, що звернення громадян до криптографічних сервісів є автентичними та надійно захищеними від атак перехоплення (Man-In-The-Middle).

13.5 Захищений месенджер SYNRC CHAT

Рішення `synrc/chat` демонструє застосування інфраструктурних стандартів безпеки для створення захищеного корпоративного месенджера. Він базується на імплементації протоколу *Zen Crypted Buddha* та глибоко інтегрований з екосистемою Erlang/OTP (`Mnesia` для маршрутизації та зберігання повідомлень).

Головною відмінністю SYNRC CHAT є наскрізне використання сертифікації:

- Використання X.509 CMS Envelopes для управління ключами та захисту вмісту.
- Імплементація ASN.1 визначених протоколів та DER бінарної серіалізації поверх класичних транспортів.
- Підтримка сучасних криптографічних примітивів: X25519, X448, SECP384r1.
- Вбудована інтеграція з CA через протоколи CMP та EST для автоматичного отримання та оновлення сертифікатів клієнтів за допомогою еліптичної криптографії.

За замовчуванням кожен пакет у месенджері підписується (Sign/Verify) та шифрується (Encrypt/Decrypt), що перетворює його на інструмент з рівнем гарантії безпеки (*High Assurance*), придатний для систем розвідки та оперативного управління.

Так як якісну безпечну розподілену інфраструктуру яка відповідає міжнародним і українським стандартами неможливо створити без CA/CMS, OCSP, TSP, LDAP, DNS/DNSSEC, MQTT серверів, то всі ці сервери є фундаментом продуктів SYNRC які формують перший рівень фреймворку Сохацького який умовно називається Security або Безпека Підприємства. На цьому фреймворку побудовані головні інфраструктурні елементи країни, а також реєстрові системи.

Загалом, LDAP це дуже древня і надійна технологія яка присутня буквально у всіх топових компаніях, корпораціях, великих і малих бізнесах. Так, є сучасній Identity Server продукти (Hashicorp) які не підтримуються LDAP, але це поодинокі непопулярні маргінальні екземпляри. SYNRC LDAP це гарний початок для малих, середніх і великих бізнесів навести порядок в штатних розкладах, календарях, задачах, ресурсах підприємства, таких як автопарки, IoT, реєстрах персональних і промислових комп'ютерів, портів, правил ABAC, правил маршрутизації, контактних книгах користувачів, одним словом все для чого створений і де використовується LDAP.

13.6 Система директорії підприємства LDAP

Пройшло 13 років з того часу як компанія SYNRC випустила була свій власний брендований LDAP Directory Server на Erlang з підтримкою MongoDB.

Взагалі баз які підтримують префіксний і суфіксний пошук по B-Tree таблицям не так багато, в основному це складні SQL та MongoDB. Хоча такі бази як Mnesia, LMDB, BDB-похідні (RocksDB, LevelDB) немають повнотекстового пошуку, це можна реалізувати шляхом повного траверса, що на цих базах зазвичай працює швидко (якшо на C), і ше швидше якшо база підтримує mmap (LMDB).

Непереможний по перформенсу станом на зараз є OpenLDAP (LMDB), однак нам хочеться мати свою імплементацію, яку можна було би використовувати як фірмове сховище даних для директорії ресурсів підприємства згідно як міжнародних стандартів так і стандартів України. Ми захотіли відмовитися від зовнішньої бази даних (MongoDB) що ускладнює розробку, і хотілося вибрати шось вбудовуване для Erlang, вибирали між zambal/elmdb та elixir-sqlite/exqlite, переміг SQLite тому шо хотілося мати мінімальну ідіоматичну імплементацію, така шоб з одного боку була зрозуміла навіть людям без глибокої освіти в Computer Science, а з іншого боку відкривала двері в підтримку інших промислових корпоративних SQL джерел даних (Oracle, T-SQL, PostgreSQL).

На відміну від попередньої версії SYNRC LDAP яка робилася під стартап PEOPLE|SYNC який ми хотіли продати PEOPLE|NET, а потім Київстар, як SyncML стартап по синхронізації контактних книг, ця версія робиться для стартапа SYNRC CHAT для розвідки. В цій версії ми значну увагу приділили сумісністю з клієнтами, такими як Apache Directory Studio, а також усіма LDIF файлами які ми змогли знайти в інтернеті. Сучасна версія `synrc/ldap` реалізована всього у 300 рядках коду на Elixir, але при цьому забезпечує абсолютну сумісність із низкою фундаментальних IETF RFC (2849, 3296, 3671-3673, 4510-4518, 5480). Архітектура підтримує гаряче перемикання бекендів зберігання даних: SQLITE, LMDB та MONGODB, задовольняючи як вимоги до вбудовуваних рішень (in-memory SQLite), так і високонавантажених кластерів (LMDB).

13.6.1 Вертикальні бази

Я вперше познайомився з вертикальними базами коли працював в International Land Systems, Inc. Тоді в нас була система документообігу на вертикальній базі, які дуже часто використовуються в системах документообігу. Наприклад таку схему даних використовує Alfresco, а також SQL розширення для зберігання XML у Oracle та інших SQL базах.

Основна ідея вертикальних баз, або схем, полягає в тому шо об'єкти зберігаються не у плоских таблицях де кожен атрибут це окрема колонка, а всі атрибути та їх значення зберігаються в трьох колонках, де перша — це номер об'єкта, друга — ім'я атрибута, а третя — значення атрибуту. Це дозволяє тримати розріжені (атрибутами) об'єкти, та певним чином спростити управління базою. Всі наївні імплементації вертикальних баз страждаються по перформенсу, тому потрібно бути обрежним. Наприклад, ми не рекомендуємо використовувати SYNRC LDAP де об'єм директоріє більше ніж пів

мільйона співробітників, наприклад для Walmart. Для великих корпорацій краще брати OpenLDAP.

13.6.2 Предметна область

13.6.2.1 Netscape, Sun DS, 389 DS, Oracle

PEOPLE|SYNC стартап SYNRC 2007 року підтримував також роботу з Sun Directory Server. Його родовід бере початок від сервера OpenLDAP, в 1996 року стартував форк Netscape Directory Server. Після банкротства Netscape право на код викупила компанія AOL, яка ліцензувала право на розробку компанії Sun Microsystems, зберігши право на код. В 2009 році Sun DS був переіменований на 389 Directory Server, а Oracle почала свій форк Sun DS під назвою Oracle Directory Server. 389 Directory Server також можна зустріти під іменами Fedora DS та Red Hat DS, оскільки це основні донори проекту.

13.6.2.2 Microsoft Active Directory

Традиційно кожна корпорація займається розробкою свого LDAP сервера, компанія Microsoft випустила свій перший в 1999 році. Active Directory у якості бекенда використовує Extensible Storage Engine ESENT.DLL, також відомий як JetDB, на яцій побудований також Windows Registry, Microsoft Access, та можливо і інші внутрішні продукти компанії Microsoft.

13.6.2.3 OpenLDAP, Apple Open Directory

Були часи, що OpenLDAP був вбудований в кожну версію Mac OS X, але Apple почала розвивати свій власний Open Directory Server, оскільки безпека кожної корпорації полягає у тому числі в брендованих LDAP серверах під свої потреби та політику розробки..

13.6.3 TCP сервер

Я неодноразово використовував написання LDAP серверу у своїх Erlang курсах, а також на конференціях у якості майстер-класу по програмуванню, де ми з аудиторією пишемо всі разом LDAP сервер за 45 хвилин з моїми коментарями та інтеракцією. Рекорд на відео був поставлений 30 хвилин, так шо це не прікол, я дійсню MVP всіх продуктів SYNRC можу написати за 30 хвилин кожен. Власне це є одним з критеріїв SYNRC, що тісно переплетено з показником LOC. Ця версія SYNRC LDAP з підтримкою SQLite займає 300 рядків коду і проходить всі LDIF тест сюїти.

Перед початком поставте Erlang та його Erlang AST фронтенд Elixir. Ставити можна завжди тільки Elixir, Erlang піде як залежність в будь-якому пекедж менеджері.

```
# apt install elixir
```

Створюємо папку проекту і в ній створюємо файл mix.exs для уніфікованого депенденсі і пекадж менеджера Erlang і Elixir мов, mix.

Крім класичної прелюдії mix.exs, нам цікавий параметр exqlite, це ім'я бібліотеки пакетного менеджера hex.pm, яка містить в собі 8-мегабайтний Сі файл, і FFI обгортку для нього яка в Erlang світі називається NIF.

```
defmodule LDAP.Mixfile do
  use Mix.Project
  def project(), do:
    [ app: :ldap, version: "8.7.20", deps: deps(),
      releases: [ ldap: [ include_executables_for: [:unix],
                          cookie: "SYNRC:LDAP" ] ] ]
  def application(), do:
    [ mod: {LDAP, []},
      extra_applications: [ :eldap, :asn1 ] ] end
  def deps(), do:
    [ {:exqlite, "~> 0.13.14"} ]
end
```

Далі пишемо найпростіший ідіоматичний Erlang TCP сервер. Довгий час я використовував класичні, розширені версії на недокументованій функції prim , 5/, , , , .!

Якшо в config/config.exs немає параметра ldap:intance то створюється нова SQLite база по рендомному хешу з налаштуваннями по перформансу: 1) відключений журнал, 2) ін-меморі буфер, 3) великий кеш, 4) примусова синхронність; які визначаються відповідними SQL прагмами.

Архітектура TCP сервера відповідає POSIX, ми створюємо лістенер, який на кожне вхідне TCP повідомлення стартує акцептори, які стартують некотрольований Erlang процес — лупер, який обслуговує вхідне TCP повідомлення. Для декодування використовується згенерований ASN.1 компілятором енкодер/декодер LDAP протоколу по файлу LDAP.ans1, який можна знайти прямо в RFC IETF на нормативно-правових актах України.

```
# erlc LADP.asn1
```

Після геренації покладіть файли в LDAP.erl та LDAP.hrl в папку src проекту. А в папці lib створіть обгортку для згенерованих рекордів за допомогою Record,

```
defmodule DS do
    require Record
    Enum.each(Record.extract_all(from_lib: "ldap/include/LDAP.hrl"),
             fn {name, definition} -> Record.defrecord(name, definition) end)
end
```

а також створіть козу Erlang аплікейшина в Elixir синтаксисі, файл ldap.ex:

```
defmodule LDAP do
   import Exqlite.Sqlite3
   require DS
   use Application
   use Supervisor
   def code(), do: :binary.encode_hex(:crypto.strong_rand_bytes(8))
   def init([]), do: {:ok, { {:one_for_one, 5, 10}, []} }
   def start(_, _) do
       :logger.add_handlers(:ldap)
       :supervisor.start_link({:local, LDAP}, LDAP, [])
   end
   def initDB(path) do
       {:ok, conn} = open(path)
```

```
        :logger.info 'SYNRC LDAP Instance: ~p', [path]
        :logger.info 'SYNRC LDAP Connection: ~p', [conn]
        execute(conn, "create table ldap (rdn text,att text,val binary)")
        :ok = execute(conn, "PRAGMA journal_mode = OFF;")
        :ok = execute(conn, "PRAGMA temp_store = MEMORY;")
        :ok = execute(conn, "PRAGMA cache_size = 1000000;")
        :ok = execute(conn, "PRAGMA synchronous = 0;")
        conn
    end
    def listen(port,path) do
        conn = initDB(path)
        {:ok, socket} = :gen_tcp.listen(port,
          [:binary, {:packet, 0}, {:active, false}, {:reuseaddr, true}])
        accept(socket,conn)
    end
    def accept(socket,conn) do
        {:ok, fd} = :gen_tcp.accept(socket)
        :erlang.spawn(fn -> loop(fd, conn) end)
        accept(socket,conn)
    end

    def start() do
        :erlang.spawn(fn ->
            listen(:application.get_env(:ldap,:port,1489),
                   :application.get_env(:ldap,:instance,code())) end)
    end
    def answer(response, no, op, socket) do
        message = DS."LDAPMessage"(messageID: no, protocolOp: {op, response})
        {:ok, bytes} = :'LDAP'.encode(:'LDAPMessage', message)
        send = :gen_tcp.send(socket, :erlang.iolist_to_binary(bytes))
    end
    def loop(socket, db) do
        case :gen_tcp.recv(socket, 0) do
             {:ok, data} ->
                  case :'LDAP'.decode(:'LDAPMessage',data) do
                       {:ok,decoded} ->
                           {:'LDAPMessage', no, payload, _} = decoded
                           message(no, socket, payload, db)
                           loop(socket, db)
                       {:error,reason} ->
                          :logger.error 'ERROR: ~p', [reason]
                         :exit
                  end
             {:error, :closed} -> :exit
        end
    end
end
```

Цей сервер вже може відповідати на запити ldapmodify але буде їх блокувати так як поки що не відповідає належним чином. Запустити програму можна класичними мантрами Elixir, а в Elixir Shell виконати функцію запуску TCP лістенера, для цього перевпевніться що дефаултний порт 1389 вільний.

```
# mix deps.get
# iex -S mix
> LDAP.start
#PID<0.311.0>
iex(2)>
04:58:26.030 [info]  SYNRC LDAP Instance: "416C4C41ED2C7060"
04:58:26.030 [info]  SYNRC LDAP Connection: #Reference<
     0.1146704550.396492828.212314>
iex(3)>
nil
```

13.6.3.1 BIND

Для удачного демо я раджу починати з функції BIND. Для цього створимо в базі записи по яким будемо аутентифікуватися.

```
createDN(conn, "dc=synrc,dc=com",
    [ attr("dc",["synrc"]), attr("objectClass",["top","domain"]) ])
createDN(conn, "ou=schema",
    [ attr("ou",["schema"]), attr("objectClass",["top","domain"]) ])
createDN(conn, "cn=tonpa,dc=synrc,dc=com",
    [ attr("cn",["tonpa"]),attr("uid",["1000"]),
      attr("objectClass",["inetOrgPerson","posixAccount"]) ])
createDN(conn, "cn=rocco,dc=synrc,dc=com",
    [ attr("cn",["rocco"]),attr("uid",["1001"]),
      attr("objectClass",["inetOrgPerson","posixAccount"]) ])
createDN(conn, "cn=admin,dc=synrc,dc=com",
    [ attr("rootpw",["secret"]), attr("cn",["admin"]),
      attr("objectClass",["inetOrgPerson"]) ])

def appendNotEmpty([]),  do: []
def appendNotEmpty(res) do
    res ++ case res do [] -> [] ; _ -> ',' end
end
def createDN(db, dn, attributes) do
    norm  = :lists.foldr(fn {:PartialAttribute, att, vals}, acc ->
            :lists.map(fn val -> [qdn(dn),att,val] end, vals) ++
                                  acc end, [], attributes)
    {_,p} = :lists.foldr(fn x, {acc,res} ->
             {acc + length(x), appendNotEmpty(res)
              ++ :io_lib.format('(?~p,?~p,?~p)', [acc+1,acc+2,acc+3])}
            end, {0,[]}, norm)
    {:ok, statement} = prepare(db,
          'insert into ldap (rdn,att,val) values ' ++ p ++ '')
    :ok = bind(db, statement, :lists.flatten(norm))
    :done = step(db, statement)
end
def message(no, socket, {:bindRequest, {_,_,bindDN,{:simple, password}}}, db)
   do
    sql = "select rdn, att from ldap where " <>
          "rdn = ?1 and att = 'rootpw' and val = ?2"
    {:ok, statement} = prepare db, sql
    bind(db, statement, [hash(qdn(bindDN)),password])
    case step(db, statement) do
        :done ->  code = :invalidCredentials
                  :logger.error 'BIND Error: ~p', [code]
                  response = DS."BindResponse"(resultCode: code,
                      matchedDN: "", diagnosticMessage: 'ERROR')
                  answer(response, no, :bindResponse, socket)
         {:row,[dn,password]} ->
                  :logger.info 'BIND DN: ~p', [bindDN]
                  response = DS."BindResponse"(resultCode: :success,
                       matchedDN: "", diagnosticMessage: 'OK')
                  answer(response, no, :bindResponse, socket)
    end
end
def message(no, socket, {:bindRequest, {_,_,bindDN,creds}}, db) do
    code = :authMethodNotSupported
    :logger.info 'BIND ERROR: ~p', [code]
    response = DS."BindResponse"(resultCode: code,
       matchedDN: "", diagnosticMessage: 'ERROR')
    answer(response, no, :bindResponse, socket)
end
```

13.6.3.2 ADD

```
def message(no, socket, {:addRequest, {_,dn, attributes}}, db) do
      {:ok, statement} = prepare(db, "select rdn, att, val from ldap where rdn =
       ?1")
      bind(db, statement, [hash(qdn(dn))])
      case step(db, statement) do
            {:row, _} ->
                :logger.info 'ADD ERROR: ~p', [dn]
                resp = DS.'LDAPResult'(resultCode: :entryAlreadyExists,
                       matchedDN: dn, diagnosticMessage: 'ERROR')
                answer(resp, no, :addResponse, socket)
            :done ->
                createDN(db, dn, attributes)
                :logger.info 'ADD DN: ~p', [dn]
                resp = DS.'LDAPResult'(resultCode: :success,
                       matchedDN: dn, diagnosticMessage: 'OK')
                answer(resp, no, :addResponse, socket)
       end
   end
```

13.6.3.3 DSE

Якщо тестувати наш прото-сервер не ldapmodify, а Apache Directory Studio, то вона початково буде питати так званий Root DSE об'єкт запитуючи після bind, search реквест з пустим DN. Для коректної репрезентації спеціалізованої інформації ми прошиємо стандартну відповідь на цей запит для нашого фірмового SYNRC LDAP сервера версії 2.0 який підтримує протокол LDAPv3. Він підтримує SIMPLE спосіб аунтентифікації тільки (поки що) і два іменних простори ключів: dc=synrc,dc=com ou=schema, як виманає декілька RFC. Це всьо пакується у LDAPMessage і відправляється на клієнт функцією answer.

```
def attr(k,v),      do: {:PartialAttribute, k, v}
   def node(dn,attrs), do: {:SearchResultEntry, dn, attrs}

   def message(no, socket,
       {:searchRequest, {_,"",scope,_,limit,_,_,filter,attributes}}, db) do

       :logger.info 'DSE Scope: ~p', [scope]
       :logger.info 'DSE Filter: ~p', [filter]
       :logger.info 'DSE Attr: ~p', [attributes]

       :lists.map(fn response -> answer(response,no,:searchResEntry,socket) end,
         [ node("", [
              attr("supportedLDAPVersion", ['3']),
              attr("namingContexts", ['dc=synrc,dc=com','ou=schema']),
              attr("supportedControl", ['1.3.6.1.4.1.4203.1.10.1']),
              attr("supportedExtensions", ['1.3.6.1.4.1.4203.1.11.3']),
              attr("altServer", ['ldap.synrc.com']),
              attr("subschemaSubentry", ['ou=schema']),
              attr("vendorName", ['SYNRC LDAP']),
              attr("vendorVersion", ['2.0']),
              attr("supportedSASLMechanisms", ['SIMPLE']),
              attr("objectClass", ['top','extensibleObject']),
              attr("entryUUID", [code()])]),
              attr("supportedFeatures", [ '1.3.6.1.1.14',
                                          '1.3.6.1.4.1.4203.1.5.1'])
         ])

       resp = DS.'LDAPResult'(resultCode: :success, matchedDN: "",
     diagnosticMessage: 'OK')
       answer(resp, no, :searchResDone,socket)
   end
```

Після цього зможуть розгортати в інтерфейсі дерево об'єктів.

13.6.3.4 MODIFY

```
def modifyDN(db, dn, attributes), do:
    :lists.map(fn {_, :add, x}     -> modifyAdd(db,dn,x)
                  {_, :replace, x} -> modifyReplace(db,dn,x)
                  {_, :delete, x}  -> modifyDelete(db,dn,x) end, attributes)

def modifyAdd(db, dn, {_,att,[val]}) do
    {:ok, st} = prepare(db, "insert into ldap (rdn,att,val) values (?1,?2,?3)
   ")
    :logger.info 'MOD ADD RDN: ~p', [hash(qdn(dn))]
    bind(db, st, [hash(qdn(dn)),att,val])
    step(db,st)
end

def modifyReplace(db, dn, {_,att,[val]}) do
    {:ok, st} = prepare(db, "update ldap set val = ?1 where rdn = ?2 and att
   = ?3")
    :logger.info 'MOD REPLACE RDN: ~p', [hash(qdn(dn))]
    bind(db, st, [val,hash(qdn(dn)),att])
    step(db,st)
end

def modifyDelete(db, dn, {_,att,_}) do
    {:ok, st} = prepare(db, "delete from ldap where rdn = ?1 and att = ?2")
    :logger.info 'MOD DEL RDN: ~p', [hash(qdn(dn))]
    bind(db, st, [hash(qdn(dn)),att])
    res = step(db,st)
    collect0(db,st,res,[])
end

def message(no, socket, {:modifyRequest, {_,dn, attributes}}, db) do
   {:ok, statement} = prepare(db, "select rdn, att, val from ldap where rdn =
    ?1")
   bind(db, statement, [hash(qdn(dn))])
   case step(db, statement) do
        {:row, _} -> :logger.info 'MOD DN: ~p', [dn]
                     modifyDN(db, dn, attributes)
                     resp = DS.'LDAPResult'(resultCode: :success,
                         matchedDN: dn, diagnosticMessage: 'OK')
                     answer(resp, no, :modifyResponse, socket)
        :done ->     :logger.info 'MOD ERROR: ~p', [dn]
                     resp = DS.'LDAPResult'(resultCode: :noSuchObject,
                        matchedDN: dn, diagnosticMessage: 'ERROR')
                     answer(resp, no, :modifyResponse, socket)
   end
end
```

13.6.3.5 MODIFY DN

```
def modifyRDN(socket, no, db, dn, new, del) do
    {:ok, st} = prepare(db, "update ldap set rdn = ?1 where rdn = ?2")
    :logger.info 'MODIFY RDN UPDATE: ~p', [hash(qdn(dn))]
    bind(db, st, [new,hash(qdn(dn))])
    step(db,st)
end

def message(no, socket, {:modDNRequest, {_,dn,new,del,_}}, db) do
    :logger.info 'MOD RDN DN: ~p', [dn]
    :logger.info 'MOD RDN newRDN: ~p', [new]
    :logger.info 'MOD RDN deleteOldRDN: ~p', [del]
    modifyRDN(socket, no, db, dn, new, del)
    resp = DS.'LDAPResult'(resultCode: :success,
        matchedDN: dn, diagnosticMessage: 'OK')
    answer(resp, no, :modDNResponse, socket)
end
```

13.6.3.6 DELETE

```
def deleteDN(db, dn) do
    {:ok, st} = prepare(db, "delete from ldap where rdn = ?1")
    bind(db, st, [hash(qdn(dn))])
    res = step(db,st)
    collect0(db,st,res,[])
end

def message(no, socket, {:delRequest, dn}, db) do
    :logger.info 'DEL DN: ~p', [dn]
    deleteDN(db, dn)
    resp = DS.'LDAPResult'(resultCode: :success, matchedDN: dn,
  diagnosticMessage: 'OK')
    answer(resp, no, :delResponse, socket)
end
```

13.6.3.7 SEARCH

Операція `search` є фундаментальною базою для читання та вибірки даних у директорії LDAP. Вона підтримує складні фільтри, які визначаються рекурсивним типом `Filter`. У цій системі запит парситься та транслюється у внутрішнє представлення, яке у свою чергу генерує оптимізовані SQL-запити. Підтримуються всі стандартні оператори фільтрації: `present`, `equalityMatch`, `substrings`, а також логічні оператори `AND`, `OR` та `NOT`, що дозволяє дуже гнучко шукати записи у корпоративному дереві та делегувати швидкодію індексам підлеглої реляційної СКБД.

13.6.3.8 COMPARE

```
def message(no, socket, {:compareRequest, {_,dn, assertion}}, db) do
    :logger.info 'CMP DN: ~p', [dn]
    :logger.info 'CMP Assertion: ~p', [assertion]
    result = compareDN(db, db, assertion)
    resp = DS.'LDAPResult'(resultCode: :success, matchedDN: dn,
  diagnosticMessage: 'OK')
    answer(resp, no, :compareResponse, socket)
end
```

13.6.3.9 ABANDON/UNBIND

```
def message(no, socket, {:abandonRequest, _}, db), do: :gen_tcp.close(socket)
def message(no, socket, {:unbindRequest,  _}, db), do: :gen_tcp.close(socket)
```

13.6.4 Висновки

У цій статті ми переконалися, що можливо написати LDAP сервер на 300 рядків, а також що SQLite підходить для малих підприємств у якості сховища даних. Ми взяли повний контроль над продуктом, який не містить залежностей, що функціонують за межами контексту віртуальної машини, а також спростили процес розробки більш складних систем на базі цього продукту. Продукт буде корисний для апробації в підприємства зі стандартизованими та уніфікованими політиками управління ресурсами підприємствах, в телекомунікаційних продуктах, комунікаторах, месенджерах, тощо.

13.7 ASN.1 Компілятор

Питання вибору серіалізатора залежить від типу інфраструктури в компанії. Якшо ми говоримо про гомогенні інтерфейси та бібліотеки прикладного програмування то зазвичай вони достатньо розвинені аби автоматично генерувати структури для основних мов програмування для яких анонсується серіалізатор. Хоча формально йдеться про AST трансформацію з мови визначення типів структур в цільову мову програмування, такі системи називаються компіляторами мови визначення даних.

У світі існує багато рантаймів, і кожен з них пропонує свій рідний серіалізатор, який в незмінному вигляді представляє дані які циркулюють в рантаймі або віртуальній машині. Так на мовах .NET, Java, Haskell, Erlang є такі природні серіалазотори. В гомогенних архітектурах, де все побудовано навколо однії мови програмування, прийнято використовувати цей серіалізатор як основний для всіх сервісів написаних навіть на різних мовах програмування, так як BERT-RPC протокол який використовувався в Github.

Окремо виділяються формати які не прив'язані до однієї мови, а пропонуються як універсальні серіалізатори, такі як IDL COM/DCOM, ASN.1 DER, Erlang BERT/ETF, GRPC Gproto, SOAP XML/WBXML, та інші бінарні мови і серіалізатори.

Всі сертифікати в браузері, SSH ключі, PGP/GPG ключі, закриті і відкриті конверти, криптографічні повідомлення CMS, протоколи видачі сертифікатів, LDAP директорія та ще багато іншого включачи GSM/LTE та майже всі телекомунікаційні повідомлення визначаються за допомогою ASN.1.

13.7.1 Компілятори

Я хотів зробити огляд ASN.1 компіляторів, але з представлених безкоштовних жоден крім Erlang-ового не компілює повний набір ASN.1 файлів проекта SYNRC CA. Ні C-шний кацапський який використовується в Apple ASN1C, Ні F-повий ASN1SCC. Залишається сподіватися, що генератор C-шного коду Фабріса Белара ASN1CC зможе це зробити однак з огляду на його клієнтів навряд чи ми про це дізнаємося, так шоб прямо розказати в блозі.

Я подивився на помилки представлених на сайті ITU компіляторів і складається враження що сумісність з самим ж файлами дефініціями ITU ніхто не перевіряє навіть мануально. Просто вивішують в залікову таблицю будь-кого хто успішно розпарсав сабсет ASN.1 і згенерував якийсь граничні імплементація для свого випадку.

Тестовий набір файлів

Ось текстовий сет з сайту ITU яким я перевіряв компілятори.

```
AESKeyWrapWithPad-02.asn1
AESKeyWrapWithPad-88.asn1
ANSI-X9-42.asn1
ANSI-X9-62.asn1
AlgorithmInformation-2009.asn1
AttributeCertificateVersion1-2009.asn1
AuthenticationFramework.asn1
BasicAccessControl.asn1
```

```
CHAT.asn1
CMS-AES-CCM-and-AES-GCM-2009.asn1
CMSAesRsaesOaep-2009.asn1
CMSECCAlgs-2009-02.asn1
CMSECDHAlgs-2017.asn1
CertificateExtensions.asn1
Character-Coding-Attributes.asn1
Character-Presentation-Attributes.asn1
Character-Profile-Attributes.asn1
Colour-Attributes.asn1
CryptographicMessageSyntax-2009.asn1
CryptographicMessageSyntax-2010.asn1
CryptographicMessageSyntaxAlgorithms-2009.asn1
DOR-definition.asn1
Default-Value-Lists.asn1
DirectoryAbstractService.asn1
Document-Profile-Descriptor.asn1
EnrollmentMessageSyntax-2009.asn1
ExtendedSecurityServices-2009.asn1
External-References.asn1
Geo-Gr-Coding-Attributes.asn1
Geo-Gr-Presentation-Attributes.asn1
Geo-Gr-Profile-Attributes.asn1
ISO-STANDARD-9541-FONT-ATTRIBUTE-SET.asn1
ISO9541-SN.asn1
Identifiers-and-Expressions.asn1
InformationFramework.asn1
KEP.asn1
LDAP.asn1
Layout-Descriptors.asn1
Link-Descriptors.asn1
Location-Expressions.asn1
Logical-Descriptors.asn1
MultipleSignatures-2010.asn1
OCSP.asn1
PKCS-10.asn1
PKCS-12.asn1
PKCS-5.asn1
PKCS-7.asn1
PKCS-8.asn1
PKCS-9.asn1
PKIX-CommonTypes-2009.asn1
PKIX-X400Address-2009.asn1
PKIX1-PSS-OAEP-Algorithms-2009.asn1
PKIX1Explicit-2009.asn1
PKIX1Explicit88.asn1
PKIX1Implicit-2009.asn1
PKIX1Implicit88.asn1
PKIXAlgs-2009.asn1
PKIXAttributeCertificate-2009.asn1
PKIXCMP-2009.asn1
PKIXCRMF-2009.asn1
Raster-Gr-Coding-Attributes.asn1
Raster-Gr-Presentation-Attributes.asn1
Raster-Gr-Profile-Attributes.asn1
SMIMESymmetricKeyDistribution-2009.asn1
SecureMimeMessageV3dot1-2009.asn1
```

13.7.2 Фірмові і стандартні

Тут розглядаються фірмова бібліотека RPC та стандартна ASN1X компанії SYNRC для бінарної серіалізації внутрішнього бінарного представлення віртуальної машини Erlang та бінарного представлення сімейства розміточних форматів і їх парсерів BER, DER, PER, кодери і декодери яких генеруються з мови специфікацій та протоколів ASN.1.

13.7.2.1 Ericsson Erlang BERT/ETF

Серед фірмових можна відзначити Ericsson Binary Erlang Term Format, який використовується в промисловості більше 10 років починаючи з Github. SYNRC використовує бібліотеку SYNRC RPC яка генерує кодери і декодери (API SDK) для будь яких Erlang структур на наступні мови програмування та мови визначення протоколів:

1) IDL/COM;
2) JavaScript;
3) Google gproto;
4) Apple Swift;
5) Erlang validation.

Завдяки генератору BERT парсерів SYNRC RPC компанія NYNJA змогла уніфіковано працювати з об'єктами системи в двох форматах BERT і GPROTO і залишилася після апробації на BERT.

13.7.2.2 ITU IETF ASN.1 BER/DER/PER Apple/Microsoft/OpenSSL

До стандартних можна віднести бінарні формати для серіалізації сертифікатів та обгорток криптографічних ключів які використовуються в PKI X.509. Завдяки промисловому і повному компілятору ASN.1 у складі Erlang/OTP ми маємо змогу побудувати повністю API SDK інфраструктуру на Erlang мінімальними зусиллями не гублячи сумісність з фірмовим форматом BERT/ETF, так як він все одно використовується в середині віртуальної машини після конвертації. Так була продемонстрована сумісність протоколів CHAT BERT (HRL) та результату ASN.1 компіляції CHAT BER.

1) ASN1C;
2) ASN1CC;
3) ASN1SCC;
4) ASN1SCG (Swift Code Generation);
5) ASN1JCG (Java Code Generation).

В даній статті розказується про один з двох генераторів коду DER/BER/PER кодерів і декодерів SYNRC для мови Swift — ASN1SCG, який генерує код невідмінний від схеми кодування і декодування яку Apple пропонує в свої бібліотеках asn1, crypto та certificates.

13.7.2.3 Google gproto/GRPC

В першу чергу цей формат набув популярності на пристроях Apple, так як протокол основного AP сховища Apple Cassandra використовує цей формат. Значною проблему є переускладеність кодогенератора і інфраструктури компілятора.

13.7.3 Бібліотеки Apple

Починаючи з вести 2020 року, коли Apple анонсувала CryptoKit для X.509 інфраструктури зокрема, компанія випустила наступні бібліотеки, які показується стандартний механізм який Apple передбачає для Swift авторів як вони мусять користуватися PKI X.509 обгортками, сертифікатами, ключами та взагалі будь-якими ASN.1 протоколами.

1) apple/swift-asn1
2) apple/swift-crypto
3) apple/swift-certificates

SYNRC ASN1SCG — це ASN.1 компілятор в мову Swift з використанням схеми кодування Apple, написаний на мові Erlang компанією SYNRC. Тут показуються приклади генерації кодерів і декодерів для деяких структур сімейства PKI X.509 в форматі прикладів Apple (насправді відрізнити конкретно ці неможливо).

13.7.4 Техніка компіляції

Оскільки мова ASN.1 має чіткий синтаксис побудова її компілятора може базуватися на BNF парсер генераторах таких як lexx/yacc або їх аналонах на інших мовах програмування leex/yexx (Erlang), Citron (Swift), ANTLR (Java). Загалом нас цікавитимуть LL, LR, LALR, SLR. Але найшвидші парсери це потокові бінарні, Erlang-овий саме такий і займає 2000 рядків разом з токенайзером на 100 рядків.

Технічно, ASN.1 компілятори є AST транформаціями з вихідної мови в цільову, в даному випадку з Erlang шляхом Elixir в Swift. Основне завдання при цьому побудувати базовий розміточний парсер у вигляді мікробібліотеки, яку буде використовувати як хелпери згенерований код. В нашому випадку такою бібліотекою є swift-asn1 від Apple тому це теж нам полегшує завдання. Таким чином наш згенерований код сумісний з усіма екстеншинами з бібліотек Apple.

13.7.4.1 Розміточний парсер

В нас в ЗОШ №5 вчився математик Сергій Книш, який закінчив МГУ в 19 і поїхав викладати в Каліфорнію. Я коли був в Сан-Франціско заходив до нього в гості і він розказува таку байку, а він на байки багатий бо він писав мережевий протокол IPX для шкільного комплексу Корветів і БК-0010 шоб

можна було в іграшки по мережі гратися і міг в пам'яті сумувати ряди з точністю до всіх розрядів калькулятора. Так от він розказував шо коли ше тільки починався C++, то таблицю віртуальних методів використовували як масив який індексувався байткодом операції, і коли ви читали байт зі стріма ви простоо викликали функцію з цим індексом в таблиці віртуальниxx методів для десеріалізації цього типу даних. Хоча це байка але в ній є зерно істини, розрізати десеріалізатор і серіалізатор на примітивні функції, кожна з яких займається своїм байт-кодом — техніка, що притаманна багатьом бібліотечним парсерам на мовах програмування C++, Rust, Swift, Java, тощо. Приблизно така сама архітектура і у розміточного TLV парсера Apple.

13.7.4.2 Послідовності та множини

Типи `SEQUENCE` та `SET` відіграють роль структур. При їх розборі компілятор генерує код, який обробляє кожен елемент об'єкта або строго по черзі (для `SEQUENCE`), або у довільному порядку (для `SET`). Враховуючи опціональність тегів, десеріалізатор повинен гнучко опрацьовувати пропущені поля, спираючись на контекстно-залежні теги.

13.7.4.3 Рекурсивні суми та їх теги

Перше шо ви повинні зробити перед тим як публікувати ASN.1 компілятор на сайті ITU.INT це пересвідчитися шо він сприймає класичне визначення рекурсивного списку з книжки Олів'є Дебюсона [6].

13.7.4.4 Імплісіти, експлісіти та опшинали

Якщо ви хочете позначити поле додатковою інформацією, такою як можливість приймати значення відсутності (`OPTIONAL`) та вид тегування (`IMPLICIT` або `EXPLICIT`), необхідно сформувати правильний контекст розбору. Такі теги модифікують оригінальний універсальний тег об'єкта, вимагаючи від парсера правильної маршрутизації бат-коду під час десеріалізації потоку.

13.7.4.5 Переліки та цілочисельні переліки

Типи `ENUMERATED` на макрорівні зіставляються зі стандартними алгебраїчними типами в мові програмування. Компілятор гарантує, що значення, закодоване як ціле число, перетворюється на строгу типізацію цільової мови (наприклад, `enum` у Swift або Erlang атоми).

13.7.4.6 Тензори

Масиви даних (тензори) в моделі ASN.1 кодуються як `SEQUENCE OF` або `SET OF`. Компілятор перетворює їх у списки (List) або масиви (Array). Оскільки тип може містити довільну кількість елементів, десеріалізація вимагає циклічного читання структури до досягнення маркеру кінця інформаційного потоку.

13.7.5 Висновки

Найкращий спосіб вивчити ASN.1 — це написати ASN.1 компілятор. Був детально проаналізований X.680 стандарт та знайдені конкуретні переваги над іншими компіляторами: 1) більшість не підтримують рекурсивні типи даних, 2) більшість не підримують повністю всі ASN.1 файли ITU.INT, 3) більшість не підтримують тензори. Наш компілятор зроблений таким, що усуває ці недоліки. Що стосується швидкості парсерів Apple для мови Swift, то вони в середньому в два рази повільніші за fast $++21$.

Реалізація використовує потоковий бінарний парсер ASN.1 файлів на 2000 рядків коду разом з токенайзером, що йдуть в дистрибутиві Erlang/OTP в аплікейшині asn1, а сам компілятор виконаний як script файл для мови Elixir на 600 рядків і не потребує компіляції. Найближчі конкуренти asn1c і asn1icc займають 25000 рядків.

Доведення коректності компілятора має комбінаторний вигляд, а головна теорема стверджує шо за 2 проходи, можна повністю скомпілювати ASN.1 X.680 стандарт в будь яку мову програмування, кожен прохід містить доведення коректності 11 циклів, разом які використовують 50 розгалужень кожне з яких доводиться окремо в комбінаторному стилі кейс аналізом. За перший прохід будують часткові форвард декларації, які повністю стають насиченими на другому проході. В процесі першого проходу не зберігаються файли, зате між проходами не очищується контекст, який містить три типи ключів: визначення типів для носіння полів та їх властивостей, синоніми, та специфікатори тензорів. Також компілятор містить verbose опцію -v яка показує всі стадії насичення контексту.

На розробку компілятора був витрачений 1 людино-місяць та згенеровано ним 30000 рядків Swift 5.8 коду який працює на Windows/Linux/Mac. Цей код покриває всі конверти які можна знайти на сайті ITU.INT у вигляді ASN.1 файлів. Сюди входять PKIX, CMP, CMC, LDAP, CMS, PKCS, X.400, тощо. Компілятор asn1scc написаний на F який підтримується Європейським Космічним Агенством не працює на повній множині всіх цих декларацій, а займає теж немало, 32000 рядків на F.

13.8 Протокол розмежування доступу ABAC

Модель контестного або атрибутного доступу (Attribute-Based Access Control, ABAC) забезпечує гнучке розмежування повноважень. На відміну від класичної рольової моделі (RBAC), де доступ фіксується виключно за роллю користувача, ABAC використовує атрибути суб'єкта, об'єкта, операції та глобального середовища для динамічного прийняття рішень під час виконання запиту.

Реалізація `erpuno/abac` повністю відповідає міжнародному стандарту **ISO/IEC 29146:2016** (Information technology — Security techniques — A framework for access management) та рекомендаціям **NIST SP 800-162**. Архітектура бібліотеки структурно розподіляє відповідальність між базовими компонентами системи.

Запит на доступ формалізується структурою `request`, що містить `subject` (ідентифікатор користувача, сервісу або ERP Employee) та `endpoint` (API-метод або дія: VIEW, EDIT, SIGN тощо).

13.8.1 PEP (Policy Enforcement Point)

Точка забезпечення виконання (PEP, в рамках `ABAC.API`) — це програмний інтерфейс, який перехоплює запит користувача на доступ до захищеного ресурсу. Його головна мета — призупинити виконання бізнес-операції та викликати центр прийняття рішень для отримання дозволу. PEP діє як суворий шлюз, блокуючи дію суб'єкта за відсутності позитивного вердикту.

13.8.2 PIP (Policy Information Point)

Точка інформації про політики (PIP) — це системний інтерфейс для отримання додаткових даних (`ABAC.PIP`). Коли бракує атрибутів для перевірки політики (наприклад, вік користувача, геопозиція чи посада), ініціюється звернення до PIP. Він автоматично збагачує запит атрибутами з поточного контексту екосистеми (модулі ERP, BPMN-двигун BPE, реляційні бази даних або директорії LDAP).

13.8.3 PAP (Policy Administration Point)

Точка адміністрування (PAP) — компонент, що забезпечує створення, зберігання та аудит самих політик безпеки. У вітчизняному стеку для безвідмовного та високошвидкісного збереження правил доступу використовується масштабовна база даних ключ-значення `KVS`.

13.8.4 PDP (Policy Decision Point)

Точка прийняття рішень (PDP, в рамках `ABAC.PDP`) — математичне ядро моделі. У цей компонент завантажені всі доступні в організації політики доступу. Коли надходить запит від PEP, PDP агрегує атрибути, обчислює логічні предикати та виносить обґрунтований фінальний вердикт (Permit або Deny) щодо кожної поточної транзакції.

Розділ **14**

Апробація

ЄРЗ (Єдиний реєстр зброї НПУ), СУСЗЦЗ (Система управління силами та засобами цивільного захисту ДСНС), ЄІС (Єдина інформаційна система МВС), ФП МТРЗ (Функціональна підсистема матеріально-технічного та ресурсного забезпечення МВС), ГСЦ (Головний сервісний центр МВС).

Розділ **15**

Висновки

Підбиваючи підсумки нашого дослідження та практичного досвіду впровадження національних інформаційних систем, неможливо оминути ключовий фактор, який забезпечив беззаперечний успіх наших наймасштабніших проєктів — це свідомий і технологічно вивірений вибір платформи. У той час як більшість інших державних відомств та міністерств продовжують йти консервативним шляхом, обираючи громіздкі екосистеми на базі CLR (.NET) та JVM (Java Virtual Machine), ми з неймовірною бадьорістю, ентузіазмом і професійним запалом розгорнули ключові системи держави на фундаменті промислових телекомунікаційних технологій Ericsson Erlang/OTP.

Цей архітектурний вибір був продиктований суворими вимогами до відмовостійкості. Технології Erlang/OTP, створені для безперервної роботи в умовах пікових навантажень, дозволили нам у рекордно короткі терміни та з мінімальними операційними витратами імплементувати такі колосальні системи, як:

- **Депозити АТ КБ «ПриватБанк»** — серцевина найбільшого банку країни, що бездоганно і стабільно гарантує консистентність мільйонів фінансових транзакцій і збереження активів нації;

- **СЕД «МІА: Документообіг» для МВС України** — надійна і криптографічно захищена інформаційна система, яка забезпечила безперебійний обіг мільйонів організаційно-розпорядчих документів та кримінальних проваджень у найбільшому силовому відомстві;

- **СЕД «ERP/1: Документи» для МО України** — надійна і криптографічно захищена інформаційна система, яка забезпечила безперебійний обіг мільйонів організаційно-розпорядчих документів та кримінальних проваджень у найбільшому силовому відомстві;

- **ЕСОЗ для НСЗУ та МОЗ** — центральний компонент медичної реформи європейського зразка, що витримує високоінтенсивну обробку чутливих медичних даних мільйонів громадян України в режимі реального часу.

Контраст результатів вийшов вичерпним і очевидним. Замість того, щоб щороку змушувати державу витрачати сотні мільйонів та колосальні ресурси на нескінченні міграції, ліцензійні відрахування та підтримку актуальних версій і фреймворків (для латання архітектурних дір важких enterprise-рішень на базі Java чи C#), проєкти на Erlang/OTP продемонстрували, якою елегантною, економною і граційною може бути надскладна національна IT-інфраструктура. Завдяки ізоляції процесів, вбудованій філософії уникнення збоїв («let it crash») та унікальній механіці гарячого оновлення коду («hot code swapping») без зупинки роботи, ці системи здатні функціонувати роками без жодних суттєвих перезавантажень, заощаджуючи величезні кошти платників податків.

Ми довели на світовій практиці, що створення електронної держави — це не обов'язково бюрократичний довгобуд із постійним роздуванням бюджетів і штатів програмістів. З правильною архітектурною візією та надійними функціональними інструментами, цей процес перетворюється на швидке, веселе і драйвове інженерне мистецтво високого ґатунку, здатне вивести управління державними та банківськими активами на безпрецедентний рівень якості.